

Local-to-Global Software

Categories, Presheaves, Sheaves, Topoi, and Cohomology through SessionStream

A custom study text for the SessionStream project

15 August 2026

Contents

Preface	1
The pedagogical contract	1
Repository baseline	1
The SessionStream study in one page	2
A map from Goldblatt to this book	2
Notation used from the beginning	3
Part I - Universal Structure	4
1 Reading Software as Diagrams	5
1.1 Why begin with arrows?	5
1.2 A first category of typed transformations	5
1.3 Commuting diagrams as implementation-independent equations	6
1.4 Objects versus instances	6
1.5 Monic, epic, and isomorphic software maps	7
1.6 Isomorphism is not equality	7
1.7 Worked study: event identity	8
1.8 Exercises	8
1.9 Goldblatt bridge	8
2 Limits as Coherent Observation	9
2.1 The engineering problem behind a limit	9
2.2 Terminal objects: the context that carries no choice	9
2.3 Products: independent observations held together	10
2.4 Equalizers: the subspace where two computations agree	10
2.5 Pullbacks: joining observations through shared meaning	11
2.5.1 A SessionStream pullback	11
2.6 Consistent snapshots as a limiting cone	11
2.7 The universal property as API quality criterion	12
2.8 Worked study: atomic application and progress	12
2.9 Exercises	13
2.10 Goldblatt bridge	13
3 General Limits, Colimits, and Architectural Duality	14
3.1 From named constructions to one pattern	14
3.2 Recovering familiar limits from shapes	15
3.3 Limits in a poset: greatest coherent lower bounds	15
3.4 Completeness is a capability, not a default	15
3.5 Colimits: universal assembly from pieces	16
3.6 Coproducts and typed event alternatives	16
3.7 Coequalizers and semantic quotienting	17
3.8 Pushouts and integration boundaries	17
3.9 Limits, colimits, and event sourcing	17

3.10	A practical duality table	18
3.11	Exercises	18
3.12	Goldblatt bridge	18
4	SessionStream as a System of Compatible Views	19
4.1	A semantic model before an implementation model	19
4.2	The current implementation surfaces	20
4.3	The principal coordinates	20
4.4	Type and schema invariants	21
4.5	Replay coherence	21
4.6	Snapshot soundness and completeness	22
4.7	Snapshot-before-live as a gluing protocol	22
4.8	The boundary object at the cut	22
4.9	Projection progress as a visibility invariant	23
4.10	Heartbeat suspicion as local evidence	23
4.11	A consistency matrix	23
4.12	Exercises	24
4.13	Milestone	24
	Part II - Functorial Architecture	25
5	Functors as Lawful Architectural Translations	26
5.1	Why a projection is not automatically a functor	26
5.2	The prefix category of a session	26
5.3	Chunking invariance	27
5.4	Functors between schema categories	28
5.5	Forgetful functors	28
5.6	Faithful and full functors	28
5.7	Projection as an action or algebra	29
5.8	Contravariant functors as observation	29
5.9	A current SessionStream functor candidate	29
5.10	Exercises	30
5.11	Goldblatt bridge	30
6	Natural Transformations as Coherent Refactoring	31
6.1	Comparing whole translations at once	31
6.2	Projection-version migration	31
6.3	Schema evolution and event migrations	32
6.4	Instrumentation that should be natural	32
6.5	Natural isomorphism and representation independence	33
6.6	Dinaturality, laxness, and the reality of distributed systems	33
6.7	The functor category	33
6.8	Worked study: live and hydrated client paths	34
6.9	Exercises	34
6.10	Goldblatt bridge	34
7	Adjunctions, Free Histories, and Repair	35
7.1	The pattern of optimal translation	35
7.2	Free monoids and event histories	35
7.3	Unit and counit as executable maps	36
7.4	A projection as interpretation of a free history	36
7.5	Reflection: repairing an object into a subcategory	37
7.6	Sheafification preview	37
7.7	Left and right adjoints preserve different structure	37
7.8	Adjunctions versus codec pairs	38
7.9	Galois connections for parameter sufficiency	38

7.10	Exercises	38
7.11	Goldblatt bridge	39
Part III - Local-to-Global Data		40
8	The Category of Observation Contexts	41
8.1	The base space is made of questions	41
8.2	Inclusion of contexts	41
8.3	Several possible time geometries	42
8.3.1	Prefix geometry	42
8.3.2	Interval geometry	42
8.3.3	Causal geometry	42
8.3.4	Observer-time product geometry	42
8.4	Contexts as regions of observability	43
8.5	Overlaps	43
8.6	Covers	43
8.7	The topology is an engineering assumption	44
8.8	A richer context record	44
8.9	Context design mistakes	44
8.9.1	Mistake 1: contexts are implementation modules	44
8.9.2	Mistake 2: every field defines a dimension	45
8.9.3	Mistake 3: later time always contains earlier state	45
8.9.4	Mistake 4: overlap means same field name	45
8.9.5	Mistake 5: global means centralized	45
8.10	Exercises	45
8.11	Mental image checkpoint	45
9	Presheaves: Data That Restricts	46
9.1	Definition from the software operation	46
9.2	The presheaf of parameter assignments	46
9.3	The presheaf of valid assignments	47
9.4	Fibres and parameter sufficiency	47
9.5	The event-history presheaf	48
9.6	The snapshot-history presheaf	48
9.7	The presheaf of trace observations	48
9.8	The presheaf of local invariants	49
9.9	Compatible families	49
9.10	Global sections and completions	49
9.11	Separated presheaves	50
9.12	Hidden coordinates as presheaf defects	50
9.13	Exercises	50
9.14	Goldblatt bridge	51
10	Bundles, Fibres, Germs, and Sections	52
10.1	Goldblatt's geometric entry point	52
10.2	A SessionStream bundle over sessions	52
10.3	A bundle over session-cut pairs	53
10.4	Fibres over observer contexts	53
10.5	Local sections	54
10.6	Germs as local behavior near a point	54
10.7	Display spaces and the "shape above the base"	54
10.8	Sections and dependency injection	55
10.9	Sections as generalized elements	55
10.10	Exercises	55
10.11	Goldblatt bridge	56

11 Sheaves: Compatible Data That Glues	57
11.1 The missing axiom	57
11.2 The event-interval sheaf	57
11.3 Snapshot and suffix as a sheaf-like cover	58
11.4 How the WebSocket buffer realizes gluing	58
11.5 Existence failures	59
11.5.1 Global constraint	59
11.5.2 Missing event identity	59
11.5.3 Hidden dependency	59
11.5.4 Incompatible schemas	59
11.5.5 Temporal gap	59
11.5.6 Nonassociative reducer	59
11.6 Uniqueness failures	59
11.7 Gluing and database joins	60
11.8 A sheaf of deterministic projection behavior	60
11.9 Sheaf condition as an integration test generator	60
11.10 Local versus global correctness	61
11.11 Exercises	61
11.12 Goldblatt bridge	61
12 Sites, Coverage Policies, and Sheafification	62
12.1 Why a topology on points is sometimes too rigid	62
12.2 Pretopology axioms in engineering language	62
12.2.1 Identity	62
12.2.2 Stability under pullback	62
12.2.3 Transitivity	62
12.3 A temporal SessionStream site	63
12.4 A component-observation site	63
12.5 Quorum coverage	63
12.6 Authorization-sensitive coverage	64
12.7 Coverage for REST parameter sufficiency	64
12.8 Sieves	64
12.9 Sheafification	65
12.10 Software analogies for sheafification	65
12.11 Choosing the weakest useful topology	65
12.12 Exercises	65
12.13 Goldblatt bridge	66
Part IV - Topos and Local Logic	67
13 A Topos as a Universe of Varying Software Types	68
13.1 From one presheaf to all presheaves	68
13.2 SessionStream types as presheaves	68
13.3 Limits and colimits are computed contextwise	69
13.4 Why exponentials matter	69
13.5 Representable presheaves and probing a context	70
13.6 Subobjects as context-stable predicates	70
13.7 Topos does not mean “topology-shaped architecture”	71
13.8 An internal SessionStream universe	71
13.9 Elementary versus Grothendieck viewpoints	71
13.10 Exercises	72
13.11 Goldblatt bridge	72
14 Subobjects, Sieves, and Contextual Truth Values	73
14.1 Characteristic functions in $\mathbf{Set}$	73
14.2 Why two truth values are insufficient locally	73

14.3	The subobject classifier of a presheaf topos	74
14.4	A three-context chain	74
14.5	Characteristic arrows as invariant classifiers	74
14.6	Heyting algebra structure	75
14.7	SessionStream truth examples	75
14.7.1	Entity exists	75
14.7.2	Snapshot is fresh	75
14.7.3	Connection is live	75
14.7.4	Projection is complete	75
14.8	Truth versus evidence objects	75
14.9	Exercises	76
14.10	Goldblatt bridge	76
15	Kripke-Joyal Semantics and Operational Knowledge	77
15.1	Forcing notation	77
15.2	Equality	77
15.3	Conjunction and implication	78
15.4	Disjunction is local	78
15.5	Existential witnesses can be local	78
15.6	Universal quantification	79
15.7	Heartbeat: suspected is not crashed	79
15.8	Snapshot completeness as local truth	79
15.9	Tombstones and local existence	80
15.10	What local logic does not replace	80
15.11	Exercises	80
15.12	Goldblatt bridge	81
16	Geometric Morphisms and Change of Context	82
16.1	Translating between entire context universes	82
16.2	Change of base by precomposition	82
16.3	Client view as a change of base	83
16.4	Schema-version change of base	83
16.5	Transport as a geometric translation candidate	83
16.6	Sheafification and inclusion	84
16.7	Preservation and reflection of invariants	84
16.8	A geometric architecture review	84
16.9	Exercises	85
16.10	Goldblatt bridge	85
Part V	Shapes and Cohomology	86
17	Nerves: Turning Overlap into Multidimensional Shape	87
17.1	From a cover to a combinatorial object	87
17.2	What dimension means here	87
17.3	A SessionStream observer cover	88
17.4	The Čech nerve versus the simple nerve	88
17.5	The nerve of a temporal cover	89
17.6	The nerve lemma intuition	89
17.7	Filled faces as higher-order integration evidence	90
17.8	Hypergraphs, simplicial complexes, and categories	90
17.9	Exercises	90
17.10	Reading bridge	91
18	Čech Cohomology from Local Differences	92
18.1	Why groups enter	92
18.2	Cochains on a cover	92

18.3	The first coboundary	93
18.4	The second coboundary	93
18.5	Cocycles and coboundaries	93
18.6	Cohomology is not failure of the sheaf axiom	94
18.7	Constant coefficients on a graph	94
18.8	The square calculation	94
18.9	Filling the triangle	95
18.10	Homology versus cohomology	95
18.11	Nonconstant and cellular sheaves	96
18.12	Exercises	96
18.13	Reading bridge	96
19	A SessionStream Cohomology Laboratory	97
19.1	The diagnostic question	97
19.2	Edge offsets	97
19.3	An off-by-one loop	98
19.4	A more honest order-valued model	98
19.5	A cellular cursor sheaf	98
19.6	Adding higher-order witnesses	99
19.7	Duplicate parity over $\mathbb{F}_2$	99
19.8	Digest and checksum coefficients	100
19.9	Event identity is not an abelian offset	100
19.10	A concrete calculation workflow	100
19.11	Pseudocode	100
19.12	Exercises	101
19.13	Interpretation discipline	101
20	A Sheaf Audit and Capstone Tool	102
20.1	The audit workflow	102
20.1.1	Step 1: State the global claim	102
20.1.2	Step 2: Choose contexts	102
20.1.3	Step 3: Define sections	102
20.1.4	Step 4: Define restriction maps	102
20.1.5	Step 5: Declare covers	102
20.1.6	Step 6: Check matching	102
20.1.7	Step 7: Search for global sections	103
20.1.8	Step 8: Linearize selected residuals	103
20.1.9	Step 9: Interpret nonzero classes	103
20.1.10	Step 10: Change the system or the model	103
20.2	Proposed package layout	103
20.3	Fact-oriented sections	104
20.4	Global-section search	104
20.5	Ingesting SessionStream evidence	105
20.6	Report format	105
20.7	Capstone phases	106
20.7.1	Phase A - Finite toy model	106
20.7.2	Phase B - Event intervals	106
20.7.3	Phase C - Hydration model	106
20.7.4	Phase D - Trace adapter	106
20.7.5	Phase E - Cursor complex	106
20.7.6	Phase F - Higher witnesses	106
20.7.7	Phase G - Repository integration	106
20.8	A decision table	107
20.9	Research notebook prompts	107
20.10	Final synthesis	107

20.11	Capstone exercises	108
Appendix A - A Software-to-Mathematics Translation Dictionary		109
A.1	Categories and diagrams	109
A.2	Universal constructions	109
A.3	Functorial structure	110
A.4	Presheaves and sheaves	111
A.5	Topos and local logic	112
A.6	Cohomology	112
A.7	Parameter sufficiency as fibres	113
Appendix B - Proof and Modeling Techniques		114
B.1	The declaration-before-calculation rule	114
B.2	Proving a diagram commutes	114
B.3	Proving a universal property	114
B.4	Finding a counterexample	115
B.5	Proving functoriality	115
B.6	Proving naturality	115
B.7	Proving the presheaf laws	115
B.8	Testing the sheaf condition	116
B.9	Proving a predicate is a subpresheaf	116
B.10	Computing H^1 in a finite constant-coefficient complex	116
B.11	How to read a proof in this book	116
B.12	How to keep analogy under control	117
Appendix C - Hints for Every Exercise		118
C.1	Chapter 1 hints	118
C.2	Chapter 2 hints	118
C.3	Chapter 3 hints	119
C.4	Chapter 4 hints	119
C.5	Chapter 5 hints	119
C.6	Chapter 6 hints	120
C.7	Chapter 7 hints	120
C.8	Chapter 8 hints	121
C.9	Chapter 9 hints	121
C.10	Chapter 10 hints	121
C.11	Chapter 11 hints	122
C.12	Chapter 12 hints	122
C.13	Chapter 13 hints	123
C.14	Chapter 14 hints	123
C.15	Chapter 15 hints	123
C.16	Chapter 16 hints	124
C.17	Chapter 17 hints	124
C.18	Chapter 18 hints	124
C.19	Chapter 19 hints	125
C.20	Chapter 20 hints	125
Appendix D - Selected Solutions		126
D.1	Solution to E1.3 - Every isomorphism is monic and epic	126
D.2	Solution to E2.2 - Products are unique up to unique isomorphism	126
D.3	Solution to E2.7 - Locally legal numbers, globally incoherent cut	127
D.4	Solution to E3.3 - Meet and join of divergent histories	127
D.5	Solution to E4.4 - Hydration interleavings at cuts 41, 42, and 43	128
	Snapshot cut 41	128
	Snapshot cut 42	128
	Snapshot cut 43	128

D.6 Solution to E4.6 - Two cursor types	128
D.7 Solution to E5.3 - A reducer that is not functorial under batching	129
D.8 Solution to E5.7 - Temporal restriction and the current SQLite store	129
D.9 Solution to E6.2 - A final-cut migration that is not natural	130
D.10 Solution to E6.6 - Composite migrations are natural	130
D.11 Solution to E7.1 - Extending event actions to lists	131
D.12 Solution to E7.2 - Unit, counit, and a triangle identity	131
D.13 Solution to E7.3 - Wall-clock time is a missing generator	132
D.14 Solution to E8.1 - A finite base poset of observation contexts	132
D.15 Solution to E9.1 - Parameter assignments form a presheaf	133
D.16 Solution to E9.3 - Fibres and invariant sufficiency	133
D.17 Solution to E9.7 - Pairwise agreement without a valid global completion	134
D.18 Solution to E10.6 - Sections in a slice category	134
D.19 Solution to E11.1 - Event histories form a sheaf on finite intervals	135
D.20 Solutions to E11.4 and E11.5 - Existence and uniqueness for hydration gluing	135
Existence	135
Uniqueness	136
D.21 Solution to E12.1 - Interval-union covers form a pretopology	136
D.22 Solution to E12.8 - Coarse versus fine topology	136
D.23 Solution to E13.2 - Pointwise product of presheaves	136
D.24 Solution to E13.3 - Agreement as a presheaf equalizer	137
D.25 Solution to E14.2 - Sieves on a four-element chain	137
D.26 Solution to E14.5 - Failure of excluded middle	138
D.27 Solution to E15.2 - What the heartbeat phases justify	138
D.28 Solution to E15.6 - Neither crash nor non-crash is forced	139
D.29 Solution to E16.6 - Sheafification and inclusion as a geometric morphism	139
D.30 Solution to E17.1 - Three nerves from the same three labels	139
Path	140
Triangle boundary	140
Filled triangle	140
D.31 Solution to E18.1 - Cochains on a three-vertex path	140
D.32 Solution to E18.2 - Why the next coboundary is zero	140
D.33 Solution to E18.3 - Solving offsets on a square	141
D.34 Solution to E18.4 - H^1 of a triangle boundary and a filled triangle	141
Boundary triangle	141
Filled triangle	142
D.35 Solution to E18.7 - Cohomology is not failure of the sheaf axiom	142
D.36 Solution to E19.2 - A five-vertex cursor loop	142
D.37 Solution to E19.3 - Repairing an off-by-one contradiction with types	143
D.38 Solution to E20.1 - A sheaf audit of snapshot soundness	143
1. Global context	143
2. Local contexts	143
3. Sections	144
4. Restrictions	144
5. Coverage	144
6. Matching	144
7. Global sections	144
8. Current implementation evidence	144
9. Linear diagnostic	144
10. Engineering actions	144
Appendix E - A Sixteen-Week Study Program	146
Week 1 - Re-enter Goldblatt at limits	146
Week 2 - Colimits and coherent execution cuts	146
Week 3 - Functorial replay	146

Week 4 - Natural transformations and migration	147
Week 5 - Adjunctions as universal software structure	147
Week 6 - Build the context category	147
Week 7 - Presheaves and parameter fibres	147
Week 8 - Bundles, fibres, germs, and sections	148
Week 9 - The sheaf condition through hydration	148
Week 10 - Sites and operational coverage	148
Week 11 - Presheaf topoi and internal structure	148
Week 12 - Sieves and context-sensitive truth	149
Week 13 - Kripke-Joyal semantics and evidence	149
Week 14 - Change of context and geometric morphisms	149
Week 15 - From overlap shape to cohomology	149
Week 16 - SessionStream research lab	149
E.17 A sustainable review cycle	150
Appendix F - Minimum Working Glossary	151
Appendix G - Source Map, Scope, and Provenance	154
G.1 Attached book used as the pedagogical source	154
G.2 Pedagogical features adapted from Goldblatt	154
G.3 SessionStream repository basis	155
G.4 What is exact and what is proposed	155
G.5 Deliberate limits of this textbook	156
G.6 Suggested later reading	156
G.7 Final checklist for future revisions	156

Preface

This is an original, project-specific textbook for studying category theory, presheaves, sheaves, topoi, and elementary cohomological ideas through the architecture of **SessionStream**. It is not a replacement for Robert Goldblatt’s *Topoi: The Categorical Analysis of Logic*. It uses that book as a pedagogical model and a reading companion: begin with concrete constructions, translate them into arrows, isolate a universal property, test the abstraction in several settings, and then work exercises that force the abstraction to become operational.

The intended reader is a software developer who is comfortable with Go, interfaces, state machines, event-driven systems, and database invariants, and who is currently near the limits portion of Goldblatt’s Chapter 3. The book therefore starts at that boundary. It gives a compact bridge over the earlier category-theory vocabulary, develops limits through concrete SessionStream consistency problems, and then introduces only the later material needed to make presheaves, sheaves, topoi, local truth, and cohomology intelligible.

The mathematical objects in this text are not claimed to be the one canonical “category of SessionStream.” Modeling is part of the work. Different choices of objects, arrows, covers, and coefficients answer different engineering questions. Each model is stated explicitly so that its strengths and blind spots can be inspected.

The pedagogical contract

Goldblatt’s most useful pedagogical principle for this project is **particular before general**. Each substantial idea is developed in the following order:

1. Start with a concrete software situation.
2. Identify the operations that remain meaningful after implementation details are forgotten.
3. Draw the commuting diagram or local-to-global shape.
4. State the abstract definition.
5. Compare examples and nonexamples.
6. Prove a small structural fact.
7. Apply the idea to SessionStream.
8. Complete exercises in concept, proof, code, and architecture.

On a first pass, read the motivations, definitions, diagrams, and worked SessionStream examples. Skip any proof that blocks the flow. On a second pass, reconstruct the proofs and complete the exercises. On a third pass, implement the capstone instrumentation described in Chapter 20.

Repository baseline

The SessionStream study is based on repository `go-go-golems/sessionstream`, branch `main`, inspected at commit:

`7d6cdbc3864b91cf4df45ef01931298493a4208f`

The central current source files used throughout are:

`README.md`

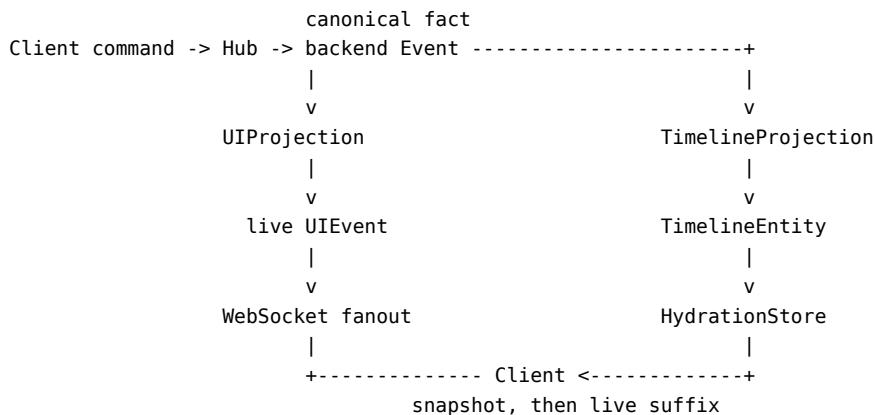
`pkg/sessionstream/types.go`

```
pkg/sessionstream/ordinals.go
pkg/sessionstream/projection.go
pkg/sessionstream/hydration.go
pkg/sessionstream/hub.go
pkg/sessionstream/hydration/sqlite/store.go
pkg/sessionstream/transport/ws/server.go
pkg/sessionstream/transport/ws/internal/heartbeat/machine.go
pkg/doc/topics/01-user-guide.md
```

The repository will evolve. Treat every code-specific statement as tied to that snapshot unless the statement is a proposed model rather than a description of implementation.

The SessionStream study in one page

SessionStream is a session-scoped, event-driven substrate. A command is routed to a handler. The handler publishes canonical backend events. Two projection families derive different views of those events: live `UIEvent` values and durable `TimelineEntity` values. A hydration store persists timeline state and provides snapshots. The WebSocket transport sends a snapshot to a subscribing client, buffers concurrent live UI batches during hydration, filters those batches relative to the snapshot ordinal, flushes the surviving suffix, and then marks the subscription live.



The mathematical theme is that no component sees “the whole execution” in the same representation. The event log, projection view, materialized entities, snapshot, WebSocket buffer, transport trace, and client state are overlapping local descriptions. The recurring question is:

When do compatible local descriptions determine one coherent global execution?

That is the local-to-global problem. Presheaves organize the local descriptions and their restrictions. Sheaves specify when compatible descriptions glue. Topos theory studies a universe of such varying descriptions and its internal logic. Cohomology can sometimes detect residual inconsistency around loops of overlap.

A map from Goldblatt to this book

Goldblatt topic	This book	Role here
Ch. 3, initial/terminal objects, products, equalizers, pullbacks, limits	Chs. 1-4	Coherent assembly and consistent cuts
Ch. 9, functors and natural transformations	Chs. 5-6	Projections, replay, migrations, lawful adapters

Goldblatt topic	This book	Role here
Ch. 15, adjunctions	Chs. 7 and 16	Free/forgetful structure, sheafification, change of base
Ch. 4.5, bundles and sheaves	Chs. 9-11	Fibres, sections, restrictions, gluing
Ch. 14, stacks, sheaves, sites, local truth	Chs. 12-15	Coverage, sieves, Kripke-Joyal semantics
Chs. 4, 10, 14, topoi and classifiers	Chs. 13-15	Context-indexed types and truth values
Ch. 16, geometric morphisms	Ch. 16	Structure-preserving translation between context universes
Not a major topic of Goldblatt	Chs. 17-19	Nerves and introductory Čech cohomology

Goldblatt’s older terminology occasionally uses **stack** for a presheaf-like construction over an ordered base. This text uses the modern term **presheaf**, reserving “stack” for its more specialized contemporary meanings.

Notation used from the beginning

A session is written s . Event ordinals are $m, n, k \in \mathbb{N}$. The event prefix through ordinal n is $E_s^{\leq n}$. A timeline state reconstructed through n is $T_s(n)$. A snapshot with cut n is $S_s(n)$. Live UI events with ordinals strictly after n and at most m are $U_s(n, m]$. A client state after applying a snapshot and suffix is $C_s(m)$.

Composition is written right-to-left: $g \circ f$ means first f , then g . An identity arrow on A is 1_A . The opposite category is $\mathcal{C}^{op}$. The category of presheaves on $\mathcal{C}$ is

$$\widehat{\mathcal{C}} = \mathbf{Set}^{\mathcal{C}^{op}}.$$

A restriction from a larger context U to a smaller context $V \subseteq U$ is written

$$\rho_V^U : \mathcal{F}(U) \rightarrow \mathcal{F}(V), \quad s \mapsto s|_V.$$

Part I - Universal Structure

Chapter 1

Reading Software as Diagrams

1.1 Why begin with arrows?

A software object is usually introduced by listing its fields. For example, the current `SessionStream` event type has a name, protobuf payload, session identifier, and ordinal. That representation is necessary for implementation, but it does not by itself tell us what properties survive a change of representation.

Category theory deliberately asks a different question:

What can be said about an object solely through the ways other objects map into it and the ways it maps into other objects?

For `SessionStream`, this is natural. A backend event matters because it can be validated, appended, projected, serialized, replayed, observed, and related to a timeline state. The set of fields is one coordinate description. The web of structure-preserving maps is the architectural meaning.

Goldblatt’s Chapter 3 develops this move as “arrows instead of membership.” Here the corresponding move is **arrows instead of field inspection**. We will still inspect fields when deriving an idea, but the resulting definition should be stated in terms of composition and identity whenever possible.

1.2 A first category of typed transformations

A category $\mathcal{C}$ consists of:

- objects $A, B, C, \dots$,
- arrows $f : A \rightarrow B$,
- a composite $g \circ f : A \rightarrow C$ whenever $f : A \rightarrow B$ and $g : B \rightarrow C$,
- an identity $1_A : A \rightarrow A$ for every object,

subject to associativity and identity laws:

$$(h \circ g) \circ f = h \circ (g \circ f),$$

$$1_B \circ f = f = f \circ 1_A.$$

A concrete `SessionStream`-flavored category might have **data types** as objects and total, deterministic, schema-respecting transformations as arrows:

```
CommandPayload -> Command -> Event -> TimelineEntity -> SnapshotEntity
```

This is only a toy category. Real handlers can fail, perform effects, emit many events, and depend on context. Those features can be represented by changing the category: partial functions, Kleisli arrows, relations, state transitions, or trace-producing processes. The first discipline is to say what category you are in before relying on a diagram.

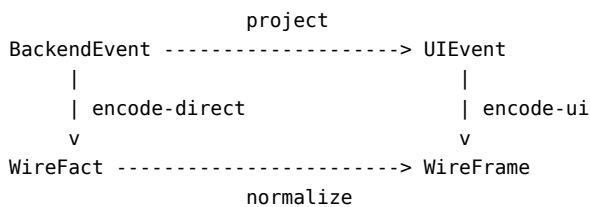
Modeling rule. An arrow is not “any code that happens to call the next thing.” It is a morphism in a declared category, so its equality and composition laws must have definite meanings.

For pure functions, equality may be extensional equality. For state machines, arrows may be traces modulo an equivalence. For database operations, equality may mean observational equivalence under all allowed queries. Category theory does not select this notion for you.

1.3 Commuting diagrams as implementation-independent equations

A diagram commutes when all directed paths with the same source and target denote the same composite arrow.

Suppose a backend event can be converted directly to a wire frame, or first projected to a UI event and then encoded:



The square commutes if

$$\text{normalize} \circ \text{encodeDirect} = \text{encodeUI} \circ \text{project}.$$

This equation is stronger than “both paths usually look similar.” It is a testable contract. It can fail because one path drops an ordinal, uses another schema version, or treats tombstones differently.

SessionStream is full of candidate commuting diagrams:

- append then query versus extend an in-memory prefix;
- replay then project versus incrementally project;
- snapshot at n then restrict to $m < n$ versus snapshot directly at m ;
- encode then decode versus identity;
- migrate then project versus project then migrate the view;
- buffer live events during hydration then flush versus process the same ordered suffix after an instantaneous snapshot.

Not every square should commute. A UI projection intentionally loses backend detail. The useful question is: **which diagrams are required to commute for the advertised invariant?**

1.4 Objects versus instances

A common category-theory confusion for programmers is mixing a type with one runtime value.

In a category of Go types and pure functions:

- Event is an object;
- one event value is an element of that object;
- Project : Event $\rightarrow$ []UIEvent is an arrow.

In a category of runtime states and legal transitions:

- one complete runtime state is an object;

- an event-processing step is an arrow;
- a sequence of steps is composition.

In a category of information contexts:

- “timeline state for session s through ordinal n ” is an object;
- restriction to an earlier prefix is an arrow;
- a presheaf will assign a set of possible observations to each context.

All three models can be useful. Trouble starts when a proof silently moves between them.

1.5 Monic, epic, and isomorphic software maps

An arrow $f : A \rightarrow B$ is **monic** if it is left-cancellable:

$$f \circ g = f \circ h \implies g = h.$$

In **Set**, this is exactly injectivity. A monic encoder preserves enough distinction that two inputs cannot become equal after encoding. A schema adapter that erases an event’s stable identity is generally not monic.

An arrow $f : A \rightarrow B$ is **epic** if it is right-cancellable:

$$g \circ f = h \circ f \implies g = h.$$

In **Set**, this is surjectivity. Outside **Set**, epic need not mean surjective on underlying values.

An arrow $f : A \rightarrow B$ is an **isomorphism** if there is $f^{-1} : B \rightarrow A$ with

$$f^{-1} \circ f = 1_A, \quad f \circ f^{-1} = 1_B.$$

A JSON representation and a protobuf representation are isomorphic only relative to a domain where encode/decode are total and preserve all distinctions. Unknown fields, normalization, default elision, integer precision, and schema evolution can destroy isomorphism.

The software lesson is not that every conversion must be invertible. It is to distinguish:

- **lossless representation change** - candidate isomorphism;
- **embedding into a richer type** - candidate monomorphism;
- **forgetful summary** - often epic in **Set**, rarely monic;
- **observational equivalence** - possible isomorphism in a quotient category even when raw bytes differ.

1.6 Isomorphism is not equality

`SessionStream` can have two independently allocated snapshots that contain equal semantic state but differ in memory identity, ordering of maps, or serialized bytes. Category theory usually cares whether there is a structure-preserving isomorphism, not whether two representations are literally identical.

This matters when specifying rebuilds. The desired claim is normally not

```
replayedSnapshot == originalSnapshot // byte-for-byte
```

but something like

$$\text{normalize}(\text{replay}(E_s^{\leq n})) = \text{normalize}(S_s(n)).$$

The normalization arrow identifies irrelevant representation choices. Choosing that arrow is part of the specification.

1.7 Worked study: event identity

The current `Event` type contains no independent logical event identifier beyond its session and ordinal. In the local publisher path the hub assigns an ordinal and immediately processes the event. A bus-backed path can derive an ordinal from stream metadata.

Suppose a producer retries one logical fact. Consider the map

$$q : \text{DeliveryAttempt} \rightarrow \text{StoredEvent}$$

that assigns a new ordinal to each accepted delivery. If two attempts for the same logical fact receive different ordinals, then a downstream observer cannot infer from stored event equality that the attempts were the same fact. The problem is not yet cohomology. The coordinate system may simply omit a stable `EventId`.

This example illustrates a recurring diagnostic sequence:

1. State what distinctions the invariant needs.
2. Ask whether the relevant map is monic with respect to those distinctions.
3. If not, decide whether information loss is intentional.
4. Add a coordinate or weaken the invariant.

1.8 Exercises

E1.1 - Concept (one star). Give three different categories in which `SessionStream` code could be studied. For each, name its objects, arrows, composition, and arrow equality.

E1.2 - Diagram (one star). Draw a square comparing incremental timeline projection with event-log replay. State exactly what equality of the two resulting timeline states means.

E1.3 - Proof (one star). Prove from the arrow definitions that every isomorphism is both monic and epic.

E1.4 - Architecture (two stars). List every conversion in the `WebSocket` path that you currently expect to be lossless. For each, specify the subset of values on which an inverse exists.

E1.5 - Code (two stars). Write a Go property test for one candidate commuting square. Generate inputs, execute both paths, normalize outputs, and compare them.

E1.6 - Modeling (two stars). Is `TimelineProjection.Project` naturally an arrow `Event -> []TimelineEntity`? Explain which arguments and effects make that type misleading. Propose a more honest categorical model.

E1.7 - Research notebook (three stars). Define an observational equivalence relation on snapshots suitable for your client. Which fields are semantically observable? Which orderings are irrelevant? Is the relation a congruence for applying live UI events?

1.9 Goldblatt bridge

Read Goldblatt Chapter 3, Sections 3.1-3.4 alongside this chapter. His progression from injective/surjective/bijective set functions to monic/epic/iso arrows is the model for our progression from concrete codecs and event identities to cancellation and invertibility.

Chapter 2

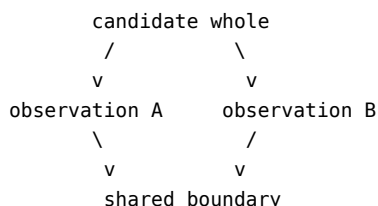
Limits as Coherent Observation

2.1 The engineering problem behind a limit

A limit begins with several objects connected by arrows - a **diagram** - and asks for a universal object that maps consistently into the whole diagram.

That abstract sentence describes a familiar engineering task. Several subsystems expose partial views. We need one candidate state whose projections agree with all of them. Among all such candidates, we want the most general one: every other consistent candidate should map to it uniquely.

The recurring shape is:



The candidate whole is not chosen because its fields look convenient. It is characterized by what maps out of it and by a universal factorization property.

Mental image. A limit is the sharpest joint viewpoint from which every observation in a diagram can be seen consistently.

2.2 Terminal objects: the context that carries no choice

An object 1 is terminal if every object A has exactly one arrow $! : A \rightarrow 1$.

In $\mathbf{Set}$, any singleton is terminal. In a category of typed records and field projections, an empty record can act terminal: every record has one projection that forgets everything.

For software reasoning, a terminal object often represents:

- no observable output,
- a completed effect whose return value is irrelevant,
- the empty context,
- the unique trivial invariant.

A global element of A is an arrow $1 \rightarrow A$. This reframes “pick a value of A ” as “map the context with one possible state into A .” Later, a global section of a presheaf or sheaf will also be an arrow from a terminal object.

2.3 Products: independent observations held together

A product of A and B is an object $A \times B$ with projections

$$\pi_A : A \times B \rightarrow A, \quad \pi_B : A \times B \rightarrow B,$$

such that for every pair $f : X \rightarrow A, g : X \rightarrow B$, there is a unique arrow

$$\langle f, g \rangle : X \rightarrow A \times B$$

with

$$\pi_A \circ \langle f, g \rangle = f, \quad \pi_B \circ \langle f, g \rangle = g.$$

The product is not merely “a struct with two fields.” The universal property says that providing one arrow into the product is equivalent to providing one arrow into each factor.

A SessionStream diagnostic record might combine an event cursor and a timeline cursor:

```
type CursorPair struct {
    Event      uint64
    Projection uint64
}
```

This record is a Set-product if all pairs are allowed. The moment we require `Projection <= Event`, it becomes a subobject of the product. If we require both cursors to be images of the same committed transaction, the correct object may be a pullback rather than a raw product.

2.4 Equalizers: the subspace where two computations agree

Given parallel arrows $f, g : A \rightarrow B$, an equalizer is an arrow $e : E \rightarrow A$ satisfying

$$f \circ e = g \circ e,$$

and universal among arrows into A with that property.

In Set,

$$E = \{a \in A \mid f(a) = g(a)\}.$$

An equalizer is therefore a canonical type of values on which two interpretations agree.

SessionStream examples include:

- events for which two schema decoders produce the same semantic payload;
- snapshots for which `max(entity.LastEventOrdinal)` agrees with the declared cut;
- traces for which incremental and replay projections agree;
- client states for which live rendering and post-reload rendering normalize to the same state.

Suppose

$$\text{Inc, Replay} : \text{Trace} \rightarrow \text{TimelineState}.$$

Then the equalizer $E \hookrightarrow \text{Trace}$ is the space of traces satisfying deterministic replay equivalence. A test suite samples this equalizer; a proof attempts to characterize all of it.

2.5 Pullbacks: joining observations through shared meaning

Given $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback is an object $A \times_C B$ with arrows to A and B whose composites into C agree, universal among all such consistent pairs.

$$\begin{array}{ccc}
 A \times_C B & \xrightarrow{\quad} & B \\
 | & & | \\
 | & & | \quad g \\
 \downarrow & & \downarrow \\
 A & \xrightarrow{\quad f \quad} & C
 \end{array}$$

In Set,

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

This is a typed join. The shared object C says what the join key means.

2.5.1 A SessionStream pullback

Let:

- A be persisted timeline entity versions;
- B be a declared snapshot cut;
- C be the ordinal domain;
- $f(a) = a.\text{LastEventOrdinal}$;
- $g(b) = b.\text{SnapshotOrdinal}$.

An equality pullback would select versions exactly at the cut. Snapshot validity usually needs an inequality rather than equality, so define a relation object

$$R = \{(x, n) \mid x \leq n\}$$

with projections to ordinals. Then a valid entity/cut pair is a pullback through R . This illustrates a practical point: universal constructions become expressive when you model the invariant as an object and arrows, not when you force every condition into equality prematurely.

A second pullback occurs during subscription authorization. A requested session and an authorization claim must map to the same tenant or principal boundary before hydration begins.

2.6 Consistent snapshots as a limiting cone

For a fixed session s , imagine a diagram containing:

- event prefix $E_s^{\leq n}$,
- timeline materialization $T_s(n)$,
- snapshot cut n ,
- entity version metadata,
- projection cursor.

A candidate snapshot state X maps to each component:

$$\begin{array}{c}
 X \\
 / \ / \ | \ \backslash \ \backslash \\
 v \ v \ v \ v \ v \\
 \text{events} \ \text{timeline} \ \text{cut} \ \text{entities} \ \text{cursor}
 \end{array}$$

The maps must satisfy compatibility equations, for example:

$$\text{fold}(E_s^{\leq n}) = T_s(n),$$

$$\max_{x \in T_s(n)} x.\text{LastEventOrdinal} \leq n,$$

$$\text{ProjectionCursor}(s) \geq n$$

under whichever cursor semantics are chosen.

A **limit** of the diagram is the universal coherent tuple. In **Set**, finite limits can be constructed from products and equalizers: first take all possible tuples, then equalize the equations they must satisfy. This is why limits are such a natural language for transactional invariants.

2.7 The universal property as API quality criterion

An API often returns a record that happens to contain several fields. A universal-property mindset asks stronger questions:

1. What observations are the projections from this record?
2. What compatibility equations hold among them?
3. Can every compatible family of observations be assembled into exactly one record?
4. Is the assembly stable under representation changes?

For example, suppose **Snapshot** contains **SessionId**, **SnapshotOrdinal**, and **Entities**. Is every combination legal? No. The session IDs must align; entity ordinals must respect the cut; payload schemas must match entity kinds; tombstones may be excluded. A mathematically honest snapshot type is therefore not the raw product

$$\text{SessionId} \times \text{Ordinal} \times \text{List}(\text{Entity}),$$

but a subobject selected by invariants, perhaps represented as an equalizer or pullback.

In conventional Go, these invariants live in constructors, database constraints, tests, and comments. Category theory helps reveal that they define an object independently of its chosen struct representation.

2.8 Worked study: atomic application and progress

The current hub processing path can append an event, obtain a view, run UI and timeline projections, apply timeline entities, advance a projection cursor, and publish UI fanout. These effects are not one categorical arrow merely because they occur in one function. Their failure boundaries matter.

Let

$$A = \text{event appended}, \quad M = \text{materialization applied}, \quad P = \text{projection cursor advanced}.$$

The advertised progress claim may be:

$$P(n) \Rightarrow A(n) \wedge M(n).$$

If Apply commits but cursor advancement fails, the local observations may be

```
event cursor      = n
materialized state = n
projection cursor = n-1
```

Each value can be locally well-formed. The tuple is not in the intended invariant subobject. A transaction that commits Apply and cursor advancement together can be understood as changing the implementation so that only points in the desired pullback/equalizer become externally visible.

This is not proof that every transaction is a limit. The precise claim is that finite-limit language describes the coherent state space a transaction is intended to expose.

2.9 Exercises

E2.1 - Universal property (one star). For a Go struct `Pair[A,B]`, write the projections and pairing operation. State and test the two product equations.

E2.2 - Proof (one star). Prove that any two products of the same pair of objects are uniquely isomorphic in a way that commutes with their projections.

E2.3 - Equalizer (one star). Define two functions from `Snapshot` to a normalized timeline state: one reads the entity payloads; the other replays the event prefix. What assumptions are needed for their equalizer to be meaningful?

E2.4 - Pullback (two stars). Model `(SessionId, Ordinal)` as a shared boundary object. Define a pullback joining a transport frame to a persisted event. Which fields must map into the boundary?

E2.5 - Database (two stars). Rewrite a snapshot SQL query conceptually as: choose a cut, choose the latest entity version not after the cut, and assemble the compatible family. Identify the product, relation, and selection/equalizer stages.

E2.6 - Architecture (two stars). List the externally visible intermediate states of `projectAndApply`. Which are intended states and which are recovery states? Draw the subobject of allowed states inside the product of all local status fields.

E2.7 - Counterexample (two stars). Construct a record with individually valid event cursor, snapshot ordinal, and entity ordinals that cannot describe one coherent cut.

E2.8 - Proof/code (three stars). Show that a finite limit in `Set` can be built as an equalizer of two maps between products. Implement this construction for a small diagram represented as finite sets and functions.

2.10 Goldblatt bridge

Study Goldblatt Sections 3.5-3.13 in parallel: initial and terminal objects, products, equalizers, and pullbacks. Pay attention to the repeated pattern: concrete set construction, arrow-only formulation, uniqueness up to a unique commuting isomorphism, and exercises using the universal property rather than element calculation.

Chapter 3

General Limits, Colimits, and Architectural Duality

3.1 From named constructions to one pattern

Products, equalizers, pullbacks, and terminal objects initially look unrelated. The definition of a limit reveals that each is the same construction applied to a different diagram shape.

Let J be a small category that describes a shape, and let

$$D : J \rightarrow \mathcal{C}$$

be a diagram in $\mathcal{C}$. A **cone** from an object X to D consists of one arrow

$$\lambda_j : X \rightarrow D(j)$$

for every object j of J , such that every diagram arrow $u : i \rightarrow j$ satisfies

$$D(u) \circ \lambda_i = \lambda_j.$$

The cone's legs are a compatible family of observations of X .

A **limit cone** (L, π_j) is terminal among cones to D : for every cone (X, λ_j) , there is exactly one arrow $u : X \rightarrow L$ such that

$$\pi_j \circ u = \lambda_j$$

for all j .

$$\begin{array}{ccccc}
 & & X & & \\
 & \cdot & | & \cdot & \\
 & \cdot & | u & \cdot & \\
 \text{lambda} & \cdot & v & \cdot & \text{lambda} \\
 & \text{Limit } L & & & \\
 & / & | & \backslash & \\
 & v & v & v & \\
 & & D & &
 \end{array}$$

This is a universal-property statement. It says that L represents the operation “give a compatible cone over D .” The limit is unique up to a unique isomorphism respecting all projections.

3.2 Recovering familiar limits from shapes

Different choices of J produce familiar objects.

Diagram shape	Limit
Empty diagram	Terminal object
Two unrelated objects	Product
Parallel arrows $A \rightrightarrows B$	Equalizer
Cospan $A \rightarrow C \leftarrow B$	Pullback
Chain of prefix contexts	Inverse limit, when it exists

The empty diagram has no compatibility equations. Every object supplies one empty cone. A terminal object is therefore a universal empty cone.

For two unrelated objects, a cone is exactly a pair of arrows. The universal cone is their product.

For parallel arrows $f, g : A \rightarrow B$, a cone consists of an arrow $x : X \rightarrow A$ for which $f \circ x = g \circ x$. The universal such cone is the equalizer.

For a cospan $A \rightarrow C \leftarrow B$, a cone is a pair of observations with equal images in C . The universal cone is the pullback.

This unification is not merely terminological. Once a theorem is proved for all limits, it automatically applies to products, equalizers, pullbacks, and any more elaborate limit.

3.3 Limits in a poset: greatest coherent lower bounds

A preorder can be regarded as a category: there is one arrow $p \rightarrow q$ exactly when $p \leq q$. A cone to a diagram is then a lower bound. A limit is a greatest lower bound.

This is valuable for software because many engineering domains carry information orders.

Let $x \leq y$ mean “ x contains no more information than y .” Then the meet $x \wedge y$ is the greatest observation contained in both. It can represent:

- the common fields of two schemas;
- the shared prefix of two traces;
- the strongest invariant implied by two specifications;
- the overlap context of two services;
- the greatest snapshot cut known safe by two replicas.

Be explicit about the order. If $x \leq y$ instead means “ x is at least as refined as y ,” then meets and joins exchange their intuitive roles. Many apparent category-theory paradoxes in software models are order-orientation errors.

3.4 Completeness is a capability, not a default

A category is **complete** if every small diagram has a limit. It is **finitely complete** if every finite diagram has a limit. A standard result is that finite completeness follows from having a terminal object and pullbacks; equivalently, from having finite products and equalizers.

Software categories are often not complete.

- A category of finite-state machines may lack a desired infinite limit.
- A category whose arrows must meet latency bounds may not contain a universal product implementation.
- A category of serializable schemas may lack an exponential or coequalizer that stays serializable.
- A category of deployed services may not contain an object representing every mathematically possible compatible tuple.

Never infer that a construction exists just because it exists in `Set`. Existence is an engineering claim about the chosen category.

For a practical architecture review, the question “does this limit exist?” becomes:

Is there an object within our allowed design vocabulary that exposes exactly the compatible families, with the required universal factorization?

A database transaction, a materialized view, or a typed aggregate may realize the limit. Sometimes no such implementation exists without changing the category - for example, by allowing a coordinator, a larger schema, or an additional persistence table.

3.5 Colimits: universal assembly from pieces

Reverse every arrow in the definition of a limit. A **cocone** from D to X has arrows $D(j) \rightarrow X$ compatible with the diagram. A **colimit** is initial among cocones.

Mental image. A limit finds a universal coherent viewpoint *into* many observations. A colimit finds a universal assembled target *out of* many pieces.

Familiar colimits include:

Diagram shape	Colimit
Empty diagram	Initial object
Two unrelated objects	Coproduct
Parallel arrows	Coequalizer
Span $A \leftarrow C \rightarrow B$	Pushout

Limits are often associated with constraints and matching. Colimits are often associated with generation, union, quotienting, and attachment. This slogan is useful but not a definition.

3.6 Coproducts and typed event alternatives

In `Set`, the coproduct $A + B$ is a disjoint union. Its injections remember whether a value came from A or B . A Go sum type emulation with a tagged union attempts to realize this:

```
type ClientFrame struct {  
    Subscribe *SubscribeFrame  
    Unsubscribe *UnsubscribeFrame  
    Ping *PingFrame  
    Pong *PongFrame  
}
```

A well-formed protobuf `oneof` is closer to a coproduct than a struct that permits several alternatives simultaneously. The coproduct universal property says that defining a consumer

$$[A \xrightarrow{f} X, B \xrightarrow{g} X]$$

is equivalent to defining one function $[f, g] : A + B \rightarrow X$ by cases.

`SessionStream`’s logical names and concrete protobuf payloads also resemble a large indexed sum. The schema registry controls which injection corresponds to each name. Top-level arbitrary maps weaken this structure because they blur the disjoint alternatives into an undifferentiated payload space.

3.7 Coequalizers and semantic quotienting

Given $f, g : A \rightrightarrows B$, a coequalizer $q : B \rightarrow Q$ satisfies

$$q \circ f = q \circ g$$

and is universal among arrows that identify the outputs of f and g .

In $\mathbf{Set}$, a coequalizer forms a quotient of B by the smallest equivalence relation needed to identify $f(a)$ with $g(a)$.

Software examples:

- treat legacy and current event names as the same semantic event;
- identify two transport encodings after normalization;
- collapse duplicate deliveries with the same stable event identity;
- quotient traces by reorderings declared observationally irrelevant;
- identify entity aliases after a migration.

Quotienting is dangerous when the equivalence relation is implicit. If two deliveries are identified only by `(SessionId, Ordinal)`, a retry that receives a fresh ordinal escapes the quotient. If two payloads are identified by lossy normalization, distinct domain facts may collapse.

The universal property forces the policy into view: every downstream map out of Q must respect the identification.

3.8 Pushouts and integration boundaries

A pushout of a span $A \leftarrow C \rightarrow B$ joins A and B while identifying the copies of the shared interface C .

$$\begin{array}{ccc} C & \xrightarrow{\quad\quad\quad} & B \\ | & & | \\ | & & | \\ \downarrow & & \downarrow \\ A & \xrightarrow{\quad\quad\quad} & A +_C B \end{array}$$

Possible software readings include:

- merge two schemas along a common version;
- combine two bounded contexts along a shared identifier contract;
- attach an extension protocol to a core protocol;
- merge two traces that share a common prefix;
- assemble two code generators around a shared intermediate representation.

A pushout is not automatically a safe merge. In a category of sets it may freely glue values, while in a category of databases the corresponding merge may violate keys or business constraints. Again, existence and meaning depend on the category.

3.9 Limits, colimits, and event sourcing

Event sourcing naturally alternates between colimit-like and limit-like views.

A log is assembled by appending generated facts. Projection folds combine a sequence into state. This feels colimit-like: history grows by adjoining pieces. But a coherent snapshot is selected by simultaneously satisfying many observations, which is limit-like.

The same operation can admit multiple categorical descriptions depending on what objects and arrows are chosen. A fold is formally an algebra for an endofunctor or a monoid action more often than it is literally a categorical colimit. Resist analogy inflation. The productive method is:

1. write the exact source and target objects;
2. write the universal property;
3. test whether the implementation realizes it;
4. keep the analogy only if the equations hold.

3.10 A practical duality table

Constraint-oriented question	Generative dual
Which values satisfy both observations?	How do we freely combine alternatives?
Equalizer	Coequalizer
Pullback / typed join	Pushout / typed merge
Product / simultaneous tuple	Coproduct / tagged alternative
Terminal / forget all information	Initial / generate uniquely into any target
Limit / universal coherent cone	Colimit / universal assembled cocone

Duality is useful for generating designs. After understanding snapshot consistency as a pullback, ask what its pushout dual would mean. The answer may be irrelevant, but the exercise exposes which arrow direction carries the semantics.

3.11 Exercises

E3.1 - Definitions (one star). Rewrite the definitions of cone, limit, cocone, and colimit without using the words “universal” or “best.” Use only arrows and unique factorization.

E3.2 - Shape recognition (one star). Give the indexing category for a product, equalizer, and pullback. Draw each diagram.

E3.3 - Poset (one star). In the prefix order on event histories, calculate the meet of two histories that share a prefix and then diverge. Does a join exist if divergent histories cannot be merged?

E3.4 - Coproduct (two stars). Treat the protobuf client-frame oneof as a coproduct. State the case-analysis universal property and identify malformed Go values that would not belong to the coproduct.

E3.5 - Coequalizer (two stars). Define a proposed equivalence relation on transport traces that ignores heartbeat frames. Which analyses factor through the quotient? Which do not?

E3.6 - Pushout (two stars). Draw a schema-migration pushout joining version 1 and version 2 through a shared stable core. Explain what conflicts prevent the pushout from existing in a category of validated schemas.

E3.7 - Completeness (three stars). Choose a category of production-safe SessionStream deployments. Argue for one finite limit it lacks. What additional architectural component would make that limit exist?

E3.8 - Dual design (three stars). Dualize a pullback-based authorization check. Decide whether the resulting pushout has any coherent security meaning or merely demonstrates the limits of analogy.

3.12 Goldblatt bridge

Goldblatt Sections 3.11-3.15 introduce limits, colimits, pullbacks, pushouts, and completeness after the concrete constructions have been developed. Reproduce his abstraction step: identify the common diagram-and-factorization pattern before memorizing terminology.

Chapter 4

SessionStream as a System of Compatible Views

4.1 A semantic model before an implementation model

We now define a compact semantic model that will be reused throughout the book. It is intentionally cleaner than the code. Its purpose is to state invariants precisely enough that the code can be compared with them.

Fix a session s . Let the canonical backend events be

$$E_s = (e_1, e_2, \dots),$$

where event e_n has ordinal n in the simplest model. The prefix through n is

$$E_s^{\leq n} = (e_1, \dots, e_n).$$

Let a deterministic timeline projection fold be

$$\text{fold}_T : T_0 \times E_s^{\leq n} \rightarrow T_s(n),$$

or recursively

$$T_s(0) = T_0, \quad T_s(n+1) = \text{step}_T(T_s(n), e_{n+1}).$$

Let the UI projection emit a finite sequence for each event:

$$\text{ui}(T_s(n-1), e_n) = u_n^1, \dots, u_n^{k_n}.$$

A snapshot is

$$S_s(n) = (s, n, \text{entities}(T_s(n))).$$

A client has a reducer

$$\text{applySnap} : C \times S \rightarrow C, \quad \text{applyUI} : C \times U \rightarrow C.$$

This model already exposes several choices that the implementation cannot decide for us:

- Is ordinal assignment dense, or merely strictly increasing?
- Is the timeline projection deterministic?
- Does a UI event carry enough information to reconstruct durable state?
- Is client state intended to be equivalent to timeline state or only visually related?
- Are tombstones represented in snapshots or only used internally?
- What is the equality relation on timeline and client states?

Every theorem below depends on explicit answers.

4.2 The current implementation surfaces

At the inspected repository snapshot:

- Command carries name, payload, and session ID.
- Event additionally carries an ordinal.
- TimelineEntity carries kind, ID, creation ordinal, last-event ordinal, payload, and tombstone status.
- HydrationStore exposes Apply, Snapshot, View, and Cursor.
- optional store capabilities expose event replay and per-projector cursors.
- Hub.projectAndApply appends an event when supported, obtains a timeline view, executes UI and timeline projections, applies timeline entities, advances the timeline projection cursor, and fans out UI events.
- the SQLite store writes each Apply call inside a transaction and stores both current entities and entity versions.
- the WebSocket server registers a subscription as hydrating before loading its snapshot, buffers concurrent UI batches, sends the snapshot, flushes batches after the cut, and then marks the subscription live.

These surfaces are already a family of overlapping views. They are not merely layers in a call stack.

4.3 The principal coordinates

The word “cursor” is dangerously overloaded. Distinguish at least:

$O_E(s)$ = highest canonical event ordinal persisted,

$O_T(s)$ = highest event promised materialized by timeline projection,

$O_S(s)$ = snapshot cut returned to a client,

$O_X(x)$ = last event ordinal represented by entity x ,

$O_U(u)$ = backend event ordinal associated with a UI batch,

$O_C(c)$ = highest ordinal the client claims incorporated.

These values may be equal in a healthy, synchronous run. They are not the same concept. Their semantic types should be thought of as distinct even if Go represents all of them as `uint64`.

A first invariant family is:

$$O_X(x) \leq O_S(snap)$$

for every entity x in snap, and

$$O_T(s) \leq O_E(s).$$

A stronger contract may require equality at specific boundaries. A weaker eventually consistent contract may permit lag. The category model must encode which inequalities are allowed.

4.4 Type and schema invariants

SessionStream’s schema registry maps logical names to concrete protobuf prototypes. A type-valid event satisfies

$$\text{descriptor}(e.\text{Payload}) = \text{registeredDescriptor}(e.\text{Name}).$$

A timeline entity similarly satisfies a kind/payload compatibility relation.

These conditions define subobjects of the raw record products. For example, let

$$R_E = \text{Name} \times \text{ProtoMessage} \times \text{SessionId} \times \text{Ordinal}.$$

The semantic event object is the subobject

$$E \hookrightarrow R_E$$

whose points obey registration, nonempty-session, nonnil-payload, and ordinal constraints.

The ban on top-level arbitrary Struct payloads is not only a style preference. A concrete message preserves a visible injection into a typed sum of event alternatives. An arbitrary map pushes critical constraints into runtime conventions that are harder to express as arrows and subobjects.

4.5 Replay coherence

A central event-sourcing square is:

$$\begin{array}{ccc}
 \text{Event prefix at } n & \xrightarrow{\text{append } e_{(n+1)}} & \text{Event prefix at } n+1 \\
 \downarrow \text{fold} & & \downarrow \text{fold} \\
 \text{Timeline } T(n) & \xrightarrow{\text{project/apply } e_{(n+1)}} & \text{Timeline } T(n+1)
 \end{array}$$

The square commutes when incremental processing agrees with replay:

$$\text{fold}_T(E_s^{\leq n+1}) = \text{step}_T(\text{fold}_T(E_s^{\leq n}), e_{n+1}).$$

This equation appears trivial because the right side resembles a recursive definition. In the implementation it can fail if projection code reads hidden mutable state, wall-clock time, randomness, network results, unversioned schemas, or a current view inconsistent with the event prefix.

Hidden-coordinate test. Whenever replay and live processing disagree, search for an input to the projection that is not represented in the canonical history or initial state.

If `time.Now()` affects a timeline entity, time is a missing coordinate. Either record the relevant time in the event or admit that replay depends on an external context.

4.6 Snapshot soundness and completeness

A snapshot at cut n should satisfy at least two different properties.

Soundness: nothing in the snapshot claims information from after the cut.

$$\forall x \in S_s(n). \text{Entities}, \quad O_X(x) \leq n.$$

Completeness: the snapshot contains exactly the durable entities specified by the timeline fold through the cut, modulo the chosen observational equivalence $\cong_T$.

$$\text{decodeSnapshot}(S_s(n)) \cong_T T_s(n).$$

Soundness without completeness permits missing entities. Completeness without soundness is ill-typed because the alleged cut is wrong. Testing only the maximum entity ordinal checks part of soundness but not completeness.

The SQLite store's version table makes historical asOf snapshots conceptually possible: for each (kind, id) , choose the latest version whose event ordinal is at most the cut. This realizes a temporal restriction operation that will become important when defining a presheaf over prefixes.

4.7 Snapshot-before-live as a gluing protocol

Suppose subscription begins while the system is at ordinal m . Snapshot loading returns a cut $n \leq m$. Live UI batches may arrive during the load.

The desired client-visible sequence is:

$$S_s(n), \quad U_s(n, m], \quad U_s(m, \dots).$$

The protocol must prevent:

- **gaps:** an event is represented by neither snapshot nor suffix;
- **duplicates:** an event is represented twice in a non-idempotent way;
- **reordering:** a later UI batch is delivered before an earlier surviving batch;
- **cross-session contamination:** a batch for another session enters the suffix;
- **cut confusion:** filtering uses a different ordinal interpretation from the snapshot.

The current WebSocket implementation creates a hydrating subscription before snapshot load, buffers concurrent batches, sends the snapshot, filters buffered batches by `ordinal > SnapshotOrdinal`, flushes them in stable ordinal order, handles late arrivals under the connection lock, and only then transitions to live state. This is an explicit implementation of a local-to-global assembly procedure.

The fundamental equation is app-specific. If UI events update a client reducer, write

$$\text{ApplyUI}^*(\text{ApplySnapshot}(C_0, S_s(n)), U_s(n, m]) \cong_C C_s(m).$$

Do not replace $\cong_C$ by equality until client observational equivalence is defined.

4.8 The boundary object at the cut

Ordinary sheaf gluing relies on overlaps. Snapshot $[0, n]$ and a strict suffix $(n, m]$ appear disjoint. The software protocol nevertheless has a shared boundary: the session and cut metadata.

Introduce a boundary context

$$B_s(n) = (s, n, \text{schema versions, projection semantics}).$$

Both snapshot and suffix restrict to this boundary:

Snapshot context $[\emptyset, n]$ $\text{-----} \rightarrow B_s(n) \text{-----} \leftarrow$ Live context $(n, m]$

Compatibility requires both sides to agree on the meaning of session, ordinal, schemas, and client reducer. The glued client execution is universal among assemblies respecting that boundary.

This additional object is important. Without it, “snapshot plus live” is only temporal concatenation. With it, the protocol becomes an explicit gluing problem over a common semantic interface.

4.9 Projection progress as a visibility invariant

Let $\text{Applied}(n)$ mean timeline state through n is durable, and $\text{Advanced}(n)$ mean the projection cursor reports n . A strong progress invariant is

$$\text{Advanced}(n) \Rightarrow \text{Applied}(n).$$

For exact progress:

$$O_T(s) = \max\{n \mid \text{Applied}(n)\}.$$

If cursor advancement and entity application are separate commits, there can be windows in which the tuple of local states does not lie in the exact-progress subobject. That may be acceptable if recovery semantics are documented. The mistake is to expose a cursor whose public meaning is stronger than its transaction boundary supports.

The error policy introduces another branch. If a timeline projection fails and policy advances anyway, the cursor no longer means “materialized through this event”; it means something closer to “processed or deliberately skipped through this event.” The semantic type changed even though the representation did not.

4.10 Heartbeat suspicion as local evidence

The heartbeat state machine has phases including booting, idle, writing, awaiting, suspected, and stopped. A matching pong before the deadline returns the machine to idle. A deadline event for the current generation moves it to suspected and requests connection closure.

The proposition produced is not

remote process has crashed.

It is

under this timeout and observation history, the connection is suspected.

This distinction will become central in local truth. The detector’s state is a section over an observation interval. A longer or differently instrumented context may support a different proposition. Silence is local evidence, not omniscient truth.

4.11 A consistency matrix

Use a matrix like the following to prevent vague claims:

Invariant	Local observations	Shared boundary	Global witness
Replay coherence	event prefix; incremental timeline	event ordinal and initial state	one normalized timeline state
Snapshot soundness	cut; entity versions	ordinal semantics	snapshot at that cut
Hydration completeness	snapshot; buffered suffix	session and cut	client execution with no gap
Projection progress	applied entities; cursor	projector/session/ordinal	committed materialization fact
Typed transport	logical name; protobuf payload	schema registry	valid frame/event value
Heartbeat suspicion	ping write; deadline; pong	generation and nonce	detector state transition

Each row suggests a diagram, a test harness, and eventually a presheaf of local observations.

4.12 Exercises

E4.1 - Semantics (one star). Define equality or observational equivalence for backend event histories, timeline states, snapshots, UI traces, and client states. Do not reuse one equality automatically.

E4.2 - Invariants (one star). Separate snapshot soundness, completeness, and freshness. Give one counterexample satisfying any two but not the third.

E4.3 - Hidden input (one star). List every non-event input available to a timeline projection. Classify each as deterministic context, recorded fact, forbidden dependency, or intentionally external coordinate.

E4.4 - Hydration trace (two stars). Construct a concrete interleaving in which events 41, 42, and 43 are published while a snapshot is loading. Trace the buffer and show the output for snapshot cuts 41, 42, and 43.

E4.5 - Boundary (two stars). Expand $B_s(n)$ into a record of every semantic assumption needed to glue snapshot and live suffix. Which assumptions are currently implicit?

E4.6 - Error policy (two stars). Give distinct names and types to a “materialized cursor” and a “processed-or-skipped cursor.” Show how conflating them can produce a false commuting diagram.

E4.7 - Transaction design (three stars). Propose a schema and transaction that atomically records entity versions and projection progress. State the limit/pullback invariant it enforces and the recovery states it eliminates.

E4.8 - Property testing (three stars). Design a state-machine property test for snapshot-before-live. Generate event publication and snapshot-completion interleavings. Specify the oracle independently of the implementation buffer.

E4.9 - Model criticism (three stars). Identify one place in this chapter where a category-theoretic analogy could be misleading. Replace it with a precise object, arrow, and equation or discard it.

4.13 Milestone

At this point you should be able to read limits as a language of coherent multi-view state. Before continuing, write a one-page specification of SessionStream containing only:

- objects or state spaces;
- arrows or transitions;
- commuting diagrams;
- invariant subobjects;
- universal constructions.

Do not mention implementation packages until the final paragraph.

Part II - Functorial Architecture

Chapter 5

Functors as Lawful Architectural Translations

5.1 Why a projection is not automatically a functor

Programmers often hear “a functor is a map” and immediately label every transformation a functor. That loses the substance of the definition.

A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ maps:

- every object A of $\mathcal{C}$ to an object $F(A)$ of $\mathcal{D}$;
- every arrow $f : A \rightarrow B$ to an arrow $F(f) : F(A) \rightarrow F(B)$;

while preserving identities and composition:

$$F(1_A) = 1_{F(A)},$$

$$F(g \circ f) = F(g) \circ F(f).$$

A `SessionStream TimelineProjection` function receives an event, session, and current view and returns entities. By itself, that function does not yet map the objects and arrows of a declared source category to those of a target category. It may participate in a functorial construction, but the categories and laws must first be supplied.

This chapter builds one honest construction.

5.2 The prefix category of a session

Fix a session s with a canonical ordered event history. Define a category **Prefix** _{s} :

- objects are event cuts $0, 1, 2, \dots$;
- there is a unique arrow $m \rightarrow n$ when $m \leq n$;
- composition is transitivity of $\leq$;
- identities are reflexivity.

The arrow $m \rightarrow n$ can be read as the suffix

$$E_s(m, n] = (e_{m+1}, \dots, e_n).$$

Now define a category **Timeline** _{s} :

- objects are valid timeline states;
- an arrow $T \rightarrow T'$ is an allowed deterministic transition induced by a canonical suffix;
- composition is sequential suffix application;
- identity is application of the empty suffix.

A deterministic, replayable projection determines a functor

$$P_T : \mathbf{Prefix}_s \rightarrow \mathbf{Timeline}_s$$

by

$$P_T(n) = T_s(n)$$

and

$$P_T(m \rightarrow n) = \text{applySuffix}_{m,n} : T_s(m) \rightarrow T_s(n).$$

The functor laws become concrete replay laws.

Identity preservation:

$$\text{applySuffix}_{n,n} = 1_{T_s(n)}.$$

Composition preservation, for $\ell \leq m \leq n$:

$$\text{applySuffix}_{\ell,n} = \text{applySuffix}_{m,n} \circ \text{applySuffix}_{\ell,m}.$$

This is not decorative abstraction. It says that batching boundaries do not change semantics.

5.3 Chunking invariance

A stream processor may receive events one at a time, in batches, or after reconnect. If the projection is functorial over prefix arrows, every partition of the same suffix yields the same transition.

For

$$0 = n_0 < n_1 < \dots < n_k = n,$$

we require

$$P_T(0 \rightarrow n) = P_T(n_{k-1} \rightarrow n_k) \circ \dots \circ P_T(n_0 \rightarrow n_1).$$

Call this **chunking invariance**.

It can fail through:

- batch-local initialization;
- inconsistent transaction boundaries;
- hidden clocks or randomness;
- nonassociative aggregation;
- an event decoder that depends on surrounding frames;
- error policy that treats a batch failure differently from the same individual failures.

Chunking invariance is an excellent property-test target. Generate a canonical event list, partition it randomly, process each partitioning, and compare normalized outcomes.

5.4 Functors between schema categories

Consider a category **Schema**:

- objects are concrete message schemas;
- arrows are total migrations preserving declared semantics;
- composition is migration composition.

A code generator can act as a functor from schemas to generated Go types and functions if it preserves identity schemas and composition of migrations in the relevant sense.

A runtime schema registry is not automatically this functor. It maps logical names to prototypes, but name lookup is an operation internal to one category of registered contracts. Still, the functor viewpoint helps articulate the desired law for a version adapter:

```

Schema v1 -----migration-----> Schema v2
  |                                   |
  | generate                         | generate
  v                                   v
Go API v1 -----adapter-----> Go API v2

```

The square should commute up to an explicitly stated natural isomorphism or observational equivalence.

5.5 Forgetful functors

A forgetful functor removes structure while preserving the underlying arrows that remain meaningful.

Examples:

- typed protobuf messages $\rightarrow$ byte strings;
- timeline entities $\rightarrow$ (kind, id, ordinal) metadata;
- authenticated subscriptions $\rightarrow$ raw session subscriptions;
- heartbeat machine states $\rightarrow$ phase labels;
- ordered event histories $\rightarrow$ multisets of event names.

The final example is usually not acceptable for SessionStream invariants because it forgets order. But it is still a possible functor into a category where arrows preserve only multiset inclusion.

Forgetful functors reveal which theorems survive abstraction. If an invariant depends only on the forgotten image, it does not need the richer representation. If two semantically different executions become equal after forgetting, no downstream analysis in the target category can distinguish them.

5.6 Faithful and full functors

A functor is **faithful** if it is injective on each hom-set: distinct arrows remain distinct. It is **full** if every target arrow between images comes from a source arrow.

These terms refine the question “did the adapter lose information?”

- A faithful trace encoder preserves distinctions between allowed transitions.
- A full generated API exposes every target transformation that is supposed to correspond to a source migration.
- A functor can be faithful but not injective on objects: different object names may represent isomorphic or even equal target structures.
- A serialization functor may preserve all data values yet fail to be full because not every byte transformation corresponds to a valid semantic message transformation.

For an API SDK, fullness can be undesirable. The target language can express many functions that should not be accepted as domain arrows.

5.7 Projection as an action or algebra

The prefix-functor model treats complete suffix applications as arrows. At the single-event level, a projection is often better described as an **action**.

Let M be the monoid of event sequences under concatenation, with empty sequence ϵ . A deterministic reducer gives a monoid homomorphism

$$\alpha : M \rightarrow \text{End}(T),$$

where $\text{End}(T)$ is the monoid of endomorphisms on timeline state. The laws are

$$\alpha(\epsilon) = 1_T,$$

$$\alpha(xy) = \alpha(y) \circ \alpha(x)$$

under right-to-left composition convention.

Equivalently, the state space is an algebra for a suitable event-processing endofunctor. This language becomes valuable when reasoning about folds, free event histories, and replay.

The distinction is worth retaining:

- **Functor over prefixes:** emphasizes states at cuts and transitions between cuts.
- **Monoid action:** emphasizes how event sequences act on one state space.
- **Algebra/fold:** emphasizes recursive consumption of constructors.

Choose the model that matches the proof.

5.8 Contravariant functors as observation

A covariant functor follows arrows forward. A **contravariant functor** $F : \mathcal{C}^{op} \rightarrow \mathcal{D}$ reverses them. Given $f : A \rightarrow B$, it produces

$$F(f) : F(B) \rightarrow F(A).$$

Observation is often contravariant. An inclusion of a smaller context into a larger one produces restriction from data on the larger context to data on the smaller.

small context V ----inclusion----> large context U
 $F(V)$ <----restriction---- $F(U)$

A presheaf is precisely a contravariant functor to Set . We postpone the full construction until Chapter 9, but note the engineering pattern now:

When expanding the context makes more data available, forgetting back to the old context reverses the inclusion arrow.

This reversal is the source of much of the local-to-global geometry.

5.9 A current SessionStream functor candidate

The SQLite entity-version store supports snapshots as of a cut. Under suitable consistency assumptions, define

$$H_s : \text{Prefix}_s^{op} \rightarrow \text{Set}$$

where $H_s(n)$ is the set of possible snapshot states through n , and an arrow $m \rightarrow n$ in **Prefix** with $m \leq n$ induces restriction

$$H_s(n) \rightarrow H_s(m)$$

by selecting entity versions as of m .

To be functorial, temporal restriction must satisfy:

$$\rho_n^n = 1,$$

$$\rho_\ell^m \circ \rho_m^n = \rho_\ell^n \quad (\ell \leq m \leq n).$$

A store that keeps only current rows may not implement this restriction. The semantic history may still define a presheaf, but the deployed persistence interface would not expose it. This difference between **semantic functor** and **implemented functor** will recur.

5.10 Exercises

E5.1 - Laws (one star). Define the prefix category formally and verify associativity and identity laws.

E5.2 - Functor (one star). State the object and arrow parts of a timeline-prefix functor. Translate both functor laws into event-processing equations.

E5.3 - Counterexample (one star). Give a reducer for which processing $[a, b]$ as one batch differs from processing $[a]$ then $[b]$. Identify the failed functor law.

E5.4 - Property test (two stars). Implement random partition tests for chunking invariance. Include failures, tombstones, and duplicate deliveries.

E5.5 - Forgetting (two stars). Define three forgetful functors from transport traces. For each, list the invariants that are preserved and destroyed.

E5.6 - Full/faithful (two stars). Analyze protobuf JSON encoding as a functor on a carefully chosen category. Is it faithful? Full? Essentially surjective? State all restrictions needed for your answer.

E5.7 - Temporal restriction (three stars). Prove or falsify the compositional law for `Snapshot(asOf)` in the current SQLite store. Distinguish semantic correctness from transaction isolation during concurrent writes.

E5.8 - Model comparison (three stars). Formalize the same timeline projection as a prefix functor and as a monoid action. Show how the two models correspond.

5.11 Goldblatt bridge

Goldblatt Chapter 9 introduces covariant and contravariant functors after much of the basic topos structure. Read Sections 9.1 and 9.3 with this chapter. Focus on preservation of identities and composition; those two laws are the difference between a principled architecture translation and an arbitrary mapping.

Chapter 6

Natural Transformations as Coherent Refactoring

6.1 Comparing whole translations at once

Suppose $F, G : \mathcal{C} \rightarrow \mathcal{D}$ are two functors. A natural transformation

$$\eta : F \Rightarrow G$$

assigns to every object A an arrow

$$\eta_A : F(A) \rightarrow G(A)$$

such that every arrow $f : A \rightarrow B$ in $\mathcal{C}$ makes the naturality square commute:

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

$$\begin{array}{ccc} F(A) & \xrightarrow{\eta_A} & G(A) \\ | & & | \\ | F(f) & & | G(f) \\ \downarrow & & \downarrow \\ F(B) & \xrightarrow{\eta_B} & G(B) \end{array}$$

A natural transformation is not just a collection of per-type adapters. It is a family coherent with every allowed transition.

Software reading. A migration is natural when migrating before evolution gives the same result as evolving before migration.

6.2 Projection-version migration

Let $P_1, P_2 : \mathbf{Prefix}_s \rightarrow \mathbf{Timeline}$ be two timeline projection versions. A migration family

$$\eta_m : P_1(n) \rightarrow P_2(n)$$

is natural when, for every $m \leq n$,

$$P_2(m \rightarrow n) \circ \eta_m = \eta_n \circ P_1(m \rightarrow n).$$

This equation compares two upgrade strategies:

1. migrate the old state at cut m , then process new events with version 2;
2. process the suffix with version 1, then migrate the resulting state at cut n .

If the square commutes, rolling migration can happen at arbitrary cuts without changing semantics. If it does not, a full rebuild may be required.

The naturality test is stronger than comparing migration of one final snapshot. It quantifies over every transition in the source category.

6.3 Schema evolution and event migrations

Let E_1, E_2 be functors assigning versioned event representations to contexts, and let T_1, T_2 be corresponding timeline functors. There may be an event migration $\mu : E_1 \Rightarrow E_2$ and a state migration $\eta : T_1 \Rightarrow T_2$.

A projection compatibility condition becomes a larger commuting diagram:

```

old events -----old projection-----> old timeline
      |                                   |
      | event migration                   | state migration
      v                                   v
new events -----new projection-----> new timeline
    
```

The exact categorical setting may involve functor composition rather than one square, but the engineering requirement is clear:

$$\text{projectNew} \circ \text{migrateEvents} \cong \text{migrateState} \circ \text{projectOld}.$$

When this fails, the mismatch should be explicit. Perhaps the new projection intentionally derives information unavailable in old events. Then no natural migration can reconstruct it without another data source.

6.4 Instrumentation that should be natural

SessionStream transport observers record stages such as subscription, snapshot loading, buffering, sending, heartbeat events, and errors. Instrumentation is intended not to alter protocol behavior.

Model an uninstrumented execution functor F and an instrumented execution functor G whose states include an observation log. The forgetful component

$$\eta_A : G(A) \rightarrow F(A)$$

removes observations. Naturality requires that forgetting observations after an instrumented transition equals performing the uninstrumented transition after forgetting them.

```

instrumented before ----step----> instrumented after
      |                                   |
      | forget log                       | forget log
      v                                   v
plain before -----step-----> plain after
    
```

This is the categorical form of “observability must not change semantics.” Bounded best-effort dispatch complicates the log but should not change the protocol state after forgetting.

A callback that blocks a critical path, mutates shared protocol state, or changes scheduling can violate this naturality claim.

6.5 Natural isomorphism and representation independence

A natural transformation $\eta : F \Rightarrow G$ is a **natural isomorphism** if every component η_A is an isomorphism. Then F and G are the same translation up to a coherent representation change.

This is a strong formulation of representation independence.

Examples might include:

- timeline states stored as normalized protobufs versus equivalent relational rows;
- two client cache layouts with reversible conversion at every context;
- dense event ordinals versus another coordinate system with a reversible monotone translation;
- a renamed schema family with no semantic change.

A final-state bijection is insufficient. The conversions must commute with every transition.

6.6 Dinaturality, laxness, and the reality of distributed systems

Many software migrations do not commute exactly. They may commute only up to:

- canonical normalization;
- eventual convergence;
- a comparison arrow rather than equality;
- an error bound;
- a refinement relation;
- a coherent 2-cell in a bicategory.

This leads to lax natural transformations, enriched categories, profunctors, and higher categories. Those tools are real, but exact naturality should be tested first. It provides the baseline against which relaxation is measured.

For example, if a UI migration preserves semantics only after all events through the same cut are processed, naturality may hold on a subcategory of quiescent boundaries rather than on every prefix arrow.

6.7 The functor category

For categories $\mathcal{C}$ and $\mathcal{D}$, functors $\mathcal{C} \rightarrow \mathcal{D}$ can themselves be objects of a category

$$\mathcal{D}^{\mathcal{C}},$$

whose arrows are natural transformations.

This is a major conceptual step. Whole implementations become points in a design space, and coherent refactors become arrows between them.

For SessionStream, one might define a category whose objects are lawful timeline projection functors and whose arrows are natural migrations. Composition of migrations is componentwise:

$$(\theta \circ \eta)_A = \theta_A \circ \eta_A.$$

Identity migration is the family of identity arrows. The category laws follow from those in $\mathcal{D}$.

This perspective supports questions such as:

- Is there a canonical migration from every legacy projection into a normalized projection?
- Do two migration paths commute?
- Does a migration have an inverse?
- Is there a limit representing a projection satisfying several compatibility requirements?

6.8 Worked study: live and hydrated client paths

Define two functors from a category of execution intervals to client states:

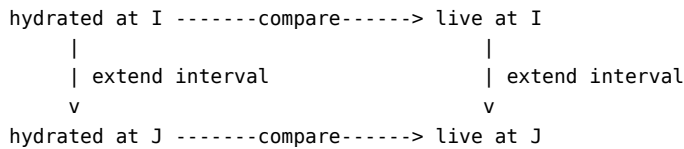
- L : observe the client continuously through live UI events;
- H : disconnect, hydrate from a snapshot at some cut, then consume the surviving suffix.

A family of comparison maps

$$\eta_I : H(I) \rightarrow L(I)$$

normalizes the hydrated client state to the continuously live representation.

Naturality says that after extending an execution interval, comparing the two client states is independent of whether extension happens before or after comparison.



If every η_I is an isomorphism, reconnect is transparent at all cuts. In practice the live UI path may contain ephemeral animations or progress signals intentionally absent from snapshots. Then the comparison might target a quotient of client states that forgets ephemeral details.

The lesson is to pick the codomain where the desired naturality is actually true.

6.9 Exercises

E6.1 - Naturality (one star). Write the naturality square and explain each path in words for a timeline-state migration.

E6.2 - Counterexample (one star). Define a migration that works at one final cut but is not natural over prefix extension.

E6.3 - Observer (two stars). Model instrumentation with a forgetful natural transformation. Identify one implementation choice that could make the square fail observationally.

E6.4 - Client paths (two stars). Define an equivalence relation that forgets ephemeral UI details. Determine whether hydration and continuous-live paths become naturally isomorphic after quotienting.

E6.5 - Testing (two stars). Generate random cut pairs $m \leq n$ and test a migration naturality square. Explain how to avoid sharing implementation code between the two paths.

E6.6 - Composition (two stars). Given migrations $v1 \rightarrow v2$ and $v2 \rightarrow v3$, prove their componentwise composite is natural. Translate the proof into a test strategy.

E6.7 - Laxness (three stars). Find a SessionStream comparison that commutes only up to an inequality of ordinals. Specify a preorder-enriched category in which the comparison can be expressed.

E6.8 - Design category (three stars). Define objects and arrows for a category of lawful projections. What is arrow equality? Which migrations are isomorphisms?

6.10 Goldblatt bridge

Read Goldblatt Section 9.2 on natural transformations and equivalence of categories. Reconstruct every naturality square using a software migration. The key idea is not “generic mapping” but coherence with all arrows in the source.

Chapter 7

Adjunctions, Free Histories, and Repair

7.1 The pattern of optimal translation

An adjunction relates functors

$$F : \mathcal{C} \rightleftarrows \mathcal{D} : G$$

through a natural bijection

$$\mathcal{D}(F(A), B) \cong \mathcal{C}(A, G(B)).$$

We write

$$F \dashv G$$

and say F is left adjoint to G , while G is right adjoint to F .

A left adjoint typically builds the freest object satisfying a structural requirement. A right adjoint typically extracts or organizes all compatible observations. These are tendencies, not definitions.

The bijection says that an arrow out of the constructed object $F(A)$ is the same data as an arrow from A into the underlying object $G(B)$, naturally in both variables.

7.2 Free monoids and event histories

The standard software-relevant example is the free-monoid adjunction.

Let $U : \mathbf{Mon} \rightarrow \mathbf{Set}$ forget monoid structure. Let $L : \mathbf{Set} \rightarrow \mathbf{Mon}$ send a set E to the monoid E^* of finite lists under concatenation.

Then

$$L \dashv U$$

because monoid homomorphisms

$$E^* \rightarrow M$$

correspond naturally to ordinary functions

$$E \rightarrow U(M).$$

A function telling how each individual event acts on a state extends uniquely to an action of every finite event history, provided composition is monoidal.

This is the mathematical genesis of a fold. You specify generator behavior; freeness supplies behavior on arbitrary sequences.

For SessionStream, event histories are not literally the free monoid on untyped events when session IDs, ordinals, schemas, and causal constraints matter. The free object should be built from **valid event generators and equations**. For example, ordinals may impose a path category rather than arbitrary concatenation.

7.3 Unit and counit as executable maps

Every adjunction has a unit

$$\eta : 1_{\mathcal{C}} \Rightarrow GF$$

and counit

$$\varepsilon : FG \Rightarrow 1_{\mathcal{D}}.$$

For the free-monoid adjunction:

- $\eta_E : E \rightarrow U(E^*)$ sends a generator to its singleton list;
- $\varepsilon_M : (U(M))^* \rightarrow M$ multiplies a list of monoid elements.

They satisfy the triangle identities, which express that freely adding structure and then interpreting it behaves coherently.

A software developer should look for these concrete maps. If an alleged adjunction has no plausible unit, counit, and triangle laws, it is probably only a metaphor.

7.4 A projection as interpretation of a free history

Let each canonical event determine an endomorphism of timeline state:

$$\phi : E \rightarrow \text{End}(T).$$

By freeness, ϕ extends uniquely to a monoid homomorphism

$$\overline{\phi} : E^* \rightarrow \text{End}(T).$$

Then replay is evaluation of $\overline{\phi}$ on a history.

This yields two useful proof obligations:

1. each event's meaning is represented by a state endomorphism;
2. history meaning is the unique homomorphic extension.

Hidden inputs violate the first obligation because the event no longer determines an endomorphism by itself. Nonassociative batching violates the second.

7.5 Reflection: repairing an object into a subcategory

Suppose $\mathcal{A}$ is a full subcategory of $\mathcal{C}$, with inclusion

$$I : \mathcal{A} \hookrightarrow \mathcal{C}.$$

If I has a left adjoint

$$R : \mathcal{C} \rightarrow \mathcal{A}, \quad R \dashv I,$$

then $\mathcal{A}$ is reflective and R is a **reflection**. It sends an arbitrary object to a universal corrected object in $\mathcal{A}$.

Software analogies include:

- normalize arbitrary records into canonical records;
- complete a partial configuration with defaults;
- quotient raw syntax into semantic equivalence classes;
- validate-and-repair a trace into a well-formed trace;
- transform a presheaf into its associated sheaf.

The last is literal mathematics: sheafification is left adjoint to the inclusion of sheaves into presheaves under standard hypotheses.

A validator that merely rejects invalid input is not a reflection. A repair is reflective only if every map from the raw object to a valid object factors uniquely through the repaired object.

7.6 Sheafification preview

Later we will define a category of presheaves $\mathbf{PSh}(\mathcal{C})$ and its full subcategory of sheaves $\mathbf{Sh}(\mathcal{C}, J)$. Under a Grothendieck topology J , the inclusion

$$I : \mathbf{Sh}(\mathcal{C}, J) \hookrightarrow \mathbf{PSh}(\mathcal{C})$$

has a left adjoint

$$a : \mathbf{PSh}(\mathcal{C}) \rightarrow \mathbf{Sh}(\mathcal{C}, J).$$

The unit

$$\eta_F : F \rightarrow I(aF)$$

maps local data into its sheafified form.

For software intuition, sheafification can be read as a universal completion that adds or identifies whatever is necessary so that compatible local data glues uniquely. This is not automatically a production repair algorithm. It depends on the chosen site and may introduce equivalence classes or idealized sections that do not correspond to existing storage rows.

7.7 Left and right adjoints preserve different structure

A central theorem is:

- left adjoints preserve colimits that exist;
- right adjoints preserve limits that exist.

This matters architecturally.

If a translation is a right adjoint, it tends to preserve products, pullbacks, equalizers, and terminal objects. Therefore consistency properties expressed by limits can survive the translation.

If a translation is a left adjoint, it tends to preserve coproducts, pushouts, coequalizers, and initial objects. Therefore freely assembled or quotient-like structure can survive.

A geometric morphism between topoi is organized around an inverse-image functor that is left exact - it preserves finite limits - and has a right adjoint. That combination is why geometric morphisms are relevant to translating local-to-global worlds without destroying their finite-limit logic.

7.8 Adjunctions versus codec pairs

An encoder/decoder pair is often described as an adjunction without justification. A round-trip law

$$\text{decode} \circ \text{encode} = 1$$

describes a retraction or isomorphism candidate, not an adjunction by itself.

To establish an adjunction, identify two categories, two functors, and a natural hom-set bijection. For example, a parser from strings to syntax trees may be partial and ambiguous; a printer may normalize. Their relation is more often a Galois connection, partial isomorphism, or lens than a categorical adjunction.

Use the word precisely. Adjunction is powerful because it packages a universal optimization, not because it sounds like “two-way conversion.”

7.9 Galois connections for parameter sufficiency

In posets, an adjunction is a Galois connection:

$$F(p) \leq q \iff p \leq G(q).$$

This can model information and requirements.

Let P be a poset of parameter sets ordered by inclusion, and I a poset of decidable invariants ordered by implication or information content. A closure operation sends supplied parameters to all facts derivable from them. Another operation sends an invariant to the weakest required information.

Under favorable conditions these form an adjunction:

$$\text{derive}(P) \models I \iff P \supseteq \text{requirements}(I).$$

Database functional-dependency closure is often a more direct tool than sheaf cohomology for the question “are these REST parameters enough?” The adjunction/Galois-connection view clarifies why: one side generates consequences, the other extracts requirements.

7.10 Exercises

E7.1 - Free monoid (one star). Starting from a function `Event -> End(Timeline)`, construct the unique list action. Verify the empty-list and concatenation laws.

E7.2 - Unit/counit (one star). Write the unit and counit of the free-monoid adjunction and verify one triangle identity by element calculation.

E7.3 - Hidden coordinate (two stars). Show why a projection using wall-clock time does not factor through the free monoid on recorded events alone. Repair the generator set.

E7.4 - Reflection (two stars). Propose a normalization operation on snapshots. State the universal factorization required for it to be a reflection, not merely a cleanup function.

E7.5 - Preservation (two stars). Choose a SessionStream adapter you suspect is a right adjoint. Test whether it preserves a terminal object and pullback. Revise or reject the claim.

E7.6 - Galois connection (two stars). Define parameter closure under functional dependencies and the required-attribute set of an invariant. Determine whether they form a Galois connection in your model.

E7.7 - Codec critique (three stars). Analyze protobuf encode/decode using isomorphisms, retractions, lenses, and adjunctions. State which description is defensible under which assumptions.

E7.8 - Sheafification preview (three stars). Invent a presheaf-like collection of partial configuration fragments that fails unique gluing. Describe informally what a universal repair would need to add or identify.

7.11 Goldblatt bridge

Goldblatt Chapter 15 develops adjunctions after local truth and sheaf theory. This text introduces the pattern earlier because free histories and later sheafification are useful anchors. Return to Goldblatt Sections 15.1-15.3 after Chapters 12-16 here; the abstract examples will then have concrete context.

Part III - Local-to-Global Data

Chapter 8

The Category of Observation Contexts

8.1 The base space is made of questions

Sheaf theory begins with a base on which information varies. In classical examples the base is a topological space and the contexts are open regions. In software, the base can instead be a category or poset of observation contexts.

A context answers:

- **which session?**
- **which interval or prefix of execution?**
- **which observers or representations are visible?**
- **which schema and protocol version?**
- **which authorization boundary?**
- **which assumptions about ordering, durability, and failure are in force?**

A useful first SessionStream context is

$$U = (s, I, K),$$

where:

- s is a session;
- $I \subseteq \mathbb{N}$ is an ordinal region, often a prefix or interval;
- $K \subseteq \{E, T, S, U, W, C, O\}$ is a set of visible lenses:
 - E : canonical event log;
 - T : timeline materialization;
 - S : snapshot representation;
 - U : projected UI batches;
 - W : WebSocket subscription state and buffer;
 - C : client state;
 - O : observer/trace records.

The context can be refined later with schema, tenant, replica, and policy coordinates.

8.2 Inclusion of contexts

Write

$$V \subseteq U$$

when V asks a smaller question than U . For the simple tuple model this means:

- the session agrees;
- $I_V \subseteq I_U$;
- $K_V \subseteq K_U$.

There is one arrow

$$V \hookrightarrow U$$

for every such inclusion. This makes the contexts a poset category.

Examples:

$$(s, [0, 40], \{E\}) \hookrightarrow (s, [0, 50], \{E, T, S\}),$$

$$(s, [0, 50], \{S\}) \hookrightarrow (s, [0, 50], \{E, T, S, U, W, C\}).$$

The arrow points from the smaller region to the larger region, matching the category of open sets in topology. Data will restrict in the opposite direction.

8.3 Several possible time geometries

The set of ordinals supports more than one useful topology or context order.

8.3.1 Prefix geometry

Contexts are prefixes $[0, n]$, ordered by inclusion. This models accumulated history. Every two prefixes are comparable, so the base is a chain. Chains have no interesting overlap loops by themselves.

8.3.2 Interval geometry

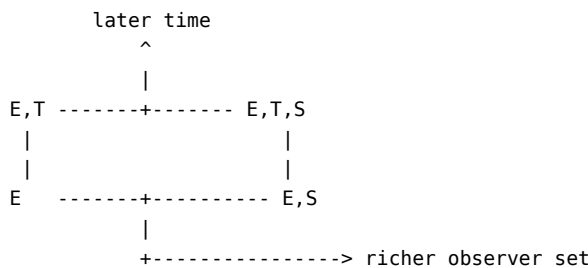
Contexts are intervals $[m, n]$, with inclusions. Overlaps can form richer patterns. This is useful for trace windows, batch processing, and local replay proofs.

8.3.3 Causal geometry

In distributed systems, contexts may be finite down-sets of a causal partial order rather than numeric intervals. Two observers can have incomparable knowledge. Their union may or may not be causally closed.

8.3.4 Observer-time product geometry

Take the product of an ordinal geometry with an observer/lens poset. Even when time is a chain, the observer dimension creates squares, cubes, and higher overlap patterns.



The multidimensional shape in this book primarily comes from these products and their overlaps, not from imagining source files embedded in ordinary three-dimensional space.

8.4 Contexts as regions of observability

Suppose the whole execution context is

$$X = (s, [0, 100], \{E, T, S, U, W, C, O\}).$$

Subcontexts include:

- database-only history $(s, [0, 100], \{E, T, S\})$;
- connection hydration window $(s, [75, 100], \{S, U, W, C\})$;
- heartbeat trace $(s, [90, 100], \{W, O\})$;
- event-prefix replay context $(s, [0, 80], \{E, T\})$.

An architecture does not need a component that literally stores X . The maximal context can be an ideal semantic object used to ask whether the local views admit one coherent completion.

This distinction is essential. A global section can represent a logically coherent execution even when no single process observes it atomically. Conversely, every process can hold locally valid state while no global section exists.

8.5 Overlaps

The overlap of two contexts is their meet when it exists:

$$U \cap V = (s, I_U \cap I_V, K_U \cap K_V).$$

The overlap contains exactly the questions both contexts can answer.

For example:

$$U = (s, [0, 50], \{E, T\}),$$

$$V = (s, [40, 70], \{T, S\}),$$

then

$$U \cap V = (s, [40, 50], \{T\}).$$

Compatibility on this overlap means that both views induce the same timeline information over ordinals 40 through 50.

If the lens intersection is empty, the contexts may still share session/cut/schema metadata. It is often wise to make boundary metadata an explicit lens rather than declaring the overlap empty.

8.6 Covers

A family of subcontexts $\{U_i \hookrightarrow U\}$ **covers** U when it collectively counts as enough local observation for the intended semantics.

In an ordinary topological space, open sets cover when their union is the region. In software, “enough” may depend on policy:

- intervals whose union contains every event ordinal;
- event and timeline views that jointly expose all facts needed by an invariant;
- a snapshot prefix and a live suffix with a common boundary;
- a quorum of replicas;
- API parameter groups whose union determines an operation;
- authorized views that collectively reveal a transaction without violating access policy.

A cover is not merely a set union. It declares which local evidence is admissible for reconstructing or verifying global information.

8.7 The topology is an engineering assumption

Choosing covers determines what the model treats as local and what it permits to be glued.

If every pair of components is declared a cover, pairwise consistency may be treated as enough even when a three-way invariant exists. If only atomic transaction scopes are covers, the model becomes stricter. If quorum subsets cover a replica set, the sheaf condition expresses a consistency protocol relative to quorum assumptions.

Therefore:

The topology of a software model is a formalization of its admissible evidence and reconstruction policy.

Changing the topology can turn the same presheaf into a sheaf or a nonsheaf. This is not cheating; it reveals that “local consistency implies global consistency” is meaningful only relative to a declared notion of locality.

8.8 A richer context record

For serious SessionStream work, use something like

$$U = (s, I, K, v, p, r),$$

where:

- v is a vector of schema/protocol versions;
- p is a policy set: ordering, retry, error, and consistency semantics;
- r is a replica/deployment region.

An inclusion $V \hookrightarrow U$ then needs explicit variance rules. For example:

- time and lens coordinates grow by inclusion;
- version may map by a migration rather than inclusion;
- policies may order by logical strength;
- replica contexts may order by containment or information flow.

When the coordinate orders do not form a simple product poset, use a general category. Multiple arrows between the same contexts can represent distinct restriction paths or adapters. That distinction will let cohomology detect path-dependent translation later.

8.9 Context design mistakes

8.9.1 Mistake 1: contexts are implementation modules

Packages are not automatically meaningful regions. Two packages may expose no common semantic information, while one package may contain several distinct observation contexts.

8.9.2 Mistake 2: every field defines a dimension

A field matters only if restriction and overlap have semantic meaning. Avoid a huge coordinate product with no lawful maps.

8.9.3 Mistake 3: later time always contains earlier state

A current materialized row may overwrite old state. Semantically, later history contains earlier prefixes; operationally, the store may not expose a restriction back to them.

8.9.4 Mistake 4: overlap means same field name

Two fields named `ordinal` can have different meanings. An overlap requires a common semantic object and arrows into it.

8.9.5 Mistake 5: global means centralized

A global section is a coherent assignment over the modeled whole, not necessarily one centralized data structure.

8.10 Exercises

E8.1 - Base poset (one star). Define the simple context order on triples (`session`, `interval`, `lenses`). Prove it is a partial order for a fixed session.

E8.2 - Overlap (one star). Compute five overlaps among event-log, timeline, snapshot, WebSocket, and client contexts. State the shared semantic facts, not only intersecting field names.

E8.3 - Time geometry (two stars). Compare prefix, interval, and causal contexts for modeling reconnect. Which shapes can contain nontrivial loops in their nerve?

E8.4 - Topology choice (two stars). Define two different coverage policies for the same replica system: strong consistency and quorum consistency. Explain how the choice changes the sheaf question.

E8.5 - Boundary lens (two stars). Add a boundary-metadata lens to the context model. Show how it changes the overlap of snapshot and strict live suffix contexts.

E8.6 - Version arrows (three stars). Replace the version coordinate order with a category containing multiple migrations between schema versions. Give an example where the migration path matters.

E8.7 - Architecture map (three stars). Build a complete context diagram for one real SessionStream subscription trace. Include every restriction or adapter that you intend to test.

8.11 Mental image checkpoint

Do not visualize a presheaf yet. Visualize only the base:

- points or regions are questions;
- inclusion means one question is narrower than another;
- overlap means two questions share a subquestion;
- a cover means several questions collectively count as enough to answer a larger one;
- loops arise from cycles of overlapping questions;
- higher-dimensional faces arise when three or more contexts have genuine joint overlap.

Chapter 9

Presheaves: Data That Restricts

9.1 Definition from the software operation

Suppose every context U has a set $\mathcal{F}(U)$ of possible observations, configurations, or local states. Whenever $V \hookrightarrow U$, data on U can be restricted to data on V :

$$\rho_V^U : \mathcal{F}(U) \rightarrow \mathcal{F}(V).$$

The operations must satisfy:

$$\rho_U^U = 1_{\mathcal{F}(U)},$$

$$\rho_W^V \circ \rho_V^U = \rho_W^U \quad (W \subseteq V \subseteq U).$$

Definition. A presheaf of sets on a category $\mathcal{C}$ is a functor

$$\mathcal{F} : \mathcal{C}^{op} \rightarrow \mathbf{Set}.$$

The first law says restricting to the same context does nothing. The second says forgetting in stages equals forgetting directly.

A member

$$s \in \mathcal{F}(U)$$

is a **section over** U . Its restriction to V is $s|_V$.

9.2 The presheaf of parameter assignments

Let the base category be finite sets of parameter names ordered by inclusion. For a parameter set U , define

$$\mathcal{A}(U) = \{\text{well-typed assignments to every parameter in } U\}.$$

If $V \subseteq U$, restriction drops the fields not in V .

This is the simplest software presheaf. The functor laws are ordinary record projection laws.

Example:

$U = \{\text{orderId}, \text{orderVersion}, \text{amount}, \text{currency}\}$
 $V = \{\text{orderId}, \text{amount}\}$
 $W = \{\text{orderId}\}$

Then

$\text{restrict } U \rightarrow W$
 $=$
 $\text{restrict } V \rightarrow W \text{ after } \text{restrict } U \rightarrow V.$

A global assignment is a value for every parameter in the maximal context. Local endpoint payloads are restrictions.

9.3 The presheaf of valid assignments

Now impose local constraints. Define

$$\mathcal{V}(U) = \{a \in \mathcal{A}(U) \mid a \text{ satisfies all constraints visible in } U\}.$$

For $\mathcal{V}$ to be a presheaf, validity must be preserved by restriction. This is not automatic.

Suppose a full record must satisfy $\text{amount} = \text{price} * \text{quantity}$. If a smaller context retains amount but forgets price and quantity, what does “valid” mean there? Possible choices:

1. the smaller assignment is valid if it has **some** global completion satisfying the equation;
2. it is valid only if the visible fields themselves can verify every relevant constraint;
3. constraints not expressible in the smaller context are ignored;
4. the smaller section carries a proof or summary of the forgotten constraint.

Each choice defines a different presheaf. The correct one depends on the question.

The local-to-global approach forces this ambiguity into the open.

9.4 Fibres and parameter sufficiency

Let X be the maximal semantic parameter context and $P \subseteq X$ the supplied API parameters. Restriction gives

$$r_P : \mathcal{V}(X) \rightarrow \mathcal{V}(P).$$

For a payload $p \in \mathcal{V}(P)$, its fibre is

$$r_P^{-1}(p) = \{x \in \mathcal{V}(X) \mid x|_P = p\}.$$

Interpretation:

- empty fibre: the payload has no valid global completion;
- singleton fibre: the payload determines one global state;
- multiple-element fibre: the payload is underdetermined.

For a particular invariant $I : \mathcal{V}(X) \rightarrow Q$, parameters are sufficient to decide I at payload p when

$$I(x) = I(y) \quad \text{for all } x, y \in r_P^{-1}(p).$$

This is the precise presheaf/fibre version of the original API question. It is often more useful than cohomology.

9.5 The event-history presheaf

Let contexts be ordinal intervals. Define $\mathcal{E}(I)$ to be the set of all valid event assignments on interval I . A section chooses an event for each ordinal in the interval, respecting session and schema constraints. Restriction truncates the assignment to a subinterval.

This is a presheaf because restricting a function by domain satisfies identity and composition.

If we fix one actual execution, we obtain a distinguished compatible family of sections, perhaps one section per interval. If event identity is stable and overlaps agree, these sections can glue to a global event history.

The presheaf of **possible** histories is more informative than the single actual history because it exposes ambiguity and constraints.

9.6 The snapshot-history presheaf

For prefix context $[0, n]$, define

$$\mathcal{S}(n) = \{\text{sound and complete snapshots as of } n\}.$$

For $m \leq n$, temporal restriction should produce

$$\rho_m^n : \mathcal{S}(n) \rightarrow \mathcal{S}(m).$$

How can a snapshot at n be restricted to m ? A current-state snapshot alone may not contain enough history. The operation can instead use:

- canonical event replay;
- a versioned entity store;
- inverse patches;
- persistent data structures;
- a proof-carrying snapshot that retains history.

Thus $\mathcal{S}$ may exist semantically but not be computed from the raw Snapshot value. Presheaf structure can reveal a missing API capability.

The SQLite store's entity-version table and asOf query are close to implementing this temporal restriction. A current-row-only store would not.

9.7 The presheaf of trace observations

For a context U consisting of a set of transport stages and time interval, let

$$\mathcal{O}(U) = \{\text{observer traces visible in } U\}.$$

Restriction drops records outside the interval or stage set. This can be a presheaf even when the observer is best-effort, provided sections represent what was recorded rather than the complete protocol truth.

Do not conflate:

$$\mathcal{O}(U) = \text{observed records}$$

with

$$\mathcal{P}(U) = \text{actual protocol transitions}.$$

There may be a natural transformation

$$\mathcal{P} \Rightarrow \mathcal{O}$$

that forgets unobserved details, but dropped observer records mean it is not generally injective.

9.8 The presheaf of local invariants

Let $\mathcal{F}$ be a presheaf of states. A property P defines a subpresheaf $\mathcal{P} \hookrightarrow \mathcal{F}$ when:

- $\mathcal{P}(U) \subseteq \mathcal{F}(U)$ for every context;
- if $s \in \mathcal{P}(U)$, then $s|_V \in \mathcal{P}(V)$ for every $V \subseteq U$.

That is, truth of the property is stable under restriction.

Examples:

- “all visible event payloads match registered schemas” is restriction-stable;
- “every visible entity ordinal is at most the visible cut” is stable if the cut remains visible;
- “the session contains an event named Finished” is not stable under arbitrary subinterval restriction;
- “the client will eventually reconnect” is not a local state property at all without temporal semantics.

This stability requirement anticipates the internal logic of a topos. Not every ordinary proposition defines a subobject in every context category.

9.9 Compatible families

Let $\{U_i \rightarrow U\}$ be a cover. A family

$$s_i \in \mathcal{F}(U_i)$$

is **compatible** or **matching** when for every pair i, j , the restrictions agree on the overlap:

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}.$$

Compatibility is pairwise equality after forgetting to shared information.

For three contexts, pairwise agreement does not necessarily guarantee that a global section exists. It only says the family is a candidate for gluing. Higher-order constraints may remain.

In a database analogy, every pairwise join may be nonempty while the full multiway join is empty.

9.10 Global sections and completions

If X is a maximal context, a global section is simply an element of $\mathcal{F}(X)$. Categorically, it is a natural transformation from the terminal presheaf 1 to $\mathcal{F}$.

Given local sections s_i , a global completion is $s \in \mathcal{F}(X)$ such that

$$s|_{U_i} = s_i$$

for every i .

There may be:

- no completion: obstruction or inconsistency;
- one completion: determinate gluing;
- many completions: underdetermination;
- completions that differ only by a chosen equivalence: uniqueness after quotienting.

This four-way distinction is more informative than a Boolean “consistent/inconsistent” flag.

9.11 Separated presheaves

A presheaf is **separated** for a coverage when two sections over U that agree on every member of a cover must be equal.

Formally, if

$$s|_{U_i} = t|_{U_i} \quad \text{for all } i,$$

then $s = t$.

Separatedness is the uniqueness half of the sheaf condition.

A software presheaf fails separatedness when global information is invisible in all local views. Example: two transaction records differ only by a global audit identifier, but no context in the declared cover contains that identifier. They restrict identically everywhere and therefore cannot be distinguished locally.

The repair options are:

- expose the coordinate on at least one cover member or overlap;
- quotient by the invisible distinction;
- weaken the cover;
- accept nonunique gluing.

9.12 Hidden coordinates as presheaf defects

Suppose two replay runs have identical event sections on every declared context but different output because a projector reads a global variable. The local data cannot determine the result. One of two things is wrong:

- the presheaf of inputs omitted a coordinate;
- the projection is not a natural transformation of the chosen presheaves.

This gives a disciplined debugging question:

Which context must be enlarged so that the divergent outputs restrict differently somewhere?

Adding wall-clock time, random seed, model version, tenant policy, or feature flag may restore determinacy. Alternatively, move that input into canonical event data.

9.13 Exercises

E9.1 - Presheaf laws (one star). Prove that parameter assignment with field projection is a presheaf.

E9.2 - Validity choices (one star). For the constraint `amount = price * quantity`, define three different valid-assignment presheaves and compare their sections on `{amount}`.

E9.3 - Fibres (one star). Construct a payload whose fibre of valid completions is empty, singleton, and multiple. Relate each case to API behavior.

E9.4 - Temporal restriction (two stars). Define `restrictSnapshot(n,m)` using entity versions. Prove the identity and composition laws under a fixed database history.

E9.5 - Subpresheaf (two stars). Decide which SessionStream invariants are stable under your context restrictions. Modify the context model for one invariant that is not.

E9.6 - Separatedness (two stars). Invent two distinct global execution records that restrict identically to the current component cover. Decide whether to expose or quotient the distinction.

E9.7 - Global completion (three stars). Encode a small multiway consistency problem as local sections over three contexts. Make all pairwise overlaps agree while the global completion set is empty.

E9.8 - Implementation (three stars). Design Go interfaces for a finite presheaf: contexts, inclusions, sections, and restriction maps. Add runtime checks for the functor laws.

9.14 Goldblatt bridge

Goldblatt's explicit functor language appears in Chapter 9, while his sheaf development in Chapters 4 and 14 uses varying families, restrictions, and local sections. Use the formal presheaf definition here to unify those presentations.

Chapter 10

Bundles, Fibres, Germs, and Sections

10.1 Goldblatt's geometric entry point

Goldblatt introduces bundles before sheaves: start with many sets indexed by a base, place each set over its index, take their disjoint union, and obtain a projection to the base. This gives a visual object before restriction maps and gluing become abstract.

Let B be a set of base points. For each $b \in B$, let E_b be a set, with fibres made disjoint by tagging if necessary. Define the total space

$$E = \coprod_{b \in B} E_b$$

and projection

$$p : E \rightarrow B, \quad p(e) = b \text{ when } e \in E_b.$$

Then

$$E_b = p^{-1}(b)$$

is the **fibre** or **stalk** over b . An element of a fibre is a **germ** at b in Goldblatt's introductory terminology.

A section is a map

$$s : B \rightarrow E$$

with

$$p \circ s = 1_B.$$

It chooses one element in every fibre.

10.2 A SessionStream bundle over sessions

Take the base to be active session IDs. Over session s , place the set of valid timeline states:

$$E_s = \{\text{valid timeline states for } s\}.$$

The total space consists of pairs

$$(s, T)$$

with projection $p(s, T) = s$.

A section chooses one valid timeline state for every session. A running hydration store is an implementation of part of such a section, assuming it has one current materialization per session.

This picture helps distinguish:

- one fibre: alternative states for a fixed session;
- one point in the fibre: the current materialized state;
- a section: a coherent selection across all sessions;
- the total space: every session-state possibility.

If tenant policy couples sessions, an arbitrary choice of one state per fibre may not be globally valid. Then the simple bundle omits constraints between fibres.

10.3 A bundle over session-cut pairs

Use a richer base

$$B = \{(s, n) \mid s \text{ a session, } n \text{ an ordinal cut}\}.$$

The fibre over (s, n) can be:

$$E_{(s, n)} = \mathcal{S}_s(n),$$

the set of sound and complete snapshots at that cut.

A section over a subset of the base chooses one snapshot for each session-cut pair. A deterministic replay semantics may produce a distinguished section

$$(s, n) \mapsto S_s(n).$$

Temporal restriction relates fibres over different cuts. The bundle alone only places sets over points; a presheaf additionally supplies the maps between them.

10.4 Fibres over observer contexts

Let a base point be

$$b = (s, n, K),$$

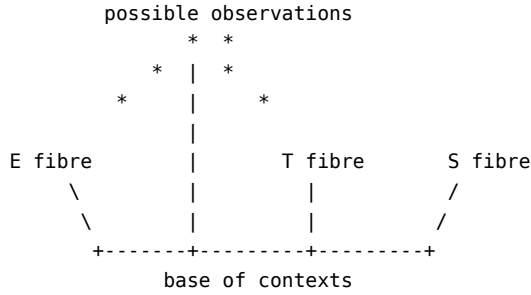
an exact session, cut, and observer lens. The fibre E_b contains every locally valid observation in that context.

Examples:

- over $(s, n, \{E\})$: valid event prefixes;
- over $(s, n, \{T\})$: valid timeline views;
- over $(s, n, \{S\})$: snapshots;
- over $(s, n, \{W\})$: WebSocket subscription states;
- over $(s, n, \{E, T, S\})$: compatible triples.

Moving across the observer dimension changes the fibre type. Restriction maps transport germs from richer fibres to poorer ones.

This is the multidimensional picture:



10.5 Local sections

In topology, a section need not be defined on the entire base. A **local section** over a region $U \subseteq B$ chooses a germ in every fibre over U , continuously or compatibly according to the structure.

Software examples:

- one connection’s observations over its subscription interval;
- one replica’s timeline states over the cuts it has materialized;
- one schema version’s interpretation over the event kinds it understands;
- one endpoint’s assignments over the parameters it receives;
- one test trace’s witness over the states it visits.

Local sections are first-class objects. Sheaf theory is not a method for pretending they are already global. It studies their restrictions, compatibility, and possible gluing.

10.6 Germs as local behavior near a point

In modern sheaf theory, the stalk $\mathcal{F}_x$ at a topological point x is built from sections on neighborhoods of x , identifying sections that agree on some smaller neighborhood. A germ records behavior “near x ” without remembering the entire neighborhood.

For an ordinal-time model, a point might be a cut n . A germ could represent projection behavior in some interval around n , modulo agreement on a smaller interval.

For a SessionStream transition, two implementations define the same germ at n if they agree on sufficiently local traces around that cut, even if they differ elsewhere.

This can formalize local bug signatures:

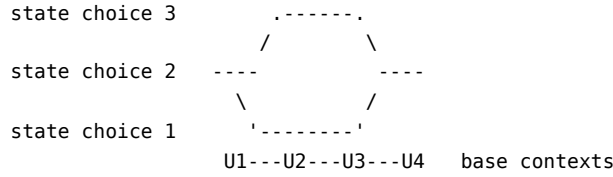
- a hydration race germ near subscription transition;
- an off-by-one germ near a snapshot cut;
- a stale-pong germ near heartbeat generation rollover.

Stalks deliberately forget global context. Cohomology and global sections study what this local forgetting cannot resolve.

10.7 Display spaces and the “shape above the base”

Under suitable conditions, a sheaf can be represented by an étale or display space $p : E \rightarrow B$ whose local sections recover the sheaf. The total space may have many sheets over the same base region.

For software intuition, imagine all valid local states floating above each context. Restriction and local continuity connect nearby possibilities. A global section is a path or surface selecting compatible possibilities everywhere.



A branch point suggests underdetermination. A missing continuation suggests obstruction. A loop that returns on another sheet suggests monodromy or holonomy. These pictures are intuition, not substitutes for the actual restriction maps.

10.8 Sections and dependency injection

A dependency-injection container chooses one implementation for each service interface. This resembles a section of a bundle of implementations over interface labels.

But a valid application requires compatibility:

- serializer and schema registry agree;
- store and projection cursor semantics agree;
- WebSocket protocol and client reducer agree;
- authorization and session routing agree.

A plain bundle allows arbitrary independent choices. A sheaf-like model adds overlap constraints. A global section then represents a coherent architecture configuration, not merely one implementation per interface.

This is a useful general software pattern:

interfaces as base points, implementations as fibres, wiring as a section, integration laws as gluing constraints.

10.9 Sections as generalized elements

In a category with terminal object 1 , an element of A is an arrow

$$1 \rightarrow A.$$

For a bundle $p : E \rightarrow B$, a section is an arrow in the slice category $\mathbf{Set}/B$ from the terminal object $1_B : B \rightarrow B$ to $p : E \rightarrow B$.

This unifies three ideas:

- ordinary value: choose one point of a set;
- bundle section: choose one point in every fibre;
- sheaf global section: choose local data coherently over the whole base.

A topos treats objects as generalized sets, so their elements are generalized sections rather than necessarily ordinary points.

10.10 Exercises

E10.1 - Bundle (one star). Build the bundle of current timeline states over three session IDs. Identify total space, projection, fibres, and one section.

E10.2 - Session-cut fibre (one star). For a toy event history, list every valid snapshot in the fibre over each cut. When is the fibre empty or multi-valued?

E10.3 - Local section (two stars). Represent one WebSocket connection trace as a local section over a time interval. What base points and fibre values are required?

E10.4 - Germ (two stars). Define an equivalence relation on projection implementations that captures agreement near a cut. Check reflexivity, symmetry, and transitivity.

E10.5 - Wiring (two stars). Model a SessionStream deployment as a section of implementation choices. Add compatibility constraints and find one locally selectable but globally invalid wiring.

E10.6 - Slice category (three stars). Prove that sections of $p: E \rightarrow B$ correspond to arrows from $\text{id}_B: B \rightarrow B$ to p in Set/B .

E10.7 - Visualization (three stars). Draw a display-space picture for a schema migration with two possible completions over one version boundary. Explain which branches are removed by additional event metadata.

10.11 Goldblatt bridge

Read Goldblatt Section 4.5 through his construction of bundles, fibres, total space, and sections. His concrete visual route is especially valuable here. Then reinterpret each fibre as a set of possible SessionStream observations rather than a set of ordinary geometric germs.

Chapter 11

Sheaves: Compatible Data That Glues

11.1 The missing axiom

A presheaf knows how to restrict. It does not promise that compatible local sections come from a global section.

Let $\{U_i \rightarrow U\}$ be a cover and let

$$s_i \in \mathcal{F}(U_i)$$

be a matching family:

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j} \quad \text{for all } i, j.$$

Definition. A presheaf $\mathcal{F}$ is a sheaf when every matching family has a unique amalgamation $s \in \mathcal{F}(U)$ satisfying

$$s|_{U_i} = s_i$$

for every i .

The axiom has two independent parts:

- **local identity / separatedness:** at most one amalgamation;
- **gluing / existence:** at least one amalgamation.

A sheaf has exactly one.

11.2 The event-interval sheaf

Let $\mathcal{E}(I)$ be valid event assignments on ordinal interval I , with restriction by truncation. Suppose intervals I_i cover I , and local histories agree on every overlap.

Define the global history pointwise: for ordinal $n \in I$, choose any I_i containing n and set

$$e(n) = e_i(n).$$

Overlap agreement makes this well-defined. The resulting history restricts to every e_i , and it is unique because every ordinal lies in some cover member.

Thus event assignments form a sheaf under ordinary interval coverage, provided “valid” constraints are local and preserved by union.

If validity includes a nonlocal condition such as “the total number of `Retry` events is even,” locally valid intervals may glue to a globally invalid history. Then the valid-history presheaf is not a sheaf for that coverage.

The failure tells us the invariant is not local with respect to those intervals.

11.3 Snapshot and suffix as a sheaf-like cover

Define a whole reconnect context $R_s(m)$ and two covering contexts:

- prefix snapshot context $P_s(n)$;
- live suffix context $L_s(n, m)$.

Their pullback/overlap is the boundary context $B_s(n)$.

$$\begin{array}{ccc}
 B_s(n) & \text{-----}> & L_s(n, m) \\
 | & & | \\
 | & & | \\
 \vee & & \vee \\
 P_s(n) & \text{-----}> & R_s(m)
 \end{array}$$

A section on $P_s(n)$ contains:

- snapshot state;
- session ID;
- cut ordinal;
- schema/reducer version;
- evidence or assumption that the snapshot represents the prefix.

A section on $L_s(n, m)$ contains:

- ordered live UI batches;
- session ID;
- base cut;
- schema/reducer version.

They match when their boundary restrictions agree.

A glued section on $R_s(m)$ is a reconstructed client execution. The sheaf claim is:

Every compatible snapshot and live suffix determine one client execution, up to the declared client-state equivalence.

This claim requires deterministic client reducers and an explicit policy for duplicates, errors, and ephemeral events.

11.4 How the WebSocket buffer realizes gluing

The current server performs an operational gluing algorithm:

1. Register the subscription in a hydrating state.
2. Load the snapshot.
3. Buffer UI batches arriving during the load.
4. Send the snapshot.
5. Discard buffered batches whose ordinal is already represented by the snapshot cut.
6. Sort/flush surviving batches.
7. Flush late batches while preventing newer live batches from overtaking them.
8. Mark the subscription live.

This algorithm attempts to construct the unique amalgamation of the prefix section and concurrent suffix section.

The finite buffer limit is a partiality boundary. On overflow, the server reports an error and closes rather than returning an alleged global section with missing data. Mathematically, the implementation refuses the gluing operation outside its bounded domain.

The contract does not replay arbitrary missed UI events from a prior disconnected period. The current transport treats `sinceSnapshotOrdinal` as advisory and sends a current durable snapshot followed by future live events. Therefore the sheaf being modeled should concern **durable reconstructed client state plus post-subscription live behavior**, not recovery of every ephemeral UI event ever missed.

11.5 Existence failures

A matching family can fail to glue for several reasons.

11.5.1 Global constraint

Three projections agree pairwise on shared fields, but no state satisfies all three constraints simultaneously.

11.5.2 Missing event identity

Local delivery contexts cannot determine whether two retries are one logical event or two events. Multiple candidate quotients exist, or no chosen policy applies.

11.5.3 Hidden dependency

Projection sections agree on recorded inputs but were computed under different unrecorded feature flags.

11.5.4 Incompatible schemas

Snapshot payloads and live UI payloads use versions with no common migration at the boundary.

11.5.5 Temporal gap

The snapshot ends at n , but the earliest available suffix begins at $n + 2$.

11.5.6 Nonassociative reducer

The same suffix applied in different buffering chunks yields different client state.

Each failure should be represented as a failed condition, not casually called a “cohomology class.” Cohomology applies after a suitable algebraic sheaf or complex is defined.

11.6 Uniqueness failures

A compatible family may have several global completions.

Examples:

- API parameters omit `orderVersion`, allowing several price histories;
- snapshots omit a stable logical event ID, allowing different retry groupings;
- local views hide tenant metadata, allowing multiple global tenant assignments;
- all cover members see only normalized payloads, while raw encodings differ;
- the UI trace does not determine which of several backend event histories produced it.

If the distinctions are intentionally unobservable, quotient global states by observational equivalence. If they matter, enrich the overlaps or contexts until the presheaf becomes separated.

11.7 Gluing and database joins

For a relational schema, local tables or projections can be treated as sections over attribute sets. Restriction is relational projection. Compatibility means projections agree on shared attributes. A global relation or tuple is an amalgamation.

The sheaf condition resembles a lossless-join property:

$$R = \bowtie_i \pi_{U_i}(R)$$

for the relevant relation and cover.

But be precise:

- a sheaf of individual assignments concerns joining tuples;
- a sheaf of relations concerns sets of tuples and may require different conditions;
- pairwise consistency does not imply a nonempty full join;
- functional dependencies and join dependencies are often the direct database tools.

SessionStream snapshot construction is similar: entity-version rows are local facts that must join through session, entity identity, schema, and cut constraints.

11.8 A sheaf of deterministic projection behavior

Let $\mathcal{P}(I)$ be projection behaviors over event interval I : functions taking an allowed boundary state to an output state and trace. Restriction to a subinterval should extract local behavior.

If local projection behaviors agree on overlaps and compose associatively, they may glue to behavior over the union. This sheaf captures modular replay reasoning.

However, extracting a subinterval behavior from a stateful fold may require boundary states. Therefore sections should include input/output interfaces:

$$s_I : T_{\min I-1} \rightarrow T_{\max I}.$$

Compatibility on adjacent intervals requires the output boundary of one to match the input boundary of the next. This is closer to a category of processes or a cosheaf of composed transitions than an ordinary set-valued sheaf. The example shows why choosing sections correctly matters.

11.9 Sheaf condition as an integration test generator

Given a declared cover, the sheaf axiom directly generates tests:

1. Generate local sections satisfying local validity.
2. Restrict every pair to overlaps.
3. Keep matching families.
4. Attempt global assembly.
5. Check existence.
6. Enumerate or vary assemblies to check uniqueness.
7. Restrict the result back and compare.

For snapshot/live:

- generate snapshot cuts and concurrent batches;
- ensure boundary metadata agrees;
- run the transport gluing algorithm;

- compare to a reference sequential reducer;
- search for a second distinct client execution with the same local restrictions.

The sheaf perspective therefore turns an abstract axiom into a systematic property-based testing plan.

11.10 Local versus global correctness

A sheaf does not mean “the system is correct.” It means one chosen kind of local data glues uniquely for one chosen coverage.

You can have:

- a sheaf of malformed payloads;
- a sheaf whose global sections violate a business rule not encoded locally;
- a sheaf over a topology too weak to expose relevant overlaps;
- a correct presheaf that is intentionally not a sheaf because global choices remain.

Always state:

(base category, coverage, section type, restriction maps, equality).

Only then ask whether it is a sheaf.

11.11 Exercises

E11.1 - Event sheaf (one star). Prove the event-assignment presheaf is a sheaf for interval covers.

E11.2 - Nonlocal constraint (one star). Add a global parity constraint and construct locally valid matching histories whose union is invalid.

E11.3 - Hydration cover (two stars). Define the prefix, suffix, boundary, and whole contexts precisely. Write all restriction maps.

E11.4 - Existence proof (two stars). Under deterministic, associative client reduction and complete suffix delivery, prove that a compatible snapshot/suffix pair has an amalgamation.

E11.5 - Uniqueness proof (two stars). State an observational equivalence under which the amalgamation in E11.4 is unique. Show why raw UI animation state may violate uniqueness.

E11.6 - Buffer overflow (two stars). Model overflow as a partial gluing operation. Compare fail-closed, resnapshot, disk-spill, and suffix-replay repairs.

E11.7 - Join dependency (three stars). Translate a three-context sheaf condition into a database join dependency. Produce a counterexample where all pairwise joins are nonempty but the full join is empty.

E11.8 - Test harness (three stars). Implement the seven-step sheaf-condition test generator for a finite context cover.

11.12 Goldblatt bridge

Goldblatt Section 4.5 introduces sheaves through bundles and local sections; Chapter 14 returns with restrictions, compatibility, covers, and local truth. Read his sheaf axiom after proving the event-interval example yourself. The definition should feel like a formal version of a merge algorithm with overlap checks.

Chapter 12

Sites, Coverage Policies, and Sheafification

12.1 Why a topology on points is sometimes too rigid

Classical sheaves live on open sets of a topological space. Software contexts may not arise from literal point sets, and coverage may depend on structured families rather than union of regions.

A **site** consists of a category $\mathcal{C}$ together with a Grothendieck topology J , or equivalently in many practical presentations a pretopology of covering families.

A covering family of an object U is a collection

$$\{U_i \rightarrow U\}$$

declared sufficient to reconstruct local data on U .

The coverage must behave coherently under identity, pullback, and refinement.

12.2 Pretopology axioms in engineering language

A common pretopology formulation requires:

12.2.1 Identity

The singleton family $\{1_U : U \rightarrow U\}$ covers U .

Engineering reading: observing the whole context is enough to observe the whole context.

12.2.2 Stability under pullback

If $\{U_i \rightarrow U\}$ covers U and $V \rightarrow U$ is another context map, then the pulled-back family

$$\{V \times_U U_i \rightarrow V\}$$

covers V .

Engineering reading: a valid observation plan remains valid when restricted to a subsystem, tenant, interval, or schema slice.

12.2.3 Transitivity

If $\{U_i \rightarrow U\}$ covers U , and each U_i is covered by $\{V_{ij} \rightarrow U_i\}$, then the composite family $\{V_{ij} \rightarrow U\}$ covers U .

Engineering reading: replacing every local observer by a sufficient local implementation still yields sufficient global coverage.

These axioms prevent “cover” from being an arbitrary label.

12.3 A temporal SessionStream site

Objects:

- session-ordinal intervals with boundary metadata.

Arrows:

- interval inclusions preserving session and version information.

Covers:

- families of intervals whose union is the target interval and whose boundary conventions agree.

The event-assignment presheaf is a sheaf on this site. A projection-behavior presheaf may require covers to overlap at enough boundary state to compose.

For reconnect, declare

$$\{P_s(n) \rightarrow R_s(m), L_s(n, m) \rightarrow R_s(m)\}$$

a cover only when the live section includes a reliable base-cut reference and all required suffix batches are available.

This prevents the topology from asserting a reconstruction capability the transport does not provide.

12.4 A component-observation site

Objects:

- combinations of event log, timeline, snapshot, WebSocket, client, and observer contexts.

Arrows:

- semantic projections/forgetful maps.

Possible covers:

- {event log, timeline} covers replay-consistency context;
- {snapshot, entity versions} covers historical materialization context;
- {snapshot, WebSocket suffix} covers reconstructed client context;
- {protocol state, observer records} may *not* cover actual execution if records can drop.

This site makes architecture assumptions explicit. If best-effort observations are declared a cover of protocol truth, the sheaf condition will fail or produce false certainty.

12.5 Quorum coverage

Let an object represent a replicated value across N replicas. Declare any quorum of size q to cover the replica set.

A sheaf for this coverage would say compatible quorum observations glue uniquely to a global value. This requires intersection properties and value/version rules.

For majorities, any two quorums intersect when

$$2q > N.$$

But intersection alone does not guarantee compatibility: replicas may hold concurrent versions. Sections need version vectors, ballots, or another consistency witness. The topology says which families count as enough; the sheaf says whether the data and protocol make them sufficient.

Pullback stability asks whether restricting a quorum cover to a subset of replicas still yields an admissible cover. The answer depends on how the base category is designed. Quorum systems are a useful reminder that a naive subset category may not encode the intended coverage laws.

12.6 Authorization-sensitive coverage

Suppose a global transaction can be reconstructed from payment, order, and shipping contexts, but one principal is authorized to see only order and shipping.

There are two possible models:

1. keep the full site but restrict available sections by authorization;
2. define a principal-specific subsite whose covers contain only authorized contexts.

The second model may make a previously global invariant undecidable because no authorized cover exposes enough information. That is a feature: the model distinguishes truth from knowability under access constraints.

A dangerous design declares unauthorized data part of an overlap implicitly. A safe site includes authorization in the context coordinate or in the morphisms.

12.7 Coverage for REST parameter sufficiency

Let objects be semantic parameter sets. A family $\{P_i \subseteq X\}$ covers X when the selected parameter groups collectively determine the required invariant under declared functional dependencies.

For a transaction invariant I , define a coverage J_I rather than one universal topology. A family covers when

$$\bigcup_i P_i$$

has closure containing every attribute needed to decide I .

The assignment presheaf is a sheaf for ordinary union covers. The presheaf of **valid transaction states** may fail to be a sheaf if local constraints do not capture a global constraint. That failure pinpoints information or coordination missing from the API surface.

12.8 Sieves

A **sieve** S on U is a collection of arrows into U closed under precomposition. If $V \rightarrow U$ belongs to the sieve and $W \rightarrow V$, then $W \rightarrow U$ also belongs.

In a context poset, a sieve is a downward-closed family of subcontexts.

A covering sieve contains enough arrows to count as a cover. Grothendieck topologies are often defined by assigning to every object the set of its covering sieves.

Software reading:

A sieve is a stable set of ways to inspect a context. Once an inspection method is accepted, every more local restriction of it is accepted too.

Sieves become the truth values of a presheaf topos in Chapter 14.

12.9 Sheafification

Given a presheaf F that does not glue, sheafification constructs an associated sheaf aF and a natural map

$$\eta_F : F \rightarrow aF$$

universal among maps from F to sheaves.

A useful informal two-stage picture for set-valued presheaves is:

1. **matching-family completion:** treat compatible local families as candidate new sections;
2. **local identification:** identify candidate sections that agree after passing to a sufficiently fine cover.

Repeating the completion/identification process yields a sheaf under standard conditions.

The unit η_F can:

- identify globally distinct sections that are locally indistinguishable;
- add global sections represented only by compatible local data;
- do both.

This is why sheafification is more than validation.

12.10 Software analogies for sheafification

Potential analogies include:

- merge configuration fragments and quotient irrelevant ordering;
- construct a canonical client state from local component views;
- add missing transition witnesses to a trace model;
- normalize versioned schemas through local migrations;
- complete a partially specified distributed state with equivalence classes of local representatives.

But sheafification is defined by a universal property relative to a site. A concrete reconciliation algorithm is sheafification only if it realizes that property. Many production repairs choose priorities, discard conflicts, or consult external truth; those are policyful operations, not universal sheafification.

12.11 Choosing the weakest useful topology

A very fine topology has many covers and therefore imposes a strong sheaf condition. A coarse topology has few covers and makes sheafhood easier.

For engineering diagnostics:

1. start with covers corresponding to actual reconstruction/verification procedures;
2. do not declare pairwise views a cover merely because they exist;
3. include boundary metadata in overlaps;
4. refine the topology when new local procedures become available;
5. compare which invariants become local under each topology.

The topology should neither promise impossible gluing nor hide important integration obligations.

12.12 Exercises

E12.1 - Pretopology (one star). Verify identity, pullback stability, and transitivity for interval-union covers.

E12.2 - Invalid coverage (one star). Propose a family that looks like a cover of reconnect state but is missing an ordinal boundary. Show why pullback or gluing fails.

E12.3 - Component site (two stars). Define a finite category of five SessionStream observers and a set of covering families. Justify every cover operationally.

E12.4 - Quorum (two stars). Model a three-replica majority cover. Define sections and restrictions sufficient for unique gluing, or construct a failure.

E12.5 - Authorization (two stars). Build principal-specific sites for an operator and an end user. Compare which global invariants have sections or truth witnesses.

E12.6 - Parameter topology (two stars). Given functional dependencies, define covers sufficient to decide a charge invariant. Check pretopology axioms or explain why the construction is only a coverage heuristic.

E12.7 - Sheafification (three stars). For a finite presheaf with two locally indistinguishable global sections, compute informally what sheafification must identify.

E12.8 - Topology comparison (three stars). Define a coarse and fine topology on the same context category. Find a presheaf that is a sheaf for the coarse topology but not the fine one.

12.13 Goldblatt bridge

Read Goldblatt Chapter 14, Sections 14.1-14.4 for restrictions, compatibility, sites, and Grothendieck topoi. His details are more general than needed for the first pass. Concentrate on why a cover is structural data and why local compatibility is tested through pullbacks.

Part IV - Topos and Local Logic

Chapter 13

A Topos as a Universe of Varying Software Types

13.1 From one presheaf to all presheaves

Fix a small context category $\mathcal{C}$. The presheaves on $\mathcal{C}$, together with natural transformations, form a category

$$\widehat{\mathcal{C}} = \mathbf{Set}^{\mathcal{C}^{op}}.$$

An object of $\widehat{\mathcal{C}}$ is a type whose values vary by context and restrict lawfully. An arrow is a context-uniform transformation: a natural transformation.

This category is not just a container of examples. It has the structural features of a universe of sets:

- finite limits;
- finite colimits and, in fact, all small limits and colimits;
- exponentials, or internal function spaces;
- a subobject classifier Ω ;
- an internal intuitionistic logic.

A category with finite limits, exponentials, and a subobject classifier is an **elementary topos**. A category equivalent to sheaves on a site is a **Grothendieck topos**. Every presheaf category on a small category is a Grothendieck topos and therefore an elementary topos.

Mental image. A topos is a mathematical universe in which “sets,” “functions,” “subsets,” and “truth values” are interpreted contextually.

13.2 SessionStream types as presheaves

In the ordinary Go universe, a type has one global set of possible values. In a presheaf universe, a type has a set of values at every context.

Possible objects include:

$$\text{Events}(U) = \text{valid event histories visible in } U,$$

$$\text{Timeline}(U) = \text{valid timeline observations visible in } U,$$

$\text{Snapshot}(U) = \text{snapshot information visible in } U,$

$\text{Client}(U) = \text{client states possible in } U,$

$\text{InvariantWitness}(U) = \text{proof objects available in } U.$

A projection becomes a natural transformation such as

$$P : \text{Events} \Rightarrow \text{Timeline}$$

when it commutes with every context restriction.

$$\begin{array}{ccc} \text{Events}(U) & \xrightarrow{\quad P_U \quad} & \text{Timeline}(U) \\ | & & | \\ | \text{ restrict} & & | \text{ restrict} \\ \downarrow & & \downarrow \\ \text{Events}(V) & \xrightarrow{\quad P_V \quad} & \text{Timeline}(V) \end{array}$$

Naturality says projecting a rich context and then forgetting equals forgetting the events first and projecting locally.

This may fail for a projection that needs information absent from V . That failure is not a technical nuisance: it says the proposed transformation is not internal to the chosen context universe.

13.3 Limits and colimits are computed contextwise

In a presheaf category, limits and colimits are computed pointwise.

For presheaves F, G , the product has

$$(F \times G)(U) = F(U) \times G(U),$$

with restriction performed componentwise.

The terminal presheaf has

$$1(U) = \{*\}$$

for every U .

The equalizer of $\alpha, \beta : F \Rightarrow G$ has

$$E(U) = \{x \in F(U) \mid \alpha_U(x) = \beta_U(x)\}.$$

Coproducts and coequalizers are also formed contextwise, with the resulting restriction maps inherited naturally.

This is powerful for software modeling. Once event, snapshot, and client observations are presheaves, consistent tuples and invariant subobjects can be constructed at every context using the same finite-limit expressions.

13.4 Why exponentials matter

A topos has an exponential G^F , an internal object of maps from F to G , characterized by

$$\text{Hom}(H \times F, G) \cong \text{Hom}(H, G^F)$$

naturally in H .

In $\mathbf{Set}$, G^F is the set of all functions $F \rightarrow G$. In a presheaf category, the exponential at context U is not generally just the set of pointwise functions $F(U) \rightarrow G(U)$. A local function must behave naturally under every subcontext of U .

One useful formula is

$$(G^F)(U) \cong \mathbf{Nat}(yU \times F, G),$$

where $yU = \mathcal{C}(-, U)$ is the representable presheaf at U .

Software interpretation:

A function available at context U is a family of operations valid not only on current values, but on all ways the context can be restricted, with coherent behavior across those restrictions.

This is much closer to a lawful component or generic handler than an arbitrary callback stored in a field.

13.5 Representable presheaves and probing a context

For an object $U \in \mathcal{C}$, define the representable presheaf

$$yU(V) = \mathcal{C}(V, U).$$

Its elements at V are the ways V maps into U , usually the ways V sits as a subcontext of U .

The Yoneda lemma states

$$\mathbf{Nat}(yU, F) \cong F(U).$$

An element of $F(U)$ is therefore equivalent to a natural way of responding to every probe $V \rightarrow U$.

This gives a categorical version of interface-based observation:

- the context U is known through all maps into it;
- a section over U is known through all its restrictions;
- equality of sections can be tested by their natural behavior under probes.

For `SessionStream`, a snapshot section over a reconnect context is not merely a struct. Via Yoneda, it is equivalent to a coherent response to every subcontext query supported by the model.

13.6 Subobjects as context-stable predicates

A subobject $P \hookrightarrow F$ selects a subset

$$P(U) \subseteq F(U)$$

at every context, stable under restriction.

Thus a predicate internal to the presheaf topos must be hereditary:

$$x \in P(U) \implies x|_V \in P(V).$$

Examples:

- schema-valid event history;

- ordinal-sound snapshot, when cut metadata restricts with the entities;
- heartbeat state with a matching current generation;
- trace whose visible records obey local ordering.

Nonexamples under arbitrary interval restriction:

- history contains a final event;
- at least ten tokens were generated;
- the session will eventually finish.

Those may require different contexts, temporal modalities, or predicates defined on future-closed regions.

13.7 Topos does not mean “topology-shaped architecture”

The word comes historically from categories of sheaves, but an elementary topos is characterized categorically. You do not obtain a topos merely by drawing components as a mesh.

To claim that a software model forms a topos, you need an actual category with the required structure. The easiest rigorous route is:

1. define a small context category $\mathcal{C}$;
2. take the presheaf category $\widehat{\mathcal{C}}$, which is a topos;
3. choose a Grothendieck topology J ;
4. take the sheaf topos $\mathbf{Sh}(\mathcal{C}, J)$.

Your particular event, snapshot, or configuration presheaf is then an object *inside* that topos. `SessionStream` itself is not automatically a topos.

13.8 An internal `SessionStream` universe

Once the objects are presheaves, familiar software constructions can be interpreted internally:

- `SessionId` becomes a context-varying object;
- `Event` becomes a typed sum or dependent family;
- timeline entities form another object;
- valid snapshots form a subobject;
- projections are arrows;
- configuration choices are sections;
- invariant proofs are sections of subobjects or dependent objects;
- function objects represent restriction-stable handlers;
- truth values become sieves of contexts.

This universe can reason about a client-visible session without pretending all data is globally available. A statement may hold over one context, fail over another, and be undecided globally.

13.9 Elementary versus Grothendieck viewpoints

The **elementary** viewpoint asks for finite limits, exponentials, and a subobject classifier. It supports internal logic without requiring a presentation by a site.

The **Grothendieck** viewpoint starts with a site and studies sheaves on it. It makes locality, covers, and geometric morphisms explicit.

For this project:

- use the Grothendieck/site view to model observation contexts and gluing;

- use the elementary view to reason abstractly about internal types, predicates, truth, and functions;
- move between them only with stated theorems.

Goldblatt develops both perspectives. The software bridge benefits from keeping both visible.

13.10 Exercises

E13.1 - Objects/arrows (one star). Define three SessionStream presheaves and one natural transformation between them.

E13.2 - Pointwise product (one star). Construct the product of event and timeline presheaves. Describe its sections and restrictions.

E13.3 - Equalizer (two stars). Build the presheaf of contexts where incremental and replay projections agree as an equalizer.

E13.4 - Exponential (two stars). Explain why arbitrary functions $F(U) \rightarrow G(U)$ do not necessarily form $(G^F)(U)$. Give a function that fails restriction coherence.

E13.5 - Yoneda (two stars). For a finite context poset, compute the representable yU and exhibit the bijection between natural transformations $yU \rightarrow F$ and elements of $F(U)$.

E13.6 - Internal predicate (two stars). Take the snapshot-soundness predicate. Prove it is or is not a subpresheaf for your chosen restrictions.

E13.7 - Topos discipline (three stars). Write a short critique of the phrase “our microservice architecture is a topos.” Replace it with a precise statement involving a presheaf or sheaf topos.

13.11 Goldblatt bridge

Goldblatt Chapter 4 defines elementary topoi through finite-limit structure, exponentiation, and a subobject classifier. Chapter 9 shows functor categories inheriting topos structure. Read those passages while keeping one small context category and its presheaves in view.

Chapter 14

Subobjects, Sieves, and Contextual Truth Values

14.1 Characteristic functions in Set

For a subset $A \subseteq X$, the characteristic function

$$\chi_A : X \rightarrow \{0, 1\}$$

sends members of A to 1 and nonmembers to 0. The subset is recovered as the pullback of $\text{true} : \{*\} \rightarrow \{0, 1\}$ along χ_A .

A subobject classifier in a category is an object Ω with an arrow

$$\text{true} : 1 \rightarrow \Omega$$

such that every monomorphism $m : A \hookrightarrow X$ is, up to isomorphism, the pullback of true along a unique characteristic arrow

$$\chi_m : X \rightarrow \Omega.$$

In Set , $\Omega = \{0, 1\}$. In a presheaf topos, truth values are richer.

14.2 Why two truth values are insufficient locally

Consider the proposition:

"this snapshot is verified coherent"

At a rich database context, the event log, entity versions, and cursor may be available, so the proposition can be verified. At a client-only context, only the snapshot payload is visible. At an observer context with dropped records, evidence may be partial.

A contextual truth value should record **where** the proposition holds under restriction, not only global yes/no.

For an element $x \in F(U)$ and subpresheaf $P \hookrightarrow F$, its characteristic truth value at U is the set of arrows $f : V \rightarrow U$ such that

$$F(f)(x) \in P(V).$$

That set is closed under further restriction, so it is a sieve on U .

14.3 The subobject classifier of a presheaf topos

For a presheaf category, define

$$\Omega(U) = \{\text{sieves on } U\}.$$

If $f : V \rightarrow U$, restriction

$$\Omega(U) \rightarrow \Omega(V)$$

pulls a sieve back along f : keep the arrows into V whose composite into U lies in the original sieve.

The maximal sieve containing every arrow into U is `true` at U . The empty sieve is `false`.

A truth value therefore has many possible intermediate states: the proposition may hold on some stable family of sub-contexts without holding on all of U .

14.4 A three-context chain

Let

$$U_0 \subseteq U_1 \subseteq U_2.$$

The sieves on U_2 are downward-closed collections of subcontexts. In the simple chain they are:

$$\emptyset,$$

$$\{U_0\},$$

$$\{U_0, U_1\},$$

$$\{U_0, U_1, U_2\}.$$

So $\Omega(U_2)$ has four truth values, not two.

Suppose `SnapshotSound` holds after restricting to client-visible fields U_0 only vacuously, holds genuinely at store view U_1 , but fails at full concurrency context U_2 because a race is visible. Depending on how the predicate is defined, its truth sieve might be $\{U_0, U_1\}$.

Be careful with vacuity: a predicate that cannot be expressed after restriction should not automatically be marked true. The subpresheaf must encode what counts as evidence at each context.

14.5 Characteristic arrows as invariant classifiers

Let `ValidSnapshot` $\hookrightarrow$ `Snapshot` be a subpresheaf. The characteristic arrow

$$\chi : \text{Snapshot} \rightarrow \Omega$$

assigns to every snapshot section $x \in \text{Snapshot}(U)$ the sieve

$$\chi_U(x) = \{f : V \rightarrow U \mid x|_V \text{ is valid in } V\}.$$

This is a richer diagnostic than a Boolean validator:

```
valid everywhere in the current context
valid only after forgetting concurrent-write evidence
valid only at schema level, not ordinal level
invalid in every nontrivial context
```

A tool could display the maximal subcontexts supporting an invariant. That is a direct software use of sieve-valued truth.

14.6 Heyting algebra structure

The subobjects of an object in a topos form a Heyting algebra. At a fixed context, sieves also form a Heyting algebra.

- conjunction is intersection;
- disjunction is generated union, which for sieves is ordinary union;
- implication $S \Rightarrow T$ contains those arrows $f : V \rightarrow U$ such that every further restriction belonging to S also belongs to T ;
- negation is $S \Rightarrow \emptyset$.

Unlike Boolean algebra, it need not be true that

$$S \cup \neg S = \top.$$

There can be contexts where neither a proposition nor its negation is established.

This is not fuzzy probability. Truth values are structured regions of validity or evidence.

14.7 SessionStream truth examples

14.7.1 Entity exists

At context U , a timeline entity may exist on some subcontexts but not others because earlier cuts precede its creation or later cuts contain a tombstone. The truth value records the cuts on which existence is stable.

14.7.2 Snapshot is fresh

Freshness relative to current event cursor may hold only on contexts where the event cursor is visible and equal to the snapshot cut. Restricting away the event cursor destroys the ability to assert that exact predicate unless evidence is carried.

14.7.3 Connection is live

A matching recent pong supports “not currently suspected” in the heartbeat context. It does not classify the remote process as globally alive.

14.7.4 Projection is complete

The proposition may hold in a context containing event cursor, projection cursor, and materialization evidence, but be undecidable in a snapshot-only context.

14.8 Truth versus evidence objects

A sieve-valued truth classifier records contexts where a subobject condition holds. In software, one may also want explicit evidence:

```
type SnapshotWitness struct {  
    SessionID      SessionId  
    Cut            uint64  
    EventDigest    []byte  
    EntityDigest    []byte  
    Projector       string  
    SchemaVersion   string  
}
```

Such witnesses form another presheaf. A natural transformation from witnesses to truth values forgets proof details and records only the supporting sieve.

This separates:

- proposition: snapshot is coherent;
- proof object: data demonstrating coherence;
- truth value: contexts where the proof is accepted.

Topos logic supports this distinction more naturally than global Booleans.

14.9 Exercises

E14.1 - Classifier (one star). Explain how a subset of a set is recovered as a pullback of `true` along its characteristic function.

E14.2 - Sieves (one star). List every sieve on a four-element chain. Identify top and bottom.

E14.3 - Characteristic sieve (two stars). For a finite snapshot presheaf and validity subpresheaf, compute $\chi_U(x)$ for several sections.

E14.4 - Implication (two stars). Compute the Heyting implication of two sieves on a small branching context poset.

E14.5 - Excluded middle (two stars). Find a sieve S for which $S \cup \neg S \neq \top$. Explain the corresponding software uncertainty.

E14.6 - Evidence (two stars). Design a presheaf of snapshot-coherence witnesses and a natural map to Ω .

E14.7 - Tooling (three stars). Design an invariant checker that returns a maximal supporting sieve rather than a Boolean. How would the UI display it?

14.10 Goldblatt bridge

Goldblatt Sections 4.1-4.3 motivate subobjects and subobject classifiers. Chapter 10 computes the classifier and truth arrows in presheaf topoi, and Chapter 14 connects them to local truth. Rework his classifier examples with a finite context poset before reading the general proofs.

Chapter 15

Kripke-Joyal Semantics and Operational Knowledge

15.1 Forcing notation

For a sheaf or presheaf topos, write

$$U \Vdash \varphi$$

and read: “context U forces φ ,” or “ φ holds locally over U .”

This is not an additional runtime command. It is the external description of the internal logic of the topos.

The exact clauses depend on variance and the site. The following intuition is reliable:

- assertions are evaluated at contexts;
- they must respect restriction;
- disjunction and existence may be witnessed locally on a cover;
- implication and universal claims must survive every relevant refinement/restriction.

15.2 Equality

For sections $x, y \in F(U)$,

$$U \Vdash x = y$$

when they are equal as sections over U . In a sheaf, equality can be checked locally:

$$U \Vdash x = y$$

if there is a cover $\{U_i \rightarrow U\}$ such that

$$U_i \Vdash x|_{U_i} = y|_{U_i}$$

for every i .

This is the logical form of separatedness and gluing.

For SessionStream, normalized snapshots may be equal locally on every entity-kind context and therefore equal globally, provided those contexts form a cover and no hidden global metadata remains.

15.3 Conjunction and implication

Conjunction is direct:

$$U \Vdash \varphi \wedge \psi$$

when both $U \Vdash \varphi$ and $U \Vdash \psi$.

Implication is hereditary:

$$U \Vdash \varphi \Rightarrow \psi$$

when for every context map $V \rightarrow U$,

$$V \Vdash \varphi|_V \implies V \Vdash \psi|_V.$$

A strong invariant implication must survive all local restrictions.

Example:

$$\text{ProjectionCursor} = n \Rightarrow \text{MaterializedThrough} = n.$$

To force this implication at a broad context, every subcontext capable of witnessing the premise must also witness the conclusion. If some observer sees the cursor but cannot see materialization evidence, the proposition may need a proof-carrying cursor or a different context design.

15.4 Disjunction is local

In sheaf semantics,

$$U \Vdash \varphi \vee \psi$$

can hold when U has a cover $\{U_i \rightarrow U\}$ such that each U_i forces one of the disjuncts, without one disjunct holding uniformly over all of U .

Example: an event payload may be decoded by schema version 1 on one region and version 2 on another, with the regions covering the trace. Locally, every event is v1-decodable or v2-decodable, even if neither decoder works globally.

This differs from returning one global tagged union value. Local disjunction can vary by region.

15.5 Existential witnesses can be local

The clause for

$$U \Vdash \exists x : F. \varphi(x)$$

allows a cover $\{U_i \rightarrow U\}$ and local sections $x_i \in F(U_i)$ such that

$$U_i \Vdash \varphi(x_i)$$

for every i . The witnesses need not begin as one global section.

Example: “there exists a decoder for every event in this trace” may be forced because each schema region has a local decoder. A global decoder object exists only if those local decoders glue coherently.

This is a constructive reading of existence: provide witnesses, at least locally.

15.6 Universal quantification

A universal statement

$$U \Vdash \forall x : F. \varphi(x)$$

requires φ to hold for every restriction $V \rightarrow U$ and every section $x \in F(V)$.

For an invariant such as

all timeline entities have `LastEventOrdinal` $\leq$ `SnapshotOrdinal`

universal forcing quantifies over every locally possible entity section in every subcontext, not merely the entities in one sampled snapshot.

This explains why internal universal statements can be much stronger than runtime loops over a current slice.

15.7 Heartbeat: suspected is not crashed

The heartbeat machine supplies a clean local-truth case.

At an awaiting context, the system has:

- a generation;
- a nonce;
- a successful ping-write time;
- a deadline;
- no accepted matching pong yet.

After the current-generation deadline, the machine enters `suspected` and closes the connection. The forced proposition is approximately:

$$U \Vdash \text{SuspectedUnder}(\Delta, \text{trace}).$$

It does not force

$$U \Vdash \text{RemoteCrashed}.$$

A delayed network, paused browser event loop, or scheduling stall can produce the same local section.

Nor does the earlier awaiting context force the negation of crash. Therefore the law

$$\text{RemoteCrashed} \vee \neg \text{RemoteCrashed}$$

may not be locally justified by available evidence, even though classical meta-logic says one physical situation obtains.

Topos logic models evidence and local determination, not metaphysical indeterminacy.

15.8 Snapshot completeness as local truth

Consider

$$\varphi(S) = \text{"snapshot } S \text{ is complete through its cut"}.$$

At a snapshot-only client context, φ may not be forced because completeness depends on the canonical event prefix or materialization witness. At a store context containing entity versions and projection cursor, it may be forced. At a full concurrent context, it may fail if cut and rows were not read coherently.

A proof-carrying snapshot can move evidence into the client context. Then restriction retains enough structure for φ to remain true.

This is a design consequence of internal logic:

To make an invariant locally assertable, transport the evidence needed by its characteristic subobject.

15.9 Tombstones and local existence

An entity created at ordinal 10 and tombstoned at ordinal 20 has context-dependent existence.

For prefix cuts:

$$[0, n] \Vdash \text{Exists}(x)$$

for $10 \leq n < 20$, but not after the tombstone under current-state semantics. If historical existence is the predicate, it remains true after 20. The context and predicate determine the logic.

This is a simple example of why “exists” is not one global Boolean in a temporal presheaf.

15.10 What local logic does not replace

Kripke-Joyal semantics does not replace:

- temporal logic for liveness and eventuality;
- probability for uncertain measurements;
- epistemic logic for multiple agents unless modeled appropriately;
- model checking for state-transition reachability;
- database constraints for direct relational invariants;
- type systems for compile-time guarantees.

It contributes a disciplined semantics of statements about context-varying objects. Those other logics can sometimes be internalized or combined, but not for free.

15.11 Exercises

E15.1 - Forcing clauses (one star). Write the forcing clauses for equality, conjunction, implication, disjunction, existence, and universal quantification in your own words.

E15.2 - Heartbeat (one star). List the propositions forced in each heartbeat phase. Distinguish protocol facts, timing assumptions, and remote-process claims.

E15.3 - Snapshot evidence (two stars). Design the minimal evidence object that lets a client context force snapshot soundness. What does it still need for completeness?

E15.4 - Local disjunction (two stars). Construct a cover on which `schema-v1 OR schema-v2` holds locally while neither disjunct holds globally.

E15.5 - Existential (two stars). Give a locally witnessed decoder or migration that does not glue globally. Identify the obstruction.

E15.6 - Excluded middle (two stars). Formalize one `SessionStream` proposition for which neither it nor its negation is forced at a chosen context.

E15.7 - Temporal distinction (three stars). Compare current existence, historical existence, and eventual existence of a timeline entity. Assign each to an appropriate logical framework.

E15.8 - Proof-carrying transport (three stars). Propose a protocol extension that transports invariant witnesses. Define how witnesses restrict and how the client verifies them.

15.12 Goldblatt bridge

Goldblatt Section 14.6 develops Kripke-Joyal semantics, while Chapters 8 and 10 prepare intuitionistic and presheaf truth. Read the forcing clauses after computing concrete sieves in Chapter 14 here. Keep heartbeat suspicion as the running example of evidence-relative truth.

Chapter 16

Geometric Morphisms and Change of Context

16.1 Translating between entire context universes

A functor between two context categories induces translations between their presheaf categories. The central topos-theoretic notion is a **geometric morphism**.

A geometric morphism

$$f : \mathcal{E} \rightarrow \mathcal{F}$$

consists of an adjoint pair

$$f^* : \mathcal{F} \rightleftarrows \mathcal{E} : f_*$$

with

$$f^* \dashv f_*$$

and f^* preserving finite limits. The functor f^* is called inverse image; f_* is direct image.

The naming follows sheaves on spaces: a continuous map pulls sheaves back and pushes them forward.

Why finite-limit preservation? Finite limits express products, equalizers, pullbacks, and therefore much of the logic of conjunction, equality, and coherent matching. A geometric translation should not destroy these structures.

16.2 Change of base by precomposition

Let

$$u : \mathcal{C} \rightarrow \mathcal{D}$$

be a functor between context categories. Precomposition gives

$$u^* : \widehat{\mathcal{D}} \rightarrow \widehat{\mathcal{C}}, \quad u^*(F) = F \circ u^{op}.$$

This operation views a $\mathcal{D}$ -varying type through the contexts available in $\mathcal{C}$. It preserves all pointwise limits and colimits.

Under standard smallness conditions, it has both a left and right Kan-extension adjoint:

$$\mathrm{Lan}_{u^{op}} \dashv u^* \dashv \mathrm{Ran}_{u^{op}} .$$

For software intuition:

- precomposition is adaptation by reindexing contexts;
- left Kan extension freely aggregates or extends data to the larger context system;
- right Kan extension collects all compatible ways data could be seen from the larger system.

Do not use Kan extension vocabulary without specifying u and the universal property.

16.3 Client view as a change of base

Let $\mathcal{C}_{full}$ contain event log, timeline, snapshot, WebSocket, and client contexts. Let $\mathcal{C}_{client}$ contain only client-visible contexts. The inclusion

$$i : \mathcal{C}_{client} \hookrightarrow \mathcal{C}_{full}$$

induces restriction

$$i^* : \widehat{\mathcal{C}_{full}} \rightarrow \widehat{\mathcal{C}_{client}} .$$

This forgets server-only contexts while retaining how objects behave on client contexts.

A right Kan extension can ask:

Given client-visible data, what is the space of all compatible full-context completions?

A left Kan extension can ask:

What is the freest full-context object generated by the client-visible data?

These two questions correspond roughly to inference versus free completion. They are not generally the same.

16.4 Schema-version change of base

Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be context categories for schema versions. A migration functor

$$u : \mathcal{C}_1 \rightarrow \mathcal{C}_2$$

maps old contexts to their new interpretations.

Precomposition translates new-version presheaves into old contexts. A left/right extension may construct new-version data from old observations or collect compatible migrations.

Finite-limit preservation becomes a concrete design requirement: does migration preserve compatible tuples, equalizers of validation paths, and pullback joins on session/ordinal boundaries?

A migration that collapses distinct event identities may fail to preserve a relevant pullback even if individual payloads convert successfully.

16.5 Transport as a geometric translation candidate

The WebSocket transport maps server-internal objects to wire-visible objects. To model it geometrically:

1. define a server context site and wire context site;
2. define the functor relating contexts;
3. define inverse/direct image functors on sheaves;
4. verify finite-limit preservation and adjunction.

Most ordinary serializers will not automatically yield a geometric morphism. The claim becomes plausible only after restricting to typed, valid frames and choosing semantic equality that ignores irrelevant encoding differences.

Still, the criterion is useful:

A transport preserves logical structure when valid products, equalizers, and pullbacks of observable data remain valid after translation.

For example, the pullback expressing “frame payload type agrees with logical name” should survive wire encoding and decoding.

16.6 Sheafification and inclusion

For a site $(\mathcal{C}, J)$, the inclusion

$$i_* : \mathbf{Sh}(\mathcal{C}, J) \hookrightarrow \widehat{\mathcal{C}}$$

has a left adjoint sheafification

$$i^* = a : \widehat{\mathcal{C}} \rightarrow \mathbf{Sh}(\mathcal{C}, J).$$

Sheafification is left exact, so this adjunction defines a geometric morphism from the sheaf topos into the presheaf topos.

Interpretation:

- the presheaf universe allows arbitrary local data with restriction;
- the sheaf universe contains those objects satisfying the chosen local-to-global law;
- sheafification reflects arbitrary local data into the glueable universe.

This is the cleanest exact example of “repair as a structure-preserving translation” in the book.

16.7 Preservation and reflection of invariants

A functor **preserves** a property if objects/arrows having it map to objects/arrows having it. It **reflects** a property if having it after mapping implies having it before mapping.

For a transport or migration:

- preserving monomorphisms means distinct subobjects remain embeddings;
- preserving pullbacks means typed joins and compatibility squares survive;
- reflecting isomorphisms means a target-level equivalence implies a source-level equivalence;
- preserving truth may require more than finite-limit preservation, depending on logical connectives.

An encoder can preserve validity while failing to reflect it: many malformed byte strings may not decode, but every valid event encodes to a valid frame. A lossy observer can preserve protocol errors it records but cannot reflect absence of errors.

16.8 A geometric architecture review

For one adapter, answer:

1. What are source and target context categories?
2. What is the context functor?
3. What does inverse image do to an object of varying data?
4. What compatible data does direct image collect?
5. Is there a natural hom-set bijection?
6. Does inverse image preserve terminal objects, products, equalizers, and pullbacks?
7. Which subobjects/invariants are preserved or reflected?
8. Does the functor respect the chosen coverage and sheaves?

Even a negative result is useful. It tells you exactly which logical structure the boundary discards.

16.9 Exercises

E16.1 - Reindexing (one star). Define an inclusion of client contexts into full contexts and calculate precomposition on one presheaf.

E16.2 - Kan questions (one star). For client data, state the different universal questions answered by left and right Kan extension.

E16.3 - Finite limits (two stars). Test whether a schema migration preserves a product and equalizer of simple message contracts.

E16.4 - Transport pullback (two stars). Write the pullback expressing name/payload schema agreement before and after wire encoding. Determine the assumptions under which it is preserved.

E16.5 - Preservation/reflection (two stars). Analyze the best-effort observer functor. Which error or ordering properties can it preserve? Which can it reflect?

E16.6 - Sheafification morphism (two stars). Explain why the sheafification/inclusion adjunction is a geometric morphism. Identify the left-exact functor.

E16.7 - Full review (three stars). Perform the eight-step geometric architecture review on one real SessionStream boundary.

16.10 Goldblatt bridge

Goldblatt Chapters 15 and 16 develop adjunctions, preservation, geometric morphisms, and geometric logic. At this point, return to them. Translate each preservation theorem into a question about whether a software boundary retains finite-limit invariants.

Part V - Shapes and Cohomology

Chapter 17

Nerves: Turning Overlap into Multidimensional Shape

17.1 From a cover to a combinatorial object

Let

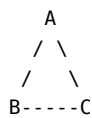
$$\mathcal{U} = \{U_i \rightarrow X\}_{i \in I}$$

be a cover. Its **nerve** is a simplicial object recording which finite intersections are nonempty or otherwise admissible.

- one vertex for each U_i ;
- one edge for each nonempty pairwise overlap $U_i \cap U_j$;
- one filled triangle for each nonempty triple overlap $U_i \cap U_j \cap U_k$;
- one tetrahedron for each nonempty four-way overlap;
- and so on.

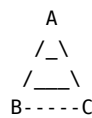
The crucial distinction is between a triangular boundary and a filled triangle.

three pairwise overlaps



1-dimensional loop

genuine triple overlap



2-dimensional face fills loop

The nerve is the first rigorous version of the “multidimensional topological-esque shape” intuition.

17.2 What dimension means here

Dimension measures the order of joint compatibility:

- dimension 0: one local context;
- dimension 1: pairwise overlap;
- dimension 2: a three-way context in which pairwise comparisons can themselves be compared;
- dimension 3: four-way joint compatibility;
- higher dimensions: larger coherent intersections.

A system can have many pairwise contracts but no three-way witness. Its nerve contains edges without the corresponding faces. Those missing faces create potential cycles.

A transaction or integration test that jointly observes three components can add a 2-simplex to the model. It “fills” a triangle of pairwise interfaces by providing a higher-order compatibility context.

This is a modeling statement, not an assertion that a SQL transaction is literally a geometric triangle.

17.3 A SessionStream observer cover

Consider five contexts:

E = event log and event cursor,

T = timeline materialization and projection cursor,

S = snapshot and entity metadata,

W = WebSocket hydration buffer and sent frames,

C = client reducer state.

Possible overlaps:

- $E \cap T$: event/session/ordinal relation used by projection;
- $T \cap S$: materialized entities and snapshot cut;
- $S \cap W$: snapshot frame and hydration boundary;
- $W \cap C$: delivered frames and client application;
- $C \cap E$: client acknowledgments or trace digests related to canonical events.

These edges form a loop:

```

E ---- T
|      |
|      |
C ---- S
 \    /
  W

```

A more literal cycle ordering is

$E \rightarrow T \rightarrow S \rightarrow W \rightarrow C \rightarrow E$.

Whether the nerve contains filling faces depends on available joint contexts:

- a database transaction may jointly observe E, T, S ;
- a transport trace may jointly observe S, W, C ;
- an end-to-end test may observe all five;
- best-effort observers may not count as reliable intersections.

The shape is therefore architecture plus evidence policy.

17.4 The Cech nerve versus the simple nerve

The simple nerve records only whether intersections exist. The **Cech nerve** retains the intersections themselves:

$$\coprod_i U_i,$$

$$\coprod_{i,j} U_i \times_X U_j,$$

$$\coprod_{i,j,k} U_i \times_X U_j \times_X U_k,$$

and higher iterated pullbacks.

This matters because different overlaps can carry different data and restriction maps. An edge labeled merely “nonempty” cannot express that event/timeline overlap contains ordinals while snapshot/client overlap contains reducer versions.

For software work, annotate every simplex with:

- shared semantic fields;
- restriction functions;
- equality relation;
- reliability assumptions;
- evidence source.

17.5 The nerve of a temporal cover

Cover interval $[0, 100]$ by

$$U_0 = [0, 60], \quad U_1 = [40, 90], \quad U_2 = [80, 100].$$

Then:

- $U_0 \cap U_1 = [40, 60]$ is nonempty;
- $U_1 \cap U_2 = [80, 90]$ is nonempty;
- $U_0 \cap U_2 = \emptyset$;
- no triple intersection exists.

The nerve is a path of two edges, with no loop.

If instead every pair overlaps but the triple intersection is empty, the nerve is a triangular loop. If all three overlap at some cut, the triangle is filled.

Thus a one-dimensional time line can yield a higher-dimensional nerve through the pattern of interval overlaps.

17.6 The nerve lemma intuition

Under suitable “good cover” conditions in topology, the nerve has the same homotopy type as the covered space. This is the nerve lemma.

We will not use the theorem as a blanket claim for software. Its lesson is methodological:

A complicated global space can sometimes be studied through the combinatorics of how simple local regions overlap.

In software, the underlying object may be a constraint space, execution space, or information system rather than a manifold. The nerve remains a useful combinatorial index even when no topological equivalence theorem applies.

17.7 Filled faces as higher-order integration evidence

Suppose three services expose pairwise contracts:

```
EventStore <-> Projector
Projector <-> SnapshotStore
SnapshotStore <-> EventStore
```

If there is no test or transaction that validates all three simultaneously, the model has a triangle boundary. Pairwise passing tests can coexist with a cyclic inconsistency.

Add an end-to-end test that:

1. appends an event;
2. projects it;
3. reads a snapshot at the reported cut;
4. compares event, materialization, and cut in one trace.

This joint witness supports adding the filled triangle (E, T, S) to the nerve.

Higher-dimensional simplices are therefore a useful inventory of **which combinations are jointly witnessed**, not merely which components communicate.

17.8 Hypergraphs, simplicial complexes, and categories

A hypergraph records multiway relationships directly. A simplicial complex additionally requires every face of a simplex to be present. A category can record multiple arrows, direction, and composition. A simplicial set can record several simplices with the same vertices and degeneracies.

Use:

- a graph for first-pass pairwise dependencies;
- a hypergraph for named multiway contracts;
- a simplicial complex for topological/cohomological calculation when face closure is appropriate;
- a simplicial set or Čech nerve when multiple distinct overlaps matter;
- a category/site when restriction direction and composition matter.

The software shape is often richer than a simple undirected graph.

17.9 Exercises

E17.1 - Nerve (one star). Compute the nerve of three overlapping ordinal intervals in three cases: path, boundary triangle, and filled triangle.

E17.2 - SessionStream cover (one star). Choose five contexts from the repository and draw their overlap graph. Label every edge with shared semantics.

E17.3 - Higher witness (two stars). Identify one pairwise contract triangle in SessionStream. Design a joint test that justifies filling it.

E17.4 - Reliability (two stars). Recompute the nerve when best-effort observer records are not accepted as reliable overlaps. Which simplices disappear?

E17.5 - Čech labels (two stars). For every pair and triple in a small cover, write the actual pullback context rather than only nonempty.

E17.6 - Model choice (two stars). Decide whether your architecture needs a graph, hypergraph, simplicial complex, simplicial set, or category. Defend the choice.

E17.7 - Multidimensional sketch (three stars). Construct the product of time, observer, and schema-version context dimensions for one feature. Draw a three-dimensional slice and its nerve.

17.10 Reading bridge

Goldblatt supplies the categorical and sheaf-theoretic preparation, but introductory Čech cohomology is not a central goal of his book. From this chapter onward, the exposition extends the local-to-global framework using standard cohomological constructions while retaining his concrete-to-abstract pedagogy.

Chapter 18

Cech Cohomology from Local Differences

18.1 Why groups enter

A set-valued sheaf lets us ask whether local sections match and glue. Cohomology requires additional algebra so that differences can be added, subtracted, and quotiented.

Let $\mathcal{A}$ be a sheaf or presheaf of abelian groups. For each context U , $\mathcal{A}(U)$ is an abelian group, and every restriction map is a group homomorphism.

Examples of coefficient data:

- integer cursor offsets;
- parity bits in $\mathbb{F}_2$;
- real-valued clock skews;
- vectors of counters;
- additive resource discrepancies;
- checksums in an abelian group.

Arbitrary events, protobuf messages, or state machines are not abelian groups. Cohomology usually studies a linearized measurement extracted from them.

18.2 Cochains on a cover

For an ordered cover $\mathcal{U} = \{U_i\}$, define the Cech cochain groups

$$C^0(\mathcal{U}, \mathcal{A}) = \prod_i \mathcal{A}(U_i),$$

$$C^1(\mathcal{U}, \mathcal{A}) = \prod_{i < j} \mathcal{A}(U_i \cap U_j),$$

$$C^2(\mathcal{U}, \mathcal{A}) = \prod_{i < j < k} \mathcal{A}(U_i \cap U_j \cap U_k),$$

and so on, omitting empty intersections.

Interpretation:

- a 0-cochain assigns one local value to every vertex/context;
- a 1-cochain assigns one value to every overlap/edge;

- a 2-cochain assigns one value to every triple overlap/face.

Cochains need not be consistent. They are raw assignments by dimension.

18.3 The first coboundary

For $x = (x_i) \in C^0$, define

$$(\delta^0 x)_{ij} = x_j|_{U_i \cap U_j} - x_i|_{U_i \cap U_j}.$$

This is the edge discrepancy between local values after both are restricted to the overlap.

If

$$\delta^0 x = 0,$$

then the local sections form a matching family.

For a sheaf, such a family glues uniquely. Therefore

$$H^0(X, \mathcal{A}) \cong \Gamma(X, \mathcal{A}),$$

the group of global sections, under suitable Cech hypotheses.

This makes H^0 immediately understandable: globally consistent additive assignments.

18.4 The second coboundary

For a 1-cochain $a = (a_{ij})$, define on triple overlaps

$$(\delta^1 a)_{ijk} = a_{jk}|_{ijk} - a_{ik}|_{ijk} + a_{ij}|_{ijk}.$$

The alternating signs compare the edge discrepancies around a triangle.

A central identity is

$$\delta^1 \circ \delta^0 = 0.$$

Differences derived from vertex values always sum to zero around every filled triangle. Algebraically, every term cancels. Geometrically, the boundary of a boundary is zero.

Higher coboundaries continue with alternating restrictions.

18.5 Cocycles and coboundaries

A 1-cochain a is a **cocycle** if

$$\delta^1 a = 0.$$

It is a **coboundary** if

$$a = \delta^0 x$$

for some 0-cochain x .

Every coboundary is a cocycle because $\delta^1 \delta^0 = 0$.

The first cohomology group is

$$H^1(\mathcal{U}, \mathcal{A}) = \frac{\ker \delta^1}{\operatorname{im} \delta^0}.$$

Read this as:

closed edge-transition data, modulo transition data explained by choosing local vertex coordinates.

A nonzero class is a pattern consistent on every filled triangle but not globally removable by changing local origins.

18.6 Cohomology is not failure of the sheaf axiom

This distinction is essential.

A sheaf already guarantees that matching 0-cochains glue. Nonzero H^1 does not mean the sheaf fails to be a sheaf.

Instead, H^1 can measure obstructions to solving a related global problem, such as:

- trivializing local transition functions;
- finding global potentials for local differences;
- extending local primitives;
- splitting a torsor;
- choosing consistent coordinates.

Sheaf cohomology can be defined as derived functors of global sections. Cech cohomology computes it under suitable conditions. For this textbook, the main use is the concrete local-coordinate obstruction.

18.7 Constant coefficients on a graph

Let the nerve be a graph and let $\mathcal{A} = \mathbb{Z}$ have constant integer values and identity restrictions.

A 0-cochain assigns an integer x_i to each vertex. The edge coboundary is

$$(\delta^0 x)_{ij} = x_j - x_i.$$

A 1-cochain assigns an integer offset a_{ij} to each oriented edge.

If the graph has no filled triangles, $C^2 = 0$, so every 1-cochain is a cocycle. It is a coboundary exactly when all cycle sums vanish.

This is the mathematics of global coordinate alignment.

18.8 The square calculation

Orient a square

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A.$$

Then

$$C^0 \cong \mathbb{Z}^4, \quad C^1 \cong \mathbb{Z}^4.$$

The coboundary matrix is

$$\delta^0 = \begin{bmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & -1 \end{bmatrix}.$$

For vertex coordinates $x = (x_A, x_B, x_C, x_D)^T$, the edge offsets are $\delta^0 x$.

Every coboundary satisfies

$$a_{AB} + a_{BC} + a_{CD} + a_{DA} = 0.$$

The cochain

$$a = (0, 0, 0, 1)$$

has cycle sum 1 and cannot equal $\delta^0 x$. It represents a nonzero class in

$$H^1 \cong \mathbb{Z}.$$

The generator records one unit of circulation around the hole.

18.9 Filling the triangle

Take three vertices with all three edges.

If there is no 2-simplex, the triangular boundary has

$$H^1 \cong \mathbb{Z}.$$

If a triple intersection fills the triangle, $C^2 \cong \mathbb{Z}$ and

$$\delta^1(a_{01}, a_{12}, a_{02}) = a_{12} - a_{02} + a_{01}.$$

The cocycle condition forces the triangle sum to vanish. Every such cocycle is a vertex coboundary, so

$$H^1 = 0$$

for the filled triangle.

A genuine three-way compatibility witness removes the independent circulation supported by the boundary.

18.10 Homology versus cohomology

Homology studies chains of simplices and cycles modulo boundaries. Cohomology studies functions or coefficients on simplices with coboundary maps.

For software diagnostics, cohomology is often more natural because measurements live **on** components, overlaps, and higher intersections:

- local coordinate values on vertices;
- discrepancy values on edges;

- consistency residuals on faces.

The two theories are related, but do not collapse them into one word “holes.” Cohomology combines the shape with a coefficient system, so the same nerve can yield different results for different data and restriction maps.

18.11 Nonconstant and cellular sheaves

In a real architecture, edge overlaps do not all carry the same data. A cellular sheaf on a graph or simplicial complex assigns vector spaces or groups to vertices and edges, with restriction maps from incident cells to overlaps.

For example:

- event store vertex carries `(eventCursor, streamId)`;
- timeline vertex carries `(projectionCursor, maxEntityOrdinal)`;
- their edge carries a shared ordinal coordinate;
- restrictions extract or translate those coordinates.

Then

$$(\delta^0 x)_e = \rho_{head,e}(x_{head}) - ho_{tail,e}(x_{tail}).$$

The kernel of δ^0 is the space of globally consistent local measurements. Higher cohomology reflects both topology and the restriction maps.

This is closer to the intended SessionStream application than a constant sheaf.

18.12 Exercises

E18.1 - Cochains (one star). For a path of three vertices, write C^0 , C^1 , and the coboundary matrix.

E18.2 - Delta squared (one star). Expand $\delta^1 \delta^0 x$ on a triple overlap and show every term cancels.

E18.3 - Square (one star). Solve $\delta^0 x = a$ for several edge-offset vectors. Verify that a solution exists exactly when the cycle sum is zero.

E18.4 - Triangle (two stars). Compute H^1 of the triangle boundary and the filled triangle over $\mathbb{R}$ using matrix ranks.

E18.5 - Parity coefficients (two stars). Repeat the square calculation over $\mathbb{F}_2$. Interpret nonzero circulation as duplicate/missing parity.

E18.6 - Nonconstant restrictions (two stars). Define vertex and edge vector spaces for event and timeline cursors. Write the sheaf coboundary matrix.

E18.7 - Concept (three stars). Explain why nonzero H^1 is not evidence that the coefficient sheaf violates the sheaf axiom.

E18.8 - Code (three stars). Implement cochain matrices and compute kernels/images over rational numbers or a finite field. Test path, cycle, and filled-triangle complexes.

18.13 Reading bridge

Use standard introductory sources on Cech or cellular sheaf cohomology for proofs beyond this chapter. Retain Goldblatt’s method: compute concrete examples before accepting the derived-functor formulation.

Chapter 19

A SessionStream Cohomology Laboratory

19.1 The diagnostic question

SessionStream uses several ordinal-bearing views. We want to know whether their local coordinate conventions can be aligned into one global ordinal coordinate.

This is a narrower and more computable question than “is SessionStream correct?”

Define vertices:

E = event store, T = timeline projection, S = snapshot, W = WebSocket delivery, C = client application.

At each vertex, choose a local integer coordinate x_i . This may be:

- event cursor;
- projection cursor;
- snapshot ordinal;
- delivered UI ordinal;
- client-applied ordinal.

Before calculating, specify their meanings. If one cursor means “last included” and another means “next expected,” the edge restriction must include that translation.

19.2 Edge offsets

For an oriented overlap $i \rightarrow j$, define an observed or contractual offset

$$a_{ij} = x_j - x_i.$$

Examples:

- timeline cursor equals event cursor: $a_{ET} = 0$;
- snapshot cut equals timeline cursor: $a_{TS} = 0$;
- first live event is strictly after snapshot cut: if the local coordinate is first-delivered ordinal, $a_{SW} = 1$ under dense ordinals;
- client applied ordinal equals delivered ordinal: $a_{WC} = 0$;
- client acknowledgment equals event cursor: $a_{CE} = 0$.

Already there is a semantic issue. `first live ordinal = snapshot cut + 1` assumes dense ordinals. Current ordinal assignment can derive values from Redis-style stream IDs, so ordinals may be strictly increasing but not consecutive. The correct edge relation may be order-based rather than a fixed integer offset.

This is a useful failure of the model: choose coefficients and equations that match the actual semantics.

19.3 An off-by-one loop

For a simplified dense-ordinal lab, suppose the edge conventions are:

$$a_{ET} = 0, \quad a_{TS} = 0, \quad a_{SW} = 1, \quad a_{WC} = 0, \quad a_{CE} = 0.$$

The circulation around the loop is

$$0 + 0 + 1 + 0 + 0 = 1.$$

No vertex coordinates can realize all these equations simultaneously if every edge is interpreted as a difference between the same kind of coordinate.

The nonzero cohomology class says:

At least one local coordinate has a different origin or semantic meaning; the loop cannot be globally aligned by merely choosing vertex offsets.

The likely repair is not “make every number equal.” It is to distinguish types:

- `SnapshotCut` = last included event;
- `NextExpected` = lower bound for next event;
- `DeliveredEventOrdinal` = actual event coordinate.

Then edge maps become typed transformations rather than contradictory equalities.

19.4 A more honest order-valued model

If ordinals are only strictly increasing, use an ordered set rather than the additive group $\mathbb{Z}$. Relations include

$$O_S < O_U$$

for delivered events after a snapshot, not $O_U - O_S = 1$.

Ordinary abelian cohomology no longer applies directly. Options include:

- map order relations to slack variables in $\mathbb{Z}_{\geq 0}$;
- analyze exact equalities only and keep inequalities in a constraint solver;
- use sheaves of posets or categories and nonabelian obstruction methods;
- linearize around an expected relation for diagnostics.

This illustrates the correct workflow: presheaf/sheaf modeling first, cohomology only after a defensible coefficient system is extracted.

19.5 A cellular cursor sheaf

Let each vertex carry a vector space of local measurements.

Example over $\mathbb{R}$:

$$F(E) = \mathbb{R}^2 \quad \text{with } (eventCursor, streamDerivedOrdinal),$$

$$F(T) = \mathbb{R}^2 \quad \text{with } (projectionCursor, maxEntityOrdinal),$$

$$F(S) = \mathbb{R}^2 \quad \text{with } (snapshotOrdinal, maxSnapshotEntityOrdinal),$$

$$F(W) = \mathbb{R}^2 \quad \text{with } (snapshotSentOrdinal, lastUIOrdinal),$$

$$F(C) = \mathbb{R}^2 \quad \text{with } (snapshotAppliedOrdinal, lastUIAppliedOrdinal).$$

Each edge stalk contains the shared coordinate to be compared. Restriction maps extract the relevant component. For example:

$$\rho_{T,TS}(p, m) = p, \quad \rho_{S,TS}(s, m) = s.$$

The degree-zero coboundary stacks all edge residuals. Its kernel is the vector space of measurements satisfying every equality constraint.

This calculation is often useful even before H^1 :

$$H^0 = \ker \delta^0$$

is the global consistency space.

19.6 Adding higher-order witnesses

Suppose event store, timeline store, and snapshot are jointly read in one transaction. Add a 2-simplex (E, T, S) . Its face restriction maps require edge discrepancies to satisfy a triangle condition.

Suppose snapshot, WebSocket server, and client are jointly traced with reliable acknowledgments. Add (S, W, C) .

These faces reduce which edge discrepancy patterns qualify as cocycles. A residual that was invisible to pairwise checks may be exposed by the 2-simplex.

An end-to-end test covering (E, T, S, W, C) can add higher-dimensional structure, but only if its observations are trustworthy and semantically joint.

19.7 Duplicate parity over $\mathbb{F}_2$

Take coefficients in

$$\mathbb{F}_2 = \{0, 1\}$$

and record whether each boundary believes an event was applied an even or odd number of times.

An edge value 1 indicates a parity disagreement. Around a cycle, a nonzero H^1 class can represent duplicate/missing parity that cannot be assigned to one local component by vertex corrections.

This is a coarse diagnostic. It cannot distinguish one duplicate from three, nor identify the event without richer coefficients. But finite-field calculations are robust and easy to automate.

19.8 Digest and checksum coefficients

A cryptographic digest is not naturally additive, but one can use:

- XOR digests in a vector space over $\mathbb{F}_2$;
- polynomial rolling hashes in a finite field;
- count vectors by event kind;
- sketches with additive merge laws.

Each local context emits a digest of the history it claims to represent. Restrictions map richer digests to shared summaries. Cohomology can then detect circulation in summaries.

Hash collisions and lossy summaries mean zero residual is evidence, not proof, unless the coefficient map is injective on the domain of interest.

19.9 Event identity is not an abelian offset

Retry identity illustrates a problem that should not be forced into this lab. Deciding whether two deliveries denote the same logical event is an equivalence or groupoid problem, not naturally an integer discrepancy.

Possible models include:

- a set-valued presheaf of candidate identity assignments;
- a groupoid of event-labelings and relabelings;
- nonabelian Čech 1-cocycles with transition bijections;
- a constraint-satisfaction problem;
- database keys and uniqueness constraints.

Only after extracting an additive invariant - for example duplicate count parity - does ordinary cohomology apply.

19.10 A concrete calculation workflow

For one recorded session:

1. Extract local measurement vectors at each context.
2. Build restriction matrices for every edge.
3. Assemble δ^0 .
4. Compute the residual $r = \delta^0 x$.
5. If solving for latent corrected coordinates, solve $\delta^0 x \approx b$.
6. Build δ^1 from reliable 2-simplices.
7. Check whether observed edge transitions are cocycles.
8. Reduce cocycles modulo coboundaries to obtain classes.
9. Map each class to a supporting cycle in the nerve.
10. Return to semantics: identify which coordinate or contract caused the circulation.

Do not stop at a Betti number. The useful output names contexts, edges, restrictions, and evidence.

19.11 Pseudocode

```
type CellID string

type Vector []float64

type LinearMap struct {
    Rows int
```

```

    Cols int
    Data [][]float64
}

type VertexStalk struct {
    Cell CellID
    Value Vector
}

type Edge struct {
    ID      CellID
    Tail    CellID
    Head    CellID
    FromTail LinearMap
    FromHead LinearMap
}

// One block row per edge:
//      rhoHead(xHead) - rhoTail(xTail)
func Coboundary0(vertices []VertexStalk, edges []Edge) LinearMap

```

For exact integer or finite-field work, do not use floating-point rank. Use rational arithmetic, Smith normal form, or finite-field elimination.

19.12 Exercises

E19.1 - Semantics first (one star). Define the exact meanings of five ordinal-bearing quantities in the current repository. State equalities and inequalities separately.

E19.2 - Cycle sum (one star). Build a five-vertex constant-offset loop and determine whether a global coordinate exists.

E19.3 - Typed repair (two stars). Replace an off-by-one contradiction by distinct ordinal types and explicit conversion arrows.

E19.4 - Cellular sheaf (two stars). Define vertex/edge stalks and restrictions for event, timeline, and snapshot measurements. Write δ^0 .

E19.5 - Higher witness (two stars). Add the (E, T, S) transaction face and calculate the new cocycle equation.

E19.6 - Parity (two stars). Design an $\mathbb{F}_2$ -valued duplicate diagnostic. State what false negatives it permits.

E19.7 - Trace implementation (three stars). Ingest observer records and store snapshots for one session, construct the complex, and report residuals with supporting cycles.

E19.8 - Model rejection (three stars). Find one SessionStream invariant that cannot be responsibly linearized. Give a set-, poset-, or groupoid-valued alternative.

19.13 Interpretation discipline

A nonzero class says the chosen local transition data cannot be globally trivialized within the chosen coefficient model. It does not directly say:

- which component is buggy;
- that the user saw an error;
- that the full system has no global execution;
- that an invariant outside the coefficient map failed;
- that the cover accurately represents all evidence.

Cohomology is a diagnostic lens, not an oracle.

Chapter 20

A Sheaf Audit and Capstone Tool

20.1 The audit workflow

Use the following workflow on any software invariant.

20.1.1 Step 1: State the global claim

Bad:

hydration is consistent

Better:

$$\text{ApplyUI}^*(\text{ApplySnapshot}(C_0, S_s(n)), U_s(n, m]) \cong_C C_s(m).$$

Specify equality, time boundary, failure policy, and admissible executions.

20.1.2 Step 2: Choose contexts

List exactly which observers or parameter sets see which facts. Add boundary contexts for shared semantics.

20.1.3 Step 3: Define sections

A section should contain enough data to state local validity. Avoid both raw implementation dumps and oversimplified summaries.

20.1.4 Step 4: Define restriction maps

Implement and test identity/composition laws. If restriction is impossible, either change the section type or admit that the presheaf is semantic rather than operational.

20.1.5 Step 5: Declare covers

Every cover must correspond to an actual reconstruction or verification policy. Check identity, pullback stability, and transitivity when claiming a site.

20.1.6 Step 6: Check matching

Compare restrictions on every overlap. Report semantic mismatches, not only field inequality.

20.1.7 Step 7: Search for global sections

Classify the completion set as empty, singleton, or multiple. Use constraint solvers, joins, replay, or explicit construction.

20.1.8 Step 8: Linearize selected residuals

Extract abelian measurements and build cochain complexes only where subtraction and composition are meaningful.

20.1.9 Step 9: Interpret nonzero classes

Map classes back to architecture cycles and missing higher-order witnesses.

20.1.10 Step 10: Change the system or the model

Possible repairs:

- add a coordinate;
- make a transaction atomic;
- add an overlap field;
- transport evidence;
- change a cover;
- add a joint test/2-simplex;
- quotient an irrelevant distinction;
- introduce a stable identity;
- weaken the invariant honestly.

20.2 Proposed package layout

A capstone research tool could live outside the production core initially:

```
cmd/sessionstream-sheaf-audit/
pkg/sheafaudit/context.go
pkg/sheafaudit/presheaf.go
pkg/sheafaudit/cover.go
pkg/sheafaudit/glue.go
pkg/sheafaudit/complex.go
pkg/sheafaudit/cohomology.go
pkg/sheafaudit/report.go
pkg/sheafaudit/sessionstream/trace_adapter.go
```

Core interfaces:

```
type ContextID string

type Context struct {
    ID          ContextID
    Session     sessionstream.SessionId
    Interval    OrdinalRegion
    Lenses      []string
    Version     map[string]string
}

type Inclusion struct {
    Smaller ContextID
    Larger  ContextID
}

type Section interface {
```

```
Context() ContextID
Equal(Section) bool
}

type Restrictor interface {
    Restrict(ctx context.Context, s Section, to ContextID) (Section, error)
}

type Cover struct {
    Whole ContextID
    Members []ContextID
}
```

Add explicit law tests:

```
Restrict(s, U) == s
Restrict(Restrict(s, V), W) == Restrict(s, W)
```

20.3 Fact-oriented sections

A general trace section can be represented as typed facts:

```
type FactKey struct {
    Domain string // event, timeline, snapshot, websocket, client
    Name   string
}

type Fact struct {
    Key      FactKey
    Value    any
    Evidence []TraceRecordID
}

type FactSection struct {
    At      ContextID
    Facts   map[FactKey]Fact
}
```

Restrictions should be registered semantic functions, not generic map filtering. For example:

- full event -> ordinal fact;
- timeline entity -> last-event ordinal;
- snapshot -> boundary record;
- WebSocket trace -> ordered delivered batches;
- heartbeat trace -> current detector evidence.

Every restriction should preserve provenance so a mismatch report can point to source records.

20.4 Global-section search

For finite models, encode global completion as a constraint problem.

Variables:

- latent canonical event identities;
- ordinal/cut coordinates;
- schema versions;
- entity versions;
- client reducer state;

- retry grouping.

Constraints:

- local section restrictions;
- overlap equality;
- ordering and cut inequalities;
- projection equations;
- schema compatibility;
- authorization boundaries.

Use:

- direct enumeration for tiny labs;
- SQL joins for relational fragments;
- SAT/SMT for logical/arithmetic constraints;
- property-based generation for counterexamples;
- graph algorithms for simple consistency;
- linear algebra for cohomology.

Return the number or classification of completions, not only one satisfying assignment.

20.5 Ingesting SessionStream evidence

Useful sources include:

- persisted backend events;
- projection cursors;
- current and historical entity rows;
- snapshots;
- WebSocket transport observer records;
- client acknowledgments or test-harness reducer states;
- heartbeat state-machine events/actions;
- schema registry descriptors.

The production observer is bounded and best-effort. Therefore observer absence cannot prove protocol-event absence. Record this as a reliability label on contexts and covers.

For deterministic labs, add a lossless in-memory observer or test trace so the corresponding context can serve as a genuine cover member.

20.6 Report format

A useful report should contain:

```
Invariant: snapshot/live reconstruction
Session:   session-42
Whole:     reconnect interval [107, 119]
Cover:     snapshot-prefix, websocket-suffix
```

Overlap checks:

```
session id           OK
snapshot cut semantics OK
schema version       MISMATCH: v3 vs v2
reducer version      UNKNOWN in suffix context
```

Global completions:

count/classification	0
first unsatisfied core	schema/reducer boundary

Linear diagnostics:

cursor H^0 residual	0
parity H^1 class	nonzero on S-W-C-S cycle

Evidence:

snapshot record #815
UI batch records #816-#829
client reducer trace #44

The report should separate:

- local validation failure;
- overlap mismatch;
- no global completion;
- multiple completions;
- nonzero cohomology class;
- missing evidence.

20.7 Capstone phases

20.7.1 Phase A - Finite toy model

Implement contexts, restrictions, covers, and gluing for parameter assignments. Compute empty/singleton/multiple fibres.

20.7.2 Phase B - Event intervals

Implement the event-history sheaf and prove/test gluing over interval covers.

20.7.3 Phase C - Hydration model

Model snapshot, boundary, and live suffix. Feed generated interleavings through a reference reducer and the current WebSocket algorithm.

20.7.4 Phase D - Trace adapter

Convert SessionStream observer records and snapshots into sections with provenance.

20.7.5 Phase E - Cursor complex

Build a cellular sheaf of ordinal measurements. Compute H^0 consistency and H^1 circulation over selected cycles.

20.7.6 Phase F - Higher witnesses

Add 2-simplices for transactions and end-to-end tests. Observe which classes disappear.

20.7.7 Phase G - Repository integration

Expose the audit as a test helper or CLI. Keep it advisory until its context model and evidence reliability are validated.

20.8 A decision table

Question	First tool
Are these parameters enough to decide an invariant?	Functional dependencies, fibres, constraint solving
Do two implementation paths agree?	Commuting diagram, equalizer, property test
Can several local records form one global state?	Presheaf sections and global-section search
Do compatible local records glue uniquely?	Sheaf condition
What counts as sufficient local evidence?	Site/coverage design
Where is a proposition locally supported?	Subobject classifier and sieves
Does a migration commute with all restrictions?	Natural transformation
Does a boundary preserve pullback invariants?	Left-exact/geometric analysis
Is there irreducible additive circulation around overlaps?	Cech/cellular cohomology
Is logical event identity missing?	Keys, groupoids, equivalence relations, CSP

20.9 Research notebook prompts

1. Which SessionStream concepts currently share uint64 but deserve distinct semantic types?
2. Which restrictions exist semantically but not operationally?
3. Which covers correspond to actual transactions or tests?
4. Where does the architecture rely on pairwise consistency without a higher-order witness?
5. Which properties are stable under restriction?
6. Which truths require transported evidence?
7. Which hidden coordinates can make replay nonfunctorial?
8. Which client-state distinctions should be quotiented as unobservable?
9. Which diagnostics are additive enough for cohomology?
10. Which topology would make a desired presheaf a sheaf, and is that topology operationally honest?

20.10 Final synthesis

The conceptual ladder is now:

Category: declare objects, arrows, composition, and equations.
Limit: construct the universal coherent joint observation.
Functor: translate architecture while preserving composition.
Natural transformation: refactor a whole translation coherently.
Presheaf: assign local data with lawful restriction.
Sheaf: require compatible local data to glue uniquely.
Topos: work in a universe of context-varying sets and logic.
Sieve/local truth: record where a proposition is supported.
Cohomology: measure selected global obstructions in algebraic local data.

For SessionStream, the durable intuition is:

- event history supplies temporal coordinates;
- projections, stores, transport, observers, and clients are local lenses;
- restriction means truncating, projecting, forgetting, or changing context;
- snapshot plus suffix is a gluing construction over a boundary;
- transaction and replay invariants define global-section constraints;
- missing event identity is a missing coordinate or quotient problem;

- local truth distinguishes evidence from omniscient claims;
- nonzero H^1 can reveal irreducible circulation in a chosen additive consistency model.

The mathematics becomes useful when each noun is accompanied by its actual objects, arrows, restriction maps, coverage, and equations.

20.11 Capstone exercises

E20.1 - Audit one invariant. Complete Steps 1-10 for snapshot soundness using current repository data structures.

E20.2 - Build the finite core. Implement Context, Section, Restrictor, and Cover, including law tests.

E20.3 - Global-section solver. Support finite enumeration and classify completion sets.

E20.4 - Hydration property suite. Compare generated concurrent traces against a sequential reference model.

E20.5 - Sieve-valued report. Return the maximal contexts supporting each invariant.

E20.6 - Cohomology engine. Compute H^0 and H^1 over finite fields for a cellular cursor sheaf.

E20.7 - Higher-order test inventory. List every 2-simplex and 3-simplex justified by current transactions or end-to-end tests. Identify missing faces.

E20.8 - Written defense. Produce a ten-page design note explaining where the sheaf model is exact, where it is analogy, and where another formalism is superior.

Appendix A - A Software-to-Mathematics Translation Dictionary

This appendix is a compact lookup table. It is deliberately bidirectional. Read the middle column from left to right when translating software into mathematics, and from right to left when turning a definition into an engineering experiment.

A.1 Categories and diagrams

Mathematical term	Operational reading	SessionStream example
Category	A declared universe of things and lawful transformations, including a chosen meaning of equality	Prefix histories and extensions; typed messages and pure conversions; state machines and trace-preserving simulations
Object	A type, state, context, contract, or semantic configuration in the declared universe	Event, a session prefix, a timeline state, a snapshot context
Arrow	A transformation admitted by the model	Prefix extension, projection, restriction, migration, replay
Identity	A no-change transformation that is neutral under composition	Empty suffix; identity schema migration; restrict a context to itself
Composition	Execute or reason through two compatible transformations as one	Append a suffix, then project; migrate, then encode
Commuting diagram	Different legal paths have the same declared observation	Replay and incremental projection yield observationally equal timeline states
Isomorphism	Lossless translation with a two-sided inverse	A codec on the precisely restricted set of round-trippable values
Monomorphism	Arrow cancellable on the left; an abstract embedding notion	Inclusion of invariant-satisfying traces into all traces, when modeled in <code>Set</code>
Epimorphism	Arrow cancellable on the right; an abstract quotient/surjection notion	Forgetting heartbeat frames into a quotient trace, in a suitable category
Opposite category	Reverse arrows while retaining composition laws	Context inclusion becomes contravariant restriction

The table does not license automatic identification. A Go injection need not be monic in a category whose arrows identify more observations, and a surjective function need not be the right quotient for an operational semantics.

A.2 Universal constructions

Construction	Question it answers	SessionStream reading
Terminal object	Is there a unique arrow from every object into one trivial target?	Forget all detail and retain only successful termination, in a category where that arrow is always defined
Initial object	Is there a unique arrow from one trivial source into every object?	Empty/generated syntax under a free construction, not usually an arbitrary runtime state
Product	What is the universal object carrying two observations at once?	A pair of compatible evidence records before enforcing shared-key agreement
Equalizer	Which inputs make two computations agree?	Histories on which incremental and replay projection coincide
Pullback	What is the universal compatible join over a shared boundary?	Snapshot and suffix agreeing on (SessionId, SnapshotOrdinal)
Coproduct	What is the universal tagged choice among alternatives?	A well-formed protobuf oneof
Coequalizer	What quotient universally identifies two descriptions?	Trace semantics that deliberately forgets selected operational events
Pushout	What is the universal amalgamation of two extensions of a common core?	Joining schema versions along stable shared fields, when conflicts are resolvable
Limit	What object contains a coherent family of observations for an entire diagram?	A globally consistent execution record satisfying all overlap equations
Colimit	What object assembles pieces while imposing declared identifications?	A merged schema or quotient trace built from local contributions

A limit is not just a record with many fields. Its defining property is unique factorization of every other coherent cone. A database transaction may *implement* a pullback-like invariant, but the transaction is not itself a pullback until the modeled objects and arrows satisfy the universal property.

A.3 Functorial structure

Term	Operational reading	Diagnostic question
Functor	A translation preserving identities and composition	Does replaying two suffixes separately agree with replaying their concatenation?
Contravariant functor	A translation reversing arrow direction	Does a larger information context restrict to a smaller one lawfully?
Natural transformation	A componentwise translation commuting with every context change	Does schema migration commute with truncation, replay, and restriction?
Natural isomorphism	A reversible natural transformation	Are two entire representations coherently interchangeable at every context?
Adjunction	A natural bijection between two kinds of map	Is adding structure free relative to forgetting it? Is repair universal relative to inclusion?
Unit	Canonical map into the round trip through an adjunction	Insert generators into free event histories

Term	Operational reading	Diagnostic question
Counit	Canonical map out of the round trip	Evaluate a free history in a concrete reducer
Reflection	Universal repair into a full subcategory	Canonically normalize local data into a separated/sheaf-like form
Left exact	Preserves finite limits	Does a boundary translation preserve terminal objects, products, equalizers, and pullbacks?

For software, the word “natural” should be read as **uniformly compatible with all admitted changes of context**, not as “intuitively reasonable.”

A.4 Presheaves and sheaves

Term	Operational reading	SessionStream reading
Base category	The contexts and admissible context changes	Session-cut-observer triples ordered by loss of information
Presheaf	Local data assigned to each context, with restriction maps	Valid traces, snapshots, cursor evidence, or parameter assignments at each observation context
Section over U	One locally valid value available in context U	A snapshot record at a cut; a trace fragment on an interval
Restriction	Forget, truncate, project, or transport to a smaller context	Drop observer-only fields; truncate to an earlier cut
Matching family	Local sections agreeing on every overlap	Snapshot and buffered suffix agree on session and boundary ordinal
Amalgamation	A section whose restrictions are the given local sections	The reconstructed client execution
Separated presheaf	At most one amalgamation for each matching family	Local observations determine no more than one global state
Sheaf	Exactly one amalgamation for each matching family over every declared cover	Snapshot-plus-suffix reconstructs one coherent client state under the protocol’s coverage assumptions
Site	A category equipped with a declared notion of cover	Contexts plus the families accepted as sufficient evidence
Sieve	A downward-closed family of refinements/arrows	All refinements under which an invariant remains supported
Sheafification	Universal conversion of a presheaf into a sheaf	Identify locally indistinguishable descriptions and add required local gluings
Stalk/germ	Behavior near a point/context after ignoring sufficiently remote detail	Local behavior around an ordinal cut or connection phase

A presheaf is a functor

$$\mathcal{F} : \mathcal{C}^{op} \rightarrow \mathbf{Set}$$

or into another coefficient category.

A.5 Topos and local logic

Term	Operational reading	SessionStream reading
Presheaf topos	A category of all context-indexed sets on a small base category	A universe containing event, snapshot, evidence, and validity presheaves
Subobject	A context-stable predicate or subtype	Sound snapshots included in all snapshots
Subobject classifier Ω	Object of context-sensitive truth values	Sieves of refinements where a property is supported
Characteristic map	Assign a truth value to each local value	Map a snapshot to the sieve of contexts where it passes soundness checks
Heyting logic	Intuitionistic logic of local evidence	“Not refuted” is weaker than “verified”; excluded middle need not hold
Forcing	A context supports a formula according to recursive semantic clauses	An observer context forces “snapshot sound” using available evidence
Geometric morphism	An adjoint change of context with a left-exact inverse-image functor	Translate between full runtime contexts and client-visible contexts while preserving finite-limit structure

Topos language is justified when an actual category of presheaves or sheaves is under discussion. “The architecture is a topos” is normally too vague to be useful.

A.6 Cohomology

Term	Operational reading	SessionStream reading
Nerve	Combinatorial shape recording nonempty or accepted overlaps	Vertices are observers; edges are pairwise shared evidence; filled triangles are trusted three-way witnesses
0-cochain	A choice of coefficient at every vertex/context	One local ordinal coordinate per subsystem
1-cochain	A choice on every pairwise overlap/edge	Claimed offset between two subsystem coordinates
Coboundary δ°	Convert local coordinates into induced pairwise differences	$x_j - x_i$ on every oriented edge
1-cocycle	Edge data satisfying every filled-face compatibility equation	Pairwise offsets whose sum vanishes around every trusted triangle
1-coboundary	Edge data explained by changing vertex coordinates	Offsets arising from a consistent global assignment
H^1	Cocycles modulo coboundaries	Irreducible additive circulation not removable by recalibrating local coordinates
Coefficients	Algebra chosen to encode the diagnostic	Integers for offsets; F_2 for parity; vector spaces for residuals

Cohomology does not automatically certify arbitrary transactional correctness. It answers the question encoded by the chosen complex, stalks, restriction maps, and coefficient algebra.

A.7 Parameter sufficiency as fibres

Let X be the full semantic state, P the visible parameter context, and

$$r_P : \mathcal{F}(X) \rightarrow \mathcal{F}(P)$$

the restriction map. For a supplied request p , its completion fibre is

$$r_P^{-1}(p).$$

The engineering classifications are:

- empty fibre: inconsistent or impossible request;
- singleton fibre: parameters uniquely determine the relevant full state;
- several completions with the same invariant value: enough to decide that invariant, but not enough to reconstruct the state;
- several completions with different invariant values: insufficient parameters for that decision.

This fibre test is often more direct than cohomology. Use the least elaborate formalism that answers the engineering question.

Appendix B - Proof and Modeling Techniques

B.1 The declaration-before-calculation rule

Every formal analysis begins with five declarations:

1. **Objects:** what counts as one state, context, trace, or contract?
2. **Arrows:** which transformations are admitted?
3. **Equality:** when are two arrows or observations considered the same?
4. **Coverage:** which local families count as sufficient to describe a region?
5. **Evidence reliability:** which stores, traces, transactions, and tests justify overlaps?

Most bad category-theoretic software analogies skip at least two of these. A useful notebook habit is to write the declarations in a boxed block before drawing any shape.

B.2 Proving a diagram commutes

To prove a square commutes:

1. Choose an arbitrary input x in the source object.
2. Compute the upper-then-right path.
3. Compute the left-then-bottom path.
4. Normalize only according to the declared arrow equality.
5. Show the results are equal for arbitrary x .

For code, turn the same proof into a property test. Keep the two implementations independent enough that shared bugs are unlikely. If both paths call the same helper that contains the defect, the test witnesses only shared implementation.

B.3 Proving a universal property

For a proposed product P of A and B :

1. Give projections $\pi_1 : P \rightarrow A$ and $\pi_2 : P \rightarrow B$.
2. For arbitrary $f : X \rightarrow A$ and $g : X \rightarrow B$, construct $\langle f, g \rangle : X \rightarrow P$.
3. Prove $\pi_1 \circ \langle f, g \rangle = f$ and $\pi_2 \circ \langle f, g \rangle = g$.
4. Let $h : X \rightarrow P$ satisfy the same equations.
5. Prove $h = \langle f, g \rangle$.

For an equalizer or pullback, replace the projection equations with the relevant compatibility equation. The last uniqueness step is essential; an object that merely stores enough fields may satisfy existence but fail universality.

B.4 Finding a counterexample

When a law seems suspicious, construct the smallest model that can violate it:

- for composition, use two events;
- for a loop obstruction, use a triangle or square;
- for local-versus-global validity, use three variables and one global parity constraint;
- for temporal races, use one cut and two events bracketing it;
- for hidden coordinates, use two executions with the same recorded prefix but different wall-clock or random inputs.

A good counterexample names which premise is retained and which conclusion fails. “It races” is not enough; give the interleaving and resulting observations.

B.5 Proving functoriality

For $F : C \rightarrow D$:

1. State F on objects.
2. State F on arrows.
3. Check that sources and targets match.
4. Prove $F(1_A) = 1_{\{F(A)\}}$.
5. Prove $F(g \circ f) = F(g) \circ F(f)$.

For event folding, the identity law is the empty suffix law and the composition law is chunking invariance. Failures frequently reveal hidden state, duplicate handling, time dependence, or mismatched error policies.

B.6 Proving naturality

For $\eta : F \Rightarrow G$, draw one square for an arbitrary arrow $f : A \rightarrow B$:

$$\begin{array}{ccc} F(A) & \xrightarrow{\eta_A} & G(A) \\ F(f) \downarrow & & \downarrow G(f) \\ F(B) & \xrightarrow{\eta_B} & G(B). \end{array}$$

Then prove

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

In a context category, f often represents restriction. The square asks whether translating first and then forgetting gives the same result as forgetting first and translating in the smaller context.

B.7 Proving the presheaf laws

For every context U , verify

$$\rho_U^U = 1_{\mathcal{F}(U)}.$$

For $W \subseteq V \subseteq U$, verify

$$\rho_W^V \circ \rho_V^U = \rho_W^U.$$

Test these laws with generated chains of contexts. Include awkward cases: deleted/tombstoned entities, empty prefixes, schema-version boundaries, and contexts with missing observer evidence.

B.8 Testing the sheaf condition

For a finite cover $\{U_i \rightarrow U\}$:

1. Generate one section $s_i \in F(U_i)$ for every cover member.
2. Restrict every pair to its pullback/overlap.
3. Reject families that do not match.
4. Enumerate or solve for global sections $s \in F(U)$ restricting to the family.
5. Classify the completion set as empty, singleton, or multiple.

The sheaf condition demands a singleton for every matching family over every accepted cover. Empty means an existence failure. Multiple means a uniqueness/separatedness failure.

B.9 Proving a predicate is a subpresheaf

A predicate $P_U \subseteq F(U)$ defines a subpresheaf when validity is stable under restriction:

$$x \in P_U \implies x|_V \in P_V \quad \text{for every } V \rightarrow U.$$

This is an excellent design filter. A property such as “contains the globally latest event” usually fails under restriction. A property such as “every entity ordinal is at most the declared cut” is often stable under temporal truncation, provided restriction also removes later entities correctly.

B.10 Computing H^1 in a finite constant-coefficient complex

Choose orientations for all edges and faces.

1. Build the vertex-edge incidence matrix D_0 .
2. Build the edge-face incidence matrix D_1 .
3. Verify $D_1 D_0 = 0$.
4. Compute $\ker(D_1)$ and $\text{im}(D_0)$.
5. Compute

$$\dim H^1 = \dim \ker D_1 - \text{rank } D_0.$$

For a graph with no filled faces, $D_1 = 0$, so every edge assignment is a cocycle. A connected tree has $H^1 = 0$. A connected graph with $e - v + 1$ independent cycles has that many dimensions of H^1 for constant field coefficients. Filling a triangle adds a face equation and may kill the corresponding cycle class.

B.11 How to read a proof in this book

Use three passes:

- **Operational pass:** identify the software states and the claimed invariant.
- **Diagram pass:** redraw the objects and arrows without prose.
- **Formal pass:** verify types, equations, and quantifiers.

When a proof feels mysterious, make the category smaller. Replace arbitrary sets by finite sets, arbitrary contexts by a four-element poset, and arbitrary coefficients by F_2 . Calculate by hand, then generalize.

B.12 How to keep analogy under control

Label claims with one of three statuses:

- **Exact model:** the declared software structure literally satisfies the definition.
- **Proposed formalization:** a mathematically precise model chosen for one analysis, not asserted to be canonical.
- **Heuristic analogy:** a shape or phrase that suggests questions but does not yet satisfy a definition.

For example, the finite event-interval assignment with lawful truncation can be an exact presheaf. Calling the WebSocket buffer “the gluing mechanism” is a proposed formalization once sections and covers are declared. Saying “a transaction fills a topological hole” is heuristic until a specific complex and face witness are constructed.

Appendix C - Hints for Every Exercise

Use this appendix only after making a genuine attempt. Most hints identify the missing construction rather than completing it.

C.1 Chapter 1 hints

E1.1. Use at least one category of pure functions, one category of traces or state transitions, and one context poset. Arrow equality must differ across them.

E1.2. Let the common source be an event prefix. One path folds incrementally; the other reloads and replays. Choose equality before claiming commutation.

E1.3. For monicity, cancel the inverse on the left. For epicity, cancel it on the right.

E1.4. Separate syntax-level decoding from semantic round trips. Unknown fields, default values, numeric precision, and normalization usually shrink the invertible subset.

E1.5. Generate well-typed protobuf values, compare normalized semantic values, and include shrinking so the minimal noncommuting input is visible.

E1.6. The method also consumes session metadata and a timeline view and may fail. Consider a reader/error Kleisli category or arrows between whole configurations.

E1.7. Define a client observation function first. Its kernel is the candidate equivalence. Then test whether applying equal live suffixes preserves the relation.

C.2 Chapter 2 hints

E2.1. Pairing is $x \mapsto \text{Pair}\{f(x), g(x)\}$. The equations are projection after pairing; uniqueness follows by field extensionality.

E2.2. Use the universal property twice to construct arrows in both directions, then use uniqueness to prove their composites are identities.

E2.3. Both functions need a common domain, common codomain, deterministic decoding, and a fixed event prefix associated with the snapshot cut.

E2.4. Map each candidate record to the same typed boundary. The pullback contains exactly pairs whose session and ordinal meanings agree.

E2.5. Start with a product of cut candidates and version rows, equalize session/key relations, then select the maximal version below the cut.

E2.6. Use coordinates such as event-appended, entities-applied, cursor-advanced, and UI-published. The allowed sub-object is defined by implication constraints among these bits.

E2.7. Pick `eventCursor=43`, `snapshotOrdinal=42`, and an included entity with `lastEventOrdinal=43`. Each number is legal in isolation.

E2.8. The product stores one value per diagram object. The two maps collect the source-side and target-side images for every diagram arrow.

C.3 Chapter 3 hints

E3.1. A limit is a cone such that every other cone has exactly one mediating arrow preserving all legs. Dualize every arrow for colimits.

E3.2. Product: discrete two-object index. Equalizer: two parallel arrows. Pullback: a cospan.

E3.3. The meet is the longest common prefix. A join would need one history extending both divergent branches.

E3.4. Case analysis gives one arrow out of the coproduct from one arrow out of each summand. Invalid oneof states include multiple active cases outside generated invariants.

E3.5. Analyses depending only on application frames factor through the quotient; liveness latency and heartbeat diagnostics do not.

E3.6. Build arrows from the stable core into both versions. A conflict means no object can receive both maps while preserving validation equations.

E3.7. Try a category that excludes deployments lacking authorization, durable storage, or atomicity. The formal limit may exist only after adjoining a coordinator/transactional component.

E3.8. The formal dual exists once the category is fixed, but authorization is contravariant in information in ways that may make the pushout operationally unsafe.

C.4 Chapter 4 hints

E4.1. Histories may use exact event identity and order; snapshots may ignore entity order; UI traces may quotient ephemeral frames; client states may use render equivalence.

E4.2. For example, a sound and complete snapshot can be stale relative to a later event cursor.

E4.3. Inspect function arguments, process globals, clocks, random sources, network calls, schema registries, and mutable external services.

E4.4. Register hydration before the load. All three batches may enter the buffer; filtering keeps only ordinals strictly greater than the returned cut.

E4.5. Include session identity, cut semantics, ordering convention, projection version, reducer semantics, deduplication identity, and payload schema compatibility.

E4.6. One cursor certifies successful materialization; the other may certify terminal handling under a skip policy. They support different downstream implications.

E4.7. Put entity-version writes and cursor advance in one transaction, with a uniqueness key containing session, projector, and ordinal.

E4.8. Define the oracle as an atomic snapshot at cut n followed by the totally ordered suffix $(n, m]$, not as a copy of the production buffering algorithm.

E4.9. Choose a phrase such as “transaction fills a face.” Either construct the relevant complex and witness, or replace it with an explicit atomicity invariant.

C.5 Chapter 5 hints

E5.1. Objects are natural-number cuts or concrete prefixes; there is at most one arrow when one prefix extends another. Poset categories make associativity automatic.

E5.2. On objects, send a prefix to its folded timeline. On arrows, send suffix extension to the induced state transition.

E5.3. Use a reducer that commits a batch summary once per call. Batch $[a, b]$ then differs from two calls because batching is an unrecorded coordinate.

E5.4. Generate a fixed logical event sequence, partition it many ways, and compare final state plus declared error/cursor observations.

E5.5. Examples: erase payloads, erase heartbeats, erase timings. State exactly which analyses cease to be possible after each erasure.

E5.6. Restrict to valid messages and canonical JSON options. Faithfulness can fail through normalization or unknown-field loss; fullness asks whether every target arrow is encoded from a source arrow.

E5.7. Test $\text{restrict}_m \circ \text{Snapshot}(n) = \text{Snapshot}(m)$. Then separately inspect whether concurrent reads observe one database snapshot.

E5.8. A prefix category is the action category of the free event monoid on states. Extension by a word acts by its reducer endomorphism.

C.6 Chapter 6 hints

E6.1. The vertical arrows extend prefixes; horizontal arrows migrate states. State migration at the larger cut must equal extending the migrated smaller state.

E6.2. Let migration inspect the final prefix length and rewrite all prior IDs only when the length exceeds a threshold.

E6.3. The instrumented execution maps to the uninstrumented one by forgetting records. Blocking callbacks can change timing and thereby violate observational commutation.

E6.4. First quotient away typing indicators or transient progress. Then compare whether both paths induce the same durable semantic state at every cut.

E6.5. Implement migration and extension through separate paths or reference models; otherwise common helper code can make the square pass vacuously.

E6.6. Substitute the two naturality equations and reassociate composition. In tests, check each migration separately and the composite against a direct v1-to-v3 reference.

E6.7. Use a preorder where $x \leq y$ means “no fresher than.” A square may commute up to $\leq$ when one path can lag but never lead.

E6.8. Objects can be projection implementations satisfying named laws; arrows are natural migrations. Isomorphisms require componentwise reversible migrations respecting every law.

C.7 Chapter 7 hints

E7.1. Map a list to composition of event endomorphisms. The empty list maps to identity and concatenation maps to composition.

E7.2. The unit sends a generator to its singleton word. The counit evaluates a word in a monoid. Expand a singleton or arbitrary word for a triangle identity.

E7.3. Two executions with identical events but different times produce different outputs, so no function from event words alone exists. Record time as an event or add it to generators.

E7.4. Specify a subcategory of normalized snapshots. Every map from a raw snapshot into a normalized object must factor uniquely through normalization.

E7.5. Right adjoints preserve limits. Find a simple terminal object or pullback that the adapter fails to preserve to reject the hypothesis quickly.

E7.6. Order parameter sets by inclusion and invariant requirements by implication. Check both directions of $\text{closure}(P) \supseteq R$ iff $P \supseteq \text{required}(R)$ for your definitions.

E7.7. Decide whether decode after encode is identity, a retraction, or normalization. Unknown fields and canonicalization distinguish the cases.

E7.8. The unit $F \rightarrow \mathbf{a}F$ should be universal among maps from F to sheaves. Identify what local identifications or gluings it forces.

C.8 Chapter 8 hints

E8.1. Use tuples `(session, cut, observer-set, schema-version)` and define an information order componentwise. Verify reflexivity, antisymmetry, and transitivity.

E8.2. In a poset, overlap is usually a meet/pullback. Compute the greatest context visible to both observers.

E8.3. Prefix intervals form a line only if histories never branch. Retries, competing replicas, and schema branches add dimensions or nontrivial arrows.

E8.4. A topology is not merely “all overlaps.” Declare which families operationally cover a context and justify identity, base change, and transitivity.

E8.5. Treat the boundary as its own context containing exactly the shared session/cut/schema data. Both past and future contexts restrict to it.

E8.6. Version change may be an arrow only when migration is lawful and directional. Invertible versions form groupoid-like fragments; lossy migrations do not.

E8.7. Draw at least time, representation, and observer axes. Mark actual restriction arrows rather than spatial proximity alone.

C.9 Chapter 9 hints

E9.1. Identity field projection changes nothing; projecting from U to V to W equals direct projection from U to W .

E9.2. Decide whether sections are raw observations, locally validated observations, or globally extendable ones. These choices produce different presheaves.

E9.3. Enumerate full states restricting to a fixed visible assignment. Compare fibre size with the invariant values on those completions.

E9.4. A restriction to an earlier cut must remove later versions and recompute derived cursor fields, not merely lower one number.

E9.5. Define an invariant stable under every restriction, then take the valid sections at each context.

E9.6. Find whether two global sections can have identical restrictions to every cover member. If so, the presheaf is not separated for that cover.

E9.7. Add a global parity or transactional constraint invisible in every local context; compatible local assignments may then have no valid global completion.

E9.8. Store contexts as finite attribute sets or IDs, restrictions as total functions, and property-test identity/composition before implementing covers.

C.10 Chapter 10 hints

E10.1. The total space is the disjoint union of all fibres. Projection sends a local datum to the context over which it is defined.

E10.2. Fix `(session, cut)`. The fibre can contain all valid snapshots, traces, or evidence packages available at exactly that base point.

E10.3. A section chooses one fibre element over every base point in its domain and must project back to that point.

E10.4. Two local sections have the same germ when they agree after restricting to some sufficiently small common neighborhood/context.

E10.5. View the étale-style total space as local values plus their context tags. Restriction behavior supplies the local wiring.

E10.6. An object of Set/B is a map $E \rightarrow B$. A section is an arrow $B \rightarrow E$ whose composite with the projection is 1_B .

E10.7. Plot contexts as base points and local values as layers above them. Draw restriction arrows diagonally between fibres; do not confuse them with base arrows.

C.11 Chapter 11 hints

E11.1. Use finite ordinal intervals, event sequences on each interval, and restriction by subinterval. Matching sequences agree on overlap and concatenate uniquely.

E11.2. Add a condition such as “the total number of events is even.” It can hold or fail globally without being decidable from arbitrary proper subintervals.

E11.3. Take a past/snapshot region and a future/live region whose pullback is the typed cut boundary. State exactly what data lives on the overlap.

E11.4. Construct the client history by taking the snapshot’s represented prefix and appending buffered/live batches with ordinal greater than the cut in order.

E11.5. Any amalgamation must restrict to the same past and suffix. If those regions cover every represented ordinal and reducer application is deterministic, the amalgamation is forced.

E11.6. Overflow means the proposed cover can no longer be represented by retained local sections. Returning an error preserves honesty; silently dropping data would fabricate gluing.

E11.7. Translate the sheaf condition for relation projections into a lossless-join dependency. Pairwise consistency may be insufficient for cyclic schemas.

E11.8. Generate matching local sections, enumerate global candidates, and check exactly one candidate. Include intentionally nonseparated and nongluable fixtures.

C.12 Chapter 12 hints

E12.1. Identity is the singleton interval. Pulling an interval-union cover back to a subinterval yields intersections that still cover. Compose covers by taking all subcovers.

E12.2. Without a shared cut object, past and future sections cannot compare ordering conventions or session identity on their overlap.

E12.3. Candidate observers include event store, projection cursor, timeline store, transport, and client. Accept a family only when a transaction or test justifies completeness.

E12.4. Majority values do not glue uniquely without assumptions on replica identity, version order, and Byzantine behavior. State the fault model.

E12.5. Restrict the base to contexts the principal may observe. Some global sections disappear because the necessary local evidence is inaccessible, not because the fact is false.

E12.6. A family of parameter sets covers when their functional-dependency closures jointly determine the required attributes. Pullback stability is the likely weak point.

E12.7. If two global sections are indistinguishable on every covering context, separated reflection identifies them before missing amalgamations are added.

E12.8. The coarser topology has fewer covers and therefore fewer sheaf obligations. Use a presheaf that fails gluing only for a cover admitted by the finer topology.

C.13 Chapter 13 hints

E13.1. Examples: event-prefix, timeline-state, and evidence presheaves. A natural transformation can project event evidence to cursor evidence.

E13.2. At each context, take Cartesian products. Restrict each coordinate independently.

E13.3. At U , retain states where the two natural transformations have equal components. Restriction preserves equality by naturality.

E13.4. A function chosen independently at U may not commute with every restriction below U . Use a function that inspects data later forgotten.

E13.5. $yU(V)$ is the set of arrows $V \rightarrow U$. A natural transformation is determined by the image of 1_U ; reconstruct it by restriction.

E13.6. Check whether restricting a sound snapshot can introduce an entity beyond the new cut or lose evidence needed by the predicate.

E13.7. State a small base category and identify the presheaf category $\text{Set}^{C^{\text{op}}}$ or a sheaf subcategory in which the model lives.

C.14 Chapter 14 hints

E14.1. Pull back the singleton inclusion $\{\text{true}\} \rightarrow \{\text{false}, \text{true}\}$ along the characteristic function. Its fibre over true is exactly the subset.

E14.2. On a chain, every downward-closed set of incoming refinements is an initial segment, including empty and full.

E14.3. Restrict the section along every arrow into U ; collect exactly those arrows where the restriction lies in the valid subpresheaf.

E14.4. Use $S \Rightarrow T = \{f: V \rightarrow U \mid \text{for every } g: W \rightarrow V, fg \in S \text{ implies } fg \in T\}$. A tiny branching poset makes the calculation visible.

E14.5. Choose a proper nonempty sieve on a branching context. Its intuitionistic negation contains refinements incompatible with it; their union may omit the undecided root.

E14.6. A witness section should restrict to smaller evidence packages. Its support map sends a witness to the refinements where verification remains valid.

E14.7. Return context IDs or a compact antichain generating the sieve. The UI can distinguish verified, refuted, and unsupported regions.

C.15 Chapter 15 hints

E15.1. Phrase each clause in terms of what must hold at the current context and all refinements or some covering family below it.

E15.2. Awaiting forces “a ping was written and its deadline has not yet elapsed” under local clock evidence; it does not force remote process liveness.

E15.3. Include session, cut, entity ordinals, schema/version, and a trusted snapshot read witness. Completeness additionally needs an event/projection frontier relation.

E15.4. Cover by two schema-version contexts, each with its own decoder. Their disjunction is locally witnessed although neither decoder handles the entire union.

E15.5. Give each cover member a local decoder choice; on overlaps the choices disagree, preventing a single global decoder.

E15.6. “The remote process has crashed” is neither established nor refuted from silence in an asynchronous context with only a timeout assumption.

E15.7. Current existence fits internal/local logic, historical existence needs temporal indexing or event history, and eventual existence needs temporal/modal logic.

E15.8. Carry signed or recomputable evidence with restriction metadata. Verification must bind the witness to session, cut, schema, and payload digest.

C.16 Chapter 16 hints

E16.1. Precompose a full-context presheaf with the inclusion functor. The result simply forgets values at contexts unavailable to the client base.

E16.2. Left Kan extension asks for the most freely generated full data; right Kan extension asks for the most constrained/compatible full data consistent with client observations.

E16.3. Products can be tested componentwise. For an equalizer, verify that migration neither creates nor erases agreement between two contract maps.

E16.4. The pullback pairs names with payloads accepted by the registry. Preservation needs injective/faithful handling of names, descriptors, and validation outcomes.

E16.5. Best-effort observation may preserve records it emits but cannot reflect absence: a missing observation may be dropped rather than absent in the source execution.

E16.6. The inclusion of sheaves into presheaves is the direct image/right adjoint; sheafification is the inverse image/left adjoint and preserves finite limits.

E16.7. Explicitly list both context categories, the base functor, inverse/direct image behavior, adjunction, finite-limit tests, and information lost.

C.17 Chapter 17 hints

E17.1. Path: only adjacent pair intersections. Boundary triangle: all pair intersections but empty triple intersection. Filled triangle: triple intersection accepted/nonempty.

E17.2. Do not draw an edge merely because components communicate. Label the common semantic field or evidence object that both restrict to.

E17.3. A transaction spanning event append, entity materialization, and cursor advance can justify a filled triangle if one atomic witness sees all three.

E17.4. Delete any edge whose sole support was best-effort observer correlation, then remove all higher simplices containing that edge.

E17.5. Compute pullbacks such as “same session and ordinal under both observers,” including schema and identity conventions.

E17.6. Use a graph for pairwise relations only, a hypergraph for arbitrary joint relations, a simplicial complex for downward-closed overlaps, and a category when directed transformations matter.

E17.7. Start with a small cube: two cuts, two observer families, two schema versions. Then mark existing arrows and accepted overlaps rather than drawing a decorative cube.

C.18 Chapter 18 hints

E18.1. With vertices $0, 1, 2$ and edges $01, 12$, D_0 sends (x_0, x_1, x_2) to $(x_1 - x_0, x_2 - x_1)$.

E18.2. Write the oriented face sum $(x_2 - x_1) - (x_2 - x_0) + (x_1 - x_0)$.

E18.3. Integrate offsets along a spanning tree. The final edge is consistent exactly when its value equals the induced path difference.

E18.4. Boundary: rank $D_0=2$, no D_1 , so $\dim H^1=3-2=1$. Filled triangle adds one independent face equation.

E18.5. Over F_2 , signs coincide. A cycle sum of 1 means odd parity of missing/duplicated contributions, but even numbers cancel.

E18.6. Vertex stalks may carry local coordinate vectors; edge stalks carry shared comparisons. Each row of D_0 is the difference of the two restriction maps.

E18.7. The sheaf axiom concerns gluing sections over covers. Cohomology measures derived global-section phenomena and can be nonzero for perfectly valid sheaves.

E18.8. Use row reduction to compute ranks and nullspaces. Test known dimensions before applying the engine to repository traces.

C.19 Chapter 19 hints

E19.1. Distinguish event-store frontier, materialized snapshot cut, projector success cursor, entity creation/update ordinals, and UI-producing event ordinal.

E19.2. Sum oriented offsets around the loop. A global coordinate exists exactly when the total is zero.

E19.3. Introduce `IncludedThrough`, `NextToConsume`, and `ProducedBy` as separate types. Make the one-step conversion an explicit arrow rather than an equality claim.

E19.4. Put measurements at vertices and common comparison spaces on edges. The coboundary row is $\rho_{\text{head}}(x_{\text{head}}) - \rho_{\text{tail}}(x_{\text{tail}})$.

E19.5. The face requires the oriented sum of its three edge residuals to vanish. Recompute $\ker D_1$ after adding it.

E19.6. Hash each logical event identity into a bit and XOR. Any even number of duplicate/missing contributions can evade detection.

E19.7. Preserve provenance for every stalk value. Reports should name the supporting records, chosen restrictions, residual vector, and a cycle basis.

E19.8. Authorization, arbitrary protobuf validity, or causal history is usually not additive. Use sets of valid assignments, posets of knowledge, or groupoids of equivalent traces.

C.20 Chapter 20 hints

E20.1. Start with `LastEventOrdinal ≤ SnapshotOrdinal` for every entity. Keep freshness and completeness as separate invariants.

E20.2. Make restrictions explicit total functions and test identity/composition before implementing any solver.

E20.3. A completion classifier needs the entire fibre, or enough search to prove zero/one/many. Record why candidates fail.

E20.4. Generate publication times and snapshot-completion points, but derive the expected sequence from an abstract cut-plus-suffix model.

E20.5. Compute all refinements where evidence proves the invariant, then downward-close the result to obtain a sieve.

E20.6. Begin over F_2 or a small prime field. Return ranks, basis vectors, and representative cycles, not only dimensions.

E20.7. A simplex requires reliable simultaneous evidence, not just pairwise messages. Tie each face to a transaction, lock, or end-to-end test.

E20.8. Organize the defense by exact models, proposed formalizations, rejected analogies, and engineering decisions changed by the analysis.

Appendix D - Selected Solutions

These solutions cover the exercises that carry the main argument of the book. They are models of proof style, not templates to copy mechanically. Many architecture exercises admit several defensible answers because changing the category, equality, or coverage changes the result.

D.1 Solution to E1.3 - Every isomorphism is monic and epic

Let $f: A \rightarrow B$ be an isomorphism with inverse $f^{-1}: B \rightarrow A$.

To prove f is monic, suppose $g, h: X \rightarrow A$ and

$$f \circ g = f \circ h.$$

Compose both sides on the left with f^{-1} :

$$f^{-1} \circ f \circ g = f^{-1} \circ f \circ h.$$

By associativity and the inverse law,

$$1_A \circ g = 1_A \circ h,$$

so $g=h$.

To prove f is epic, suppose $g, h: B \rightarrow Y$ and

$$g \circ f = h \circ f.$$

Compose both sides on the right with f^{-1} :

$$g \circ f \circ f^{-1} = h \circ f \circ f^{-1}.$$

Therefore $g=h$. Thus every isomorphism is both monic and epic. The converse is not valid in every category.

D.2 Solution to E2.2 - Products are unique up to unique isomorphism

Let (P, p_1, p_2) and (Q, q_1, q_2) both be products of A and B .

Because P is a product and Q has arrows $q_1: Q \rightarrow A$, $q_2: Q \rightarrow B$, there is a unique

$$u: Q \rightarrow P$$

such that

$$p_1 u = q_1, \quad p_2 u = q_2.$$

Similarly, because Q is a product, there is a unique

$$v : P \rightarrow Q$$

with

$$q_1 v = p_1, \quad q_2 v = p_2.$$

Now

$$p_1(uv) = (p_1 u)v = q_1 v = p_1,$$

and similarly $p_2(uv) = p_2$. Both $uv : P \rightarrow P$ and $1_P : P \rightarrow P$ have the same composites with both product projections. By the uniqueness clause for P ,

$$uv = 1_P.$$

The dual calculation gives $vu = 1_Q$. Hence u and v are inverse isomorphisms. Their equations with the projections also make the isomorphism unique.

The engineering lesson is that a product is determined by behavior, not by a preferred struct layout.

D.3 Solution to E2.7 - Locally legal numbers, globally incoherent cut

Consider the record

```

SessionId:      s-1
EventCursor:    43
ProjectionCursor: 42
SnapshotOrdinal: 42
Entity.LastEventOrdinal: 43

```

Each coordinate can be a valid uint64. The event store may legitimately contain events through 43; the projector may legitimately have completed only through 42; and 42 is a legitimate snapshot ordinal.

The inconsistency is the simultaneous claim that an entity updated by event 43 belongs to the snapshot representing the materialized prefix through 42. Snapshot soundness requires

$$\text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal}.$$

Here $43 \leq 42$ is false. The product of the local value sets contains this tuple, but the subobject of coherent cuts does not.

D.4 Solution to E3.3 - Meet and join of divergent histories

Order histories by prefix:

$$h \leq k \iff h \text{ is a prefix of } k.$$

Let

```

h = [a, b, c, d]
k = [a, b, c, e]

```

Their common lower bounds are the prefixes of $[a, b, c]$. The greatest such lower bound is

$$h \wedge k = [a, b, c].$$

A join would be a least history extending both h and k . In an ordinary linear event history, no sequence can have both $[a, b, c, d]$ and $[a, b, c, e]$ as prefixes when $d \neq e$. Therefore no join exists.

If the model adds an explicit merge event or replaces histories by branch sets, a join may exist in the new category. The failure is model-relative.

D.5 Solution to E4.4 - Hydration interleavings at cuts 41, 42, and 43

Assume the subscription is placed in the hydrating state before the snapshot load begins. While the load is outstanding, live batches produced by events 41, 42, and 43 are buffered in ordinal order:

```
buffer = [41, 42, 43]
```

The transport sends the returned snapshot first and then retains only buffered batches whose ordinal is strictly greater than `SnapshotOrdinal`.

Snapshot cut 41

The snapshot represents the durable prefix through 41. Filtering produces

```
[42, 43]
```

so the client observes

```
Snapshot(through 41), UI(42), UI(43)
```

Snapshot cut 42

The snapshot already represents durable state through 42. Filtering produces

```
[43]
```

and the client observes

```
Snapshot(through 42), UI(43)
```

Snapshot cut 43

All buffered event effects are already represented by the snapshot, so filtering produces the empty suffix:

```
Snapshot(through 43)
```

The critical equation is not “send every buffered batch.” It is

send exactly batches with ordinal $> n$,

where n is the cut certified by the snapshot. Registering hydration before the load prevents events published during the load from falling into the gap between snapshot and live subscription.

D.6 Solution to E4.6 - Two cursor types

Define two semantic types:

```
type MaterializedThrough uint64
// Every event <= n has successfully contributed the promised durable materialization.
```

```
type HandledThrough uint64
// Every event <= n has reached a terminal processing decision: applied, skipped, or quarantined.
```

Under a fail policy, these values may coincide. Under an advance-on-error policy, HandledThrough can advance while MaterializedThrough cannot.

Suppose event 18 fails timeline projection and policy permits advancing:

```
HandledThrough      = 18
MaterializedThrough = 17
```

If both are represented as one untyped ProjectionCursor=18, a downstream snapshot routine may infer that state through 18 is materialized. The false square is:

```
Event prefix through 18  ----process----> cursor 18
      |                               |
      | project                     | interpret as materialized
      v                               v
Timeline through 17  -----> Timeline through 18
```

The bottom-right object does not exist under the promised semantics. Distinct types force an explicit conversion, and no total conversion from HandledThrough to MaterializedThrough exists without additional evidence.

D.7 Solution to E5.3 - A reducer that is not functorial under batching

Let state be a list of batches, and define a batch API:

```
applyBatch(state, events) = append(state, names(events))
```

Then

```
applyBatch([], [a,b]) = [[a,b]]
```

but

```
applyBatch(applyBatch([], [a]), [b]) = [[a],[b]].
```

Thus

$$R([a,b]) \neq R([b]) \circ R([a]).$$

The failed law is preservation of composition/concatenation. Batch boundaries are semantically observable but were omitted from the generator history. There are two possible repairs:

1. make the reducer truly event-wise so batch partition is unobservable; or
2. include batch-boundary events in the source category.

D.8 Solution to E5.7 - Temporal restriction and the current SQLite store

For a fixed, nonmutating database with complete immutable entity versions, the intended historical semantics is

$$\text{restrict}_m(\text{Snapshot}(n)) = \text{Snapshot}(m) \quad (m \leq n).$$

The current SQLite Snapshot(ctx,sid,asOf) path for asOf>0 chooses the latest version of each entity whose version ordinal is at most the chosen cut. Under the following assumptions, the law is plausible and directly testable:

- event/version ordinals are immutable;
- every update and tombstone has a version row;
- entity identity is stable;
- decoding and ordering normalization are deterministic;
- restriction to m chooses the latest version at or before m and removes tombstoned entities.

The current `asOf==0` path is a different question. It first reads the session cursor and then reads the current entity table. Those operations are not expressed by the method as one explicit read transaction. Apply itself is transactional, and the store restricts its connection pool, but the returned Snapshot value does not carry a witness that its cursor and rows were observed in one database snapshot. Under concurrent mutation, the semantic law therefore requires a separate isolation argument or a read transaction; it should not be inferred merely from the type signature.

A suitable test has two layers:

1. a deterministic historical test over a quiescent database; and
2. a concurrent test or isolation proof for the “current snapshot” path.

This separates functorial temporal semantics from database scheduling.

D.9 Solution to E6.2 - A final-cut migration that is not natural

Let version 1 timeline IDs be plain strings. Define a migration at cut n by

$$\eta_n(T) = \begin{cases} T, & n < 2, \\ \text{prefix every entity ID with 'v2:',} & n \geq 2. \end{cases}$$

At final cut 2, the migration can produce a valid version 2 state. But consider extension from cut 1 to 2.

- Migrate at cut 1: IDs remain unprefixed. Then process event 2, whose version 2 projector updates `v2:item`.
- Extend in version 1 first: event 2 updates `item`. Then migrate at cut 2, yielding `v2:item`.

The first path may contain both `item` and `v2:item`, while the second contains only `v2:item`. Therefore the naturality square fails. Correctness at one final cut is weaker than coherence over every prefix arrow.

D.10 Solution to E6.6 - Composite migrations are natural

Let

$$\eta : F \Rightarrow G, \quad \theta : G \Rightarrow H$$

be natural transformations. Define their componentwise composite by

$$(\theta\eta)_A = \theta_A \circ \eta_A.$$

For any $f : A \rightarrow B$,

$$\begin{aligned} H(f) \circ (\theta_A \circ \eta_A) &= (H(f) \circ \theta_A) \circ \eta_A \\ &= (\theta_B \circ G(f)) \circ \eta_A \\ &= \theta_B \circ (G(f) \circ \eta_A) \\ &= \theta_B \circ (\eta_B \circ F(f)) \\ &= (\theta_B \circ \eta_B) \circ F(f). \end{aligned}$$

The second and fourth equalities use naturality of θ and η . Hence the composite is natural.

A test suite should check random context arrows for v1-to-v2 and v2-to-v3 separately, then compare their composite with an independently implemented direct v1-to-v3 reference.

D.11 Solution to E7.1 - Extending event actions to lists

Let T be the timeline-state set and suppose every event $e \in E$ determines an endomorphism

$$a(e) : T \rightarrow T.$$

Define the action of a word recursively:

$$\hat{a}([]) = 1_T,$$

$$\hat{a}([e_1, \dots, e_n]) = a(e_n) \circ \dots \circ a(e_1).$$

Then

$$\hat{a}(u ++ v) = \hat{a}(v) \circ \hat{a}(u),$$

with the direction determined by the convention that the first event is applied first. The empty word law follows immediately. Uniqueness follows by induction: any monoid action agreeing with $\hat{a}$ on singleton words must agree on every concatenation of singletons.

D.12 Solution to E7.2 - Unit, counit, and a triangle identity

For the free-monoid adjunction

$$F : \mathbf{Set} \rightleftarrows \mathbf{Mon} : U,$$

$F(X) = X^*$, the set of finite words on X , and U forgets multiplication.

The unit is

$$\eta_X : X \rightarrow UFX, \quad x \mapsto [x].$$

The counit is

$$\varepsilon_M : FUM \rightarrow M,$$

which multiplies a word of elements of M .

Check the triangle identity

$$\varepsilon_{FX} \circ F(\eta_X) = 1_{FX}.$$

For a word $[x_1, \dots, x_n]$, $F(\eta_X)$ produces the word of singleton words

$$[[x_1], \dots, [x_n]].$$

The counit in the free monoid concatenates those words, returning

$$[x_1, \dots, x_n].$$

Thus the composite is the identity.

D.13 Solution to E7.3 - Wall-clock time is a missing generator

Suppose a projector assigns

```
entity.CreatedAt = time.Now()
```

Two executions can have the same initial state and identical recorded event word w but run at different wall-clock times. Their outputs differ. Therefore there is no well-defined function

$$E^* \rightarrow \text{End}(T)$$

that explains the implementation: the result is not determined by the event word.

Repairs include:

- record the relevant timestamp in the canonical event;
- extend generators from `Event` to `(Event, ObservedTime)`;
- pass an explicit deterministic clock coordinate and include it in the base context;
- remove time from durable projection output.

For replayable event sourcing, recording the fact is usually the cleanest repair.

D.14 Solution to E8.1 - A finite base poset of observation contexts

Fix a schema version v . Define a context as

$$U = (s, n, O, v),$$

where s is a session, n a cut, and O a finite set of observers or visible dimensions.

Define

$$(s, m, P, v) \leq (s, n, O, v)$$

exactly when

$$m \leq n \quad \text{and} \quad P \subseteq O.$$

Contexts with different sessions or schema versions are incomparable in this simple model.

- Reflexivity follows from $n \leq n$ and $O \subseteq O$.
- Antisymmetry follows because $m \leq n \leq m$ gives $m = n$, and mutual subset inclusion gives $P = O$.
- Transitivity follows from transitivity of $\leq$ on ordinals and $\subseteq$ on observer sets.

Turn the poset into a category by placing one arrow $V \rightarrow U$ exactly when $V \leq U$. A presheaf is contravariant, so this arrow induces restriction

$$\mathcal{F}(U) \rightarrow \mathcal{F}(V) :$$

forget observers and truncate from n to m .

A richer model can add schema-migration arrows instead of making versions incomparable. That addition creates a category rather than a simple product poset.

D.15 Solution to E9.1 - Parameter assignments form a presheaf

Let the base category be finite parameter contexts ordered by inclusion. For a field set U , let

$$\mathcal{F}(U) = \prod_{p \in U} \text{Value}(p),$$

the set of typed assignments to the fields in U .

For $V \subseteq U$, define

$$\rho_V^U : \mathcal{F}(U) \rightarrow \mathcal{F}(V)$$

by dropping fields outside V .

Identity holds because dropping no fields changes nothing:

$$\rho_U^U = 1_{\mathcal{F}(U)}.$$

For $W \subseteq V \subseteq U$, dropping from U to V and then to W has the same result as dropping directly to W :

$$\rho_W^V \rho_V^U = \rho_W^U.$$

Therefore $\mathcal{F}$ is a presheaf of parameter assignments.

If $\mathcal{F}(U)$ is restricted to *locally valid* assignments, one must additionally prove that validity is preserved when fields are forgotten. Many validation rules are not stable this way; that variant may fail to be a presheaf unless validity is defined contextually.

D.16 Solution to E9.3 - Fibres and invariant sufficiency

Suppose the full order state is

`(orderId, version, price, quantity, currency)`

and the visible request is

`p = (orderId="o7", amount=30)`

with invariant

$$I(x) : \text{amount} = \text{price} \cdot \text{quantity}.$$

Let r_P forget all full-state fields except the supplied parameters. If the database permits two completions,

`x1: price=10, quantity=3, currency=USD`

`x2: price=15, quantity=2, currency=USD`

then the fibre has at least two elements. Both make I true, so the request is sufficient to decide this invariant even though it does not reconstruct the order state.

If a third completion is possible,

`x3: price=12, quantity=2, currency=USD`

then $I(x_3)$ is false. The same visible parameters now admit different invariant values, so they are insufficient.

Thus singleton fibres imply full determination, but invariant sufficiency only requires constancy of I on the fibre.

D.17 Solution to E9.7 - Pairwise agreement without a valid global completion

Let global assignments be triples $(A, B, C) \in \mathbb{F}_2^3$ constrained by

$$A + B + C = 1 \pmod{2}.$$

Take local contexts AB, BC, and AC. Choose sections

$$s_{AB} = (0, 0), \quad s_{BC} = (0, 0), \quad s_{AC} = (0, 0).$$

They agree on every one-variable overlap: all assign 0 to the shared variable. Their set-theoretic union is the unique assignment

$$(A, B, C) = (0, 0, 0).$$

But this assignment violates the global constraint because $0+0+0 \neq 1$ in $\mathbb{F}_2$. Therefore the matching family has no global section in the presheaf of *globally admissible* assignments.

The example separates two ideas:

- compatibility of shared coordinates; and
- satisfaction of a genuinely global constraint.

Local overlap agreement alone does not guarantee global admissibility unless the chosen sheaf/site model makes the constraint local.

D.18 Solution to E10.6 - Sections in a slice category

An object of the slice category $\mathbf{Set}/B$ is a function

$$p : E \rightarrow B.$$

Think of E as a total space of local values and B as the base of contexts. The fibre over $b \in B$ is

$$E_b = p^{-1}(b).$$

A section of p is a function

$$s : B \rightarrow E$$

such that

$$p \circ s = 1_B.$$

For every b , this equation says

$$p(s(b)) = b,$$

so s selects one value in the correct fibre over each base point.

For *SessionStream*, let B be session-cut pairs and let E contain tagged snapshot candidates. A section chooses one snapshot candidate for each session-cut pair without ever choosing a value over the wrong context.

D.19 Solution to E11.1 - Event histories form a sheaf on finite intervals

Fix a linearly ordered set of event ordinals and a set E of event values. Let the base regions be finite intervals. Define

$$\mathcal{F}(I) = E^I,$$

the set of event assignments on interval I . Restriction is ordinary function restriction.

Take a cover $\{I_k\}$ of an interval I and a matching family

$$s_k : I_k \rightarrow E$$

such that

$$s_i|_{I_i \cap I_j} = s_j|_{I_i \cap I_j}$$

for all i, j .

Define $s : I \rightarrow E$ by choosing any k with $n \in I_k$ and setting

$$s(n) = s_k(n).$$

This is well-defined because if n lies in two cover members, matching says both values agree. It restricts to each s_k , so an amalgamation exists.

If $t : I \rightarrow E$ is another amalgamation, every $n \in I$ lies in some I_k , and

$$t(n) = t|_{I_k}(n) = s_k(n) = s(n).$$

Thus $t=s$; the amalgamation is unique. Therefore $\mathcal{F}$ is a sheaf for interval-union covers.

D.20 Solutions to E11.4 and E11.5 - Existence and uniqueness for hydration gluing

Let the snapshot section s_P represent durable client state through cut n , and let the suffix section s_F be the totally ordered UI batches with ordinals in $(n, m]$. Assume:

1. both refer to the same session and schema family;
2. the snapshot accurately represents the durable prefix through n ;
3. every retained suffix batch has ordinal greater than n ;
4. suffix batches are delivered in ordinal order;
5. no required batch in $(n, m]$ is omitted;
6. the client reducer is deterministic for the represented inputs;
7. duplicate identity and update semantics are fixed.

Existence

Construct

$$s = \text{foldClient}(s_P, s_F).$$

By construction, restricting s to the past region returns the snapshot state. Restricting its transition trace to $(n, m]$ returns exactly the suffix. The shared boundary is $B_s(n)$, and both local sections agree there by assumptions 1-3. Therefore s is an amalgamation.

Uniqueness

Let t be another amalgamation. Its restriction at the cut is the same snapshot state, and its transition sequence after the cut is the same ordered suffix. Determinism gives the same state after the first suffix event, then after the second, and so on by induction. Therefore $t=s$ at every represented cut.

Uniqueness fails if the reducer contains hidden inputs or if two event orders are admitted. Existence fails if the buffer overflows and a necessary suffix batch is discarded. The current transport chooses explicit failure on overflow rather than claiming a nonexistent amalgamation.

D.21 Solution to E12.1 - Interval-union covers form a pretopology

Let objects be intervals and arrows be inclusions. Declare $\{I_k \rightarrow I\}$ covering when

$$I = \bigcup_k I_k.$$

Identity. The singleton family $\{I \rightarrow I\}$ covers because its union is I .

Pullback stability. Given $J \rightarrow I$, the pullback of $I_k \rightarrow I$ along $J \rightarrow I$ is $J \cap I_k \rightarrow J$. Since

$$J = J \cap I = J \cap \bigcup_k I_k = \bigcup_k (J \cap I_k),$$

the pulled-back family covers J .

Transitivity. Suppose $\{I_k \rightarrow I\}$ covers I , and each $\{I_{k\ell} \rightarrow I_k\}$ covers I_k . Then

$$I = \bigcup_k I_k = \bigcup_k \bigcup_{\ell} I_{k\ell},$$

so the composite family covers I .

Thus interval-union covers satisfy the pretopology axioms.

D.22 Solution to E12.8 - Coarse versus fine topology

Let a base category contain $U_1 \rightarrow U$, $U_2 \rightarrow U$, and their overlap, with $\{U_1, U_2\}$ capable of covering U .

Define a presheaf by

$$F(U) = \{0, 1\}, \quad F(U_1) = F(U_2) = \{*\},$$

and let both restriction maps send 0 and 1 to $*$.

For the **coarse topology** containing only identity covers, F is a sheaf: the only gluing obligations are trivial.

For the **fine topology** that also declares $\{U_1, U_2\}$ a cover, the unique local family $(*, *)$ is matching, but it has two amalgamations, 0 and 1 . Hence F is not separated and therefore not a sheaf for the fine topology.

Finer topologies impose more local-to-global equations and generally have fewer sheaves.

D.23 Solution to E13.2 - Pointwise product of presheaves

Let E be an event-evidence presheaf and T a timeline-state presheaf on the same base category. Define

$$(E \times T)(U) = E(U) \times T(U).$$

For $f : V \rightarrow U$, define restriction by

$$(E \times T)(f)(e, t) = (E(f)(e), T(f)(t)).$$

Identity and composition hold componentwise because they hold for E and T .

A section over U is merely a pair $(\text{eventEvidence}, \text{timelineState})$. It is not automatically a *compatible* pair. Compatibility is imposed by taking an equalizer or pullback inside this product.

D.24 Solution to E13.3 - Agreement as a presheaf equalizer

Let P be a history presheaf and T a timeline-state presheaf. Suppose two natural transformations

$$\alpha, \beta : P \Rightarrow T$$

compute timeline state by incremental projection and replay.

Define

$$A(U) = \{x \in P(U) \mid \alpha_U(x) = \beta_U(x)\}.$$

For $f : V \rightarrow U$ and $x \in A(U)$, naturality gives

$$\alpha_V(P(f)x) = T(f)(\alpha_U x) = T(f)(\beta_U x) = \beta_V(P(f)x).$$

Therefore the restriction $P(f)x$ lies in $A(V)$. The inclusion $A \rightarrow P$ is the equalizer of α and β in the presheaf category, computed context by context.

D.25 Solution to E14.2 - Sieves on a four-element chain

Let

$$0 < 1 < 2 < 3$$

be a context chain, with one arrow $i \rightarrow j$ whenever $i \leq j$. A sieve on the top object 3 is a set of arrows into 3 closed under precomposition. It is therefore determined by a downward-closed set of domains.

The sieves are:

$$\emptyset,$$

$$\{0 \rightarrow 3\},$$

$$\{0 \rightarrow 3, 1 \rightarrow 3\},$$

$$\{0 \rightarrow 3, 1 \rightarrow 3, 2 \rightarrow 3\},$$

and

$$\{0 \rightarrow 3, 1 \rightarrow 3, 2 \rightarrow 3, 1_3\}.$$

The first is bottom and the last is top. At object k , the same pattern gives the empty sieve plus every initial segment of the arrows with domains at most k .

Operationally, a sieve on a context records a downward-stable set of refinements where evidence continues to support a property.

D.26 Solution to E14.5 - Failure of excluded middle

Take a branching poset with a top context U and two incomparable refinements $A \rightarrow U$ and $B \rightarrow U$, with no nontrivial common refinement.

Let

$$S = \{A \rightarrow U\}.$$

This is a sieve. Its intuitionistic negation consists of arrows whose every further refinement avoids S . The arrow $B \rightarrow U$ lies in the negation of S , while $A \rightarrow U$ and 1_U do not. Hence

$$S \cup \neg S = \{A \rightarrow U, B \rightarrow U\},$$

which is not the top sieve because it omits 1_U .

A software interpretation is a deployment context in which one refinement supplies a schema- $v1$ witness and another supplies evidence incompatible with schema $v1$. At the unresolved parent context, the proposition is neither globally established nor globally refuted. Refinement determines it; the current context does not.

D.27 Solution to E15.2 - What the heartbeat phases justify

The pure heartbeat machine has phases Booting, Idle, Writing, Awaiting, Suspected, and Stopped. A careful local-logic reading is:

Phase	Locally justified proposition	Not justified
Booting	The detector has not entered its operational idle cycle	Remote endpoint status
Idle	No current challenge is awaiting acknowledgement	The peer is alive now
Writing	A challenge generation and nonce have been selected; write completion is pending	The client has received the ping
Awaiting	The ping write completed locally and a deadline is armed for this generation	The remote process is healthy or even scheduled
Suspected	Under the configured timeout and local clock observations, the expected matching pong was not accepted in time, or the ping write failed	Proof of remote crash
Stopped	The detector will perform no further heartbeat transitions	Any causal account of why the connection ended

The key distinction is between a protocol fact and an ontological claim. A deadline event supports “suspected under this timing assumption.” In an asynchronous system it does not force “the remote process crashed.”

D.28 Solution to E15.6 - Neither crash nor non-crash is forced

Let U be the context containing:

- a ping write timestamp;
- an elapsed configured deadline;
- no accepted matching pong;
- no independent process or network oracle.

Consider the proposition

$$P = \text{“the remote process has crashed.”}$$

U does not force P : a delayed network, paused browser event loop, or scheduler stall is compatible with the same evidence.

U also does not force the negation of P : an actual crash is equally compatible with the evidence.

What U can force is the weaker, operational proposition

$$\text{“the connection is suspected by the configured detector.”}$$

This example is not philosophical ornament. It prevents monitoring software from silently upgrading a local timeout judgment into an unsupported global fact.

D.29 Solution to E16.6 - Sheafification and inclusion as a geometric morphism

Let

$$a : \widehat{\mathcal{C}} \rightarrow \text{Sh}(\mathcal{C}, J)$$

be sheafification and

$$i : \text{Sh}(\mathcal{C}, J) \hookrightarrow \widehat{\mathcal{C}}$$

be inclusion. There is an adjunction

$$a \dashv i.$$

For a Grothendieck topology, sheafification preserves finite limits. Therefore a is left exact.

A geometric morphism $f : E \rightarrow F$ consists of an inverse-image functor

$$f^* : F \rightarrow E$$

that is left exact and left adjoint to a direct-image functor f_* . Taking

$$E = \text{Sh}(\mathcal{C}, J), \quad F = \widehat{\mathcal{C}}, \quad f^* = a, \quad f_* = i,$$

gives a geometric morphism from the sheaf topos into the presheaf topos.

The direction is worth checking: inverse image travels from the codomain to the domain of the geometric morphism.

D.30 Solution to E17.1 - Three nerves from the same three labels

Let the cover members be U_0, U_1, U_2 .

Path

Suppose

$$U_0 \cap U_1 \neq \emptyset, \quad U_1 \cap U_2 \neq \emptyset, \quad U_0 \cap U_2 = \emptyset.$$

The nerve has vertices 0, 1, 2 and edges 01, 12. It is a path.

Triangle boundary

Suppose every pair intersects but

$$U_0 \cap U_1 \cap U_2 = \emptyset.$$

The nerve has all three edges but no 2-simplex. Its geometric realization is a loop.

Filled triangle

Suppose the triple intersection is nonempty or the model accepts a reliable three-way witness. Then the nerve also contains the 2-simplex 012. The loop is filled.

In architecture work, “nonempty intersection” may mean an actual common context, transaction, or end-to-end evidence object. Pairwise communication alone does not justify filling the triangle.

D.31 Solution to E18.1 - Cochains on a three-vertex path

Orient the path as

$$0 \dashrightarrow 1 \dashrightarrow 2$$

Over a field k ,

$$C^0 = k^3, \quad C^1 = k^2.$$

For a vertex assignment $x = (x_0, x_1, x_2)$,

$$\delta^0 x = (x_1 - x_0, x_2 - x_1).$$

With column vectors, the coboundary matrix is

$$D_0 = \begin{bmatrix} -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}.$$

There are no 2-simplices, so $C^2 = 0$. The kernel of D_0 consists of constant assignments, giving one-dimensional H^0 . Since the graph is a tree, every edge assignment is a coboundary and $H^1 = 0$.

D.32 Solution to E18.2 - Why the next coboundary is zero

For an oriented triple $0 < 1 < 2$, let

$$(\delta^0 x)_{ij} = x_j - x_i.$$

Then

$$\begin{aligned}
 (\delta^1 \delta^0 x)_{012} &= (\delta^0 x)_{12} - (\delta^0 x)_{02} + (\delta^0 x)_{01} \\
 &= (x_2 - x_1) - (x_2 - x_0) + (x_1 - x_0) \\
 &= 0.
 \end{aligned}$$

Every vertex term occurs twice with opposite signs. This is the concrete form of

$$\delta^1 \delta^0 = 0.$$

Therefore every coboundary is automatically a cocycle, making the quotient $\ker(\delta^1)/\text{im}(\delta^0)$ well-defined.

D.33 Solution to E18.3 - Solving offsets on a square

Orient the cycle

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$

and let edge offsets be

$$a = (a_{01}, a_{12}, a_{23}, a_{30}).$$

We seek vertex coordinates satisfying

$$a_{01} = x_1 - x_0, \quad a_{12} = x_2 - x_1, \quad a_{23} = x_3 - x_2, \quad a_{30} = x_0 - x_3.$$

Set $x_0=0$. Then the first three equations force

$$x_1 = a_{01},$$

$$x_2 = a_{01} + a_{12},$$

$$x_3 = a_{01} + a_{12} + a_{23}.$$

The last equation holds exactly when

$$a_{01} + a_{12} + a_{23} + a_{30} = 0.$$

Examples:

- $(1, 2, -1, -2)$ integrates to $(0, 1, 3, 2)$ and has cycle sum 0.
- $(1, 0, 0, 0)$ has cycle sum 1, so no global coordinate exists.

Changing x_0 adds a constant to every vertex and does not change offsets. That freedom is H^0 , not an obstruction.

D.34 Solution to E18.4 - H^1 of a triangle boundary and a filled triangle

Work over $\mathbb{R}$.

Boundary triangle

There are three vertices and three edges, with no 2-simplex:

$$\dim C^0 = 3, \quad \dim C^1 = 3.$$

For a connected graph, rank $D_0=2$. Since D_1 is absent, every 1-cochain is a cocycle:

$$\dim \ker D_1 = 3.$$

Therefore

$$\dim H^1 = 3 - 2 = 1.$$

A representative is unit circulation around the boundary.

Filled triangle

Now add one face, so $C^2=\mathbb{R}$. The face coboundary has rank 1, hence

$$\dim \ker D_1 = 3 - 1 = 2.$$

But $\dim D_0$ also has dimension 2, and it lies in $\ker D_1$. Therefore

$$H^1 = 0.$$

The added face equation kills the independent circulation. In architecture terms, a reliable three-way witness can remove a pairwise-consistency degree of freedom, provided the chosen coefficients and restrictions correctly model the witness.

D.35 Solution to E18.7 - Cohomology is not failure of the sheaf axiom

The sheaf axiom is a statement about matching families over declared covers: compatible local sections must glue uniquely.

Sheaf cohomology is a derived invariant of the global-sections functor. A sheaf can satisfy every gluing axiom and still have nonzero higher cohomology because global sections need not be an exact operation. The standard constant sheaf on a circle-like space is the elementary geometric intuition: it is still a sheaf, while the topology supports a nontrivial first cohomology class.

In the software model, a nonzero H^1 therefore means:

for the selected coefficient sheaf and overlap complex, there is a cocycle not explained by a global 0-cochain.

It does **not** mean “the presheaf forgot to satisfy the sheaf condition,” and it does not by itself identify a production bug.

D.36 Solution to E19.2 - A five-vertex cursor loop

Let the oriented offsets around a five-vertex loop be

$$(0, 1, 0, -1, 0).$$

Their sum is 0, so a global coordinate exists. Set $x_0=10$; integration gives

$$x_1 = 10, \quad x_2 = 11, \quad x_3 = 11, \quad x_4 = 10,$$

and the final edge returns to $x_0=10$.

If the last offset is changed to 1, the cycle sum becomes 1. No assignment of vertex coordinates can realize all five comparisons. The residual is independent of the arbitrary starting coordinate and represents the loop obstruction.

D.37 Solution to E19.3 - Repairing an off-by-one contradiction with types

Suppose one subsystem reports the last included event and another reports the next event to consume. Treating both as `uint64` cursor creates the false equation

$$\text{includedThrough} = \text{nextToConsume}.$$

Introduce distinct types:

```
type IncludedThrough uint64
type NextToConsume uint64
type ProducedBy uint64
```

and explicit conversions:

```
func NextAfter(n IncludedThrough) NextToConsume {
    return NextToConsume(uint64(n) + 1)
}

func Previous(n NextToConsume) (IncludedThrough, bool) {
    if n == 0 { return 0, false }
    return IncludedThrough(uint64(n) - 1), true
}
```

Now an apparent +1 circulation may disappear because edges compare coordinates through the correct conversion maps rather than by subtraction in one undifferentiated stalk.

This is an important negative result: some “cohomological obstruction” candidates are merely type errors in the coefficient model.

D.38 Solution to E20.1 - A sheaf audit of snapshot soundness

We audit the claim:

$$\forall x \in \text{Entities}(S), \quad x.\text{LastEventOrdinal} \leq S.\text{SnapshotOrdinal}.$$

This is **soundness only**. It does not claim that every event through the cut is reflected, nor that the snapshot is fresh relative to the event store.

1. Global context

For one session s , let U_s be the context of an assembled snapshot response together with enough provenance to interpret its cut and entity rows.

2. Local contexts

Use:

- M_s : snapshot metadata, including session and cut;
- $R_{\{s,k,i\}}$: one entity row identified by kind and ID, including its update ordinal and payload;
- V_s : schema/version evidence used to decode rows;
- optionally Q_s : database read/transaction evidence binding metadata and rows to one observation.

3. Sections

A section contains typed facts plus provenance, not only values. For example:

```
SnapshotMeta(session=s, cut=42, source=query-17)
EntityRow(session=s, kind=Message, id=m7, last=41, source=query-17)
```

4. Restrictions

Both metadata and entity sections restrict to a boundary containing session identity and cut semantics. Entity rows also restrict to schema name/descriptor evidence.

5. Coverage

The family $\{M_s, \text{all } R_{\{s,k,i\}}, V_s\}$ covers U_s only when the architecture justifies that it is complete for the returned response. A read transaction, immutable historical asOf query, or equivalent serialization witness can justify the cover. Mere temporal proximity of separate observations does not.

6. Matching

Require:

- every row has the same session;
- every row decodes under the declared schema;
- entity keys are unique in the assembled snapshot;
- every row satisfies $\text{last} \leq \text{cut}$;
- ordering differences are normalized if order is not semantic.

7. Global sections

When matching holds, assemble the snapshot from the metadata and keyed entity rows. Under key uniqueness, the assembly is unique up to the declared ordering equivalence.

8. Current implementation evidence

At the inspected repository snapshot, Apply writes entity versions and advances the session snapshot ordinal in one SQLite transaction. Historical `Snapshot(asOf>0)` selects the latest entity version at or before the chosen cut. The current-snapshot path reads the cursor and current rows through separate store operations rather than returning an explicit shared read-transaction witness. The audit should therefore mark atomic read consistency as an evidence obligation instead of silently assuming it from the Snapshot type.

9. Linear diagnostic

A useful residual is

$$r_x = \max(0, x.\text{LastEventOrdinal} - S.\text{SnapshotOrdinal}).$$

All residuals zero are equivalent to this soundness inequality. They say nothing about missing rows or replay completeness, so higher machinery is unnecessary for the basic check.

10. Engineering actions

- retain the direct runtime invariant check;
- add historical asOf restriction tests;
- make current-snapshot isolation explicit through a read transaction or documented serialization proof;

- carry provenance in audit traces;
- keep event frontier, materialized cut, and projection cursor semantically distinct.

The result illustrates the method's intended discipline: use presheaf/sheaf language to expose contexts, restrictions, and evidence, but use a direct inequality for the invariant once the global section has been assembled.

Appendix E - A Sixteen-Week Study Program

This schedule assumes five to seven focused hours per week. Compress or expand it, but preserve the sequence: concrete diagrams, universal properties, functorial laws, context-indexed data, gluing, local logic, then cohomology.

Each week has four outputs:

1. a one-page concept note written without looking at the text;
2. one hand-drawn diagram;
3. at least two completed exercises;
4. one executable artifact: test, finite model, trace, or matrix calculation.

Week 1 - Re-enter Goldblatt at limits

Custom text: Chapters 1 and 2.

Goldblatt companion: Finish the portions of Chapter 3 leading through products, equalizers, pullbacks, and the general definition of limit.

Mathematical target: State product, equalizer, and pullback entirely through arrows and unique factorization.

SessionStream target: Draw the incremental-projection/replay square and one snapshot-boundary pullback.

Exercises: E1.2, E1.3, E2.2, E2.4.

Executable artifact: A property test for one commuting square.

Week 2 - Colimits and coherent execution cuts

Custom text: Chapters 3 and 4.

Goldblatt companion: Complete the remainder of Chapter 3 relevant to colimits, completeness, and exponentiation. Do not force every dual notion into an engineering use case.

Mathematical target: Recognize diagram shapes before naming their limits or colimits.

SessionStream target: Write separate invariants for event frontier, materialized cut, snapshot soundness, completeness, and freshness.

Exercises: E3.3, E3.5, E4.2, E4.4, E4.6.

Executable artifact: A small state-machine model of snapshot-before-live.

Week 3 - Functorial replay

Custom text: Chapter 5.

Goldblatt companion: Read the introductory parts of Chapter 9 on functors.

Mathematical target: Translate identity and composition preservation into reducer laws.

SessionStream target: Decide whether timeline projection is chunking invariant under errors, tombstones, and duplicate deliveries.

Exercises: E5.1-E5.4.

Executable artifact: Random event partition tests.

Week 4 - Natural transformations and migration

Custom text: Chapter 6.

Goldblatt companion: Continue Chapter 9 through natural transformations and functor categories.

Mathematical target: Derive a naturality square from the types without memorizing its orientation.

SessionStream target: Specify one schema or timeline migration that must commute with temporal restriction.

Exercises: E6.1, E6.2, E6.5, E6.6.

Executable artifact: A two-path migration test over generated cut pairs.

Week 5 - Adjunctions as universal software structure

Custom text: Chapter 7.

Goldblatt companion: Read selected introductory portions of Chapter 15 only after completing the free-monoid example.

Mathematical target: Understand an adjunction through a natural hom-set bijection, unit, counit, and one triangle identity.

SessionStream target: Identify one hidden coordinate that prevents an event-only reducer model.

Exercises: E7.1-E7.3, E7.5.

Executable artifact: A free event-word evaluator with law tests.

Week 6 - Build the context category

Custom text: Chapter 8.

Goldblatt companion: No new chapter required. Revisit arrows, opposite categories, and pullbacks as needed.

Mathematical target: Construct a finite poset/category of information contexts.

SessionStream target: Use time, observer, and schema coordinates; make every restriction arrow explicit.

Exercises: E8.1, E8.2, E8.5, E8.7.

Executable artifact: A machine-readable context graph.

Week 7 - Presheaves and parameter fibres

Custom text: Chapter 9.

Goldblatt companion: Read the presheaf/stack setup in Chapter 14 only far enough to compare terminology.

Mathematical target: Prove identity and composition laws for restrictions.

SessionStream target: Define a presheaf of trace facts or snapshot evidence and a fibre-based parameter-sufficiency example.

Exercises: E9.1, E9.3, E9.5, E9.7.

Executable artifact: Finite context/restriction library with property tests.

Week 8 - Bundles, fibres, germs, and sections

Custom text: Chapter 10.

Goldblatt companion: Read the bundles and sheaves discussion in Chapter 4.5.

Mathematical target: Move fluently among indexed fibres, total-space projection, and local sections.

SessionStream target: Visualize all valid local snapshot candidates above session-cut contexts.

Exercises: E10.1-E10.4, E10.6.

Executable artifact: A visualization or textual explorer of fibres and restrictions.

Week 9 - The sheaf condition through hydration

Custom text: Chapter 11.

Goldblatt companion: Begin the sheaf portions of Chapter 14.

Mathematical target: Separate matching, existence, and uniqueness.

SessionStream target: Write a formal snapshot-plus-suffix gluing proof and enumerate every assumption it uses.

Exercises: E11.1, E11.3-E11.6.

Executable artifact: Finite matching-family/global-completion checker.

Week 10 - Sites and operational coverage

Custom text: Chapter 12.

Goldblatt companion: Continue Chapter 14 through sites and Grothendieck topoi.

Mathematical target: Verify identity, pullback stability, and transitivity for one pretopology.

SessionStream target: Distinguish “components that communicate” from “local observations that jointly cover a global claim.”

Exercises: E12.1-E12.3, E12.7, E12.8.

Executable artifact: Coverage declarations with validation checks.

Week 11 - Presheaf topoi and internal structure

Custom text: Chapter 13.

Goldblatt companion: Read the elementary-topos portions of Chapter 4, then revisit products, equalizers, and exponentials in the presheaf setting.

Mathematical target: See that limits in a presheaf category are computed pointwise and understand why exponentials require restriction coherence.

SessionStream target: Build the equalizer presheaf where incremental and replay projections agree.

Exercises: E13.1-E13.5.

Executable artifact: Finite presheaf products and equalizers.

Week 12 - Sieves and context-sensitive truth

Custom text: Chapter 14.

Goldblatt companion: Read Chapter 10's treatment of presheaf truth and classifiers.

Mathematical target: Compute sieves and a characteristic sieve by hand.

SessionStream target: Replace one Boolean invariant report with a maximal supporting-context report.

Exercises: E14.1-E14.5, E14.7.

Executable artifact: Sieve calculator on a finite context poset.

Week 13 - Kripke-Joyal semantics and evidence

Custom text: Chapter 15.

Goldblatt companion: Read the local-truth/Kripke-Joyal portions of Chapter 14.

Mathematical target: Explain why local truth is monotone under refinement and why excluded middle can fail.

SessionStream target: Formalize heartbeat suspicion and snapshot evidence without overclaiming global facts.

Exercises: E15.1-E15.6.

Executable artifact: Evidence objects whose support is a sieve.

Week 14 - Change of context and geometric morphisms

Custom text: Chapter 16.

Goldblatt companion: Read selected parts of Chapters 15 and 16 on adjunctions and geometric morphisms.

Mathematical target: Identify inverse image, direct image, adjunction, and finite-limit preservation.

SessionStream target: Review one transport or schema boundary as a change of context.

Exercises: E16.1, E16.3-E16.7.

Executable artifact: A boundary review with explicit pullback tests.

Week 15 - From overlap shape to cohomology

Custom text: Chapters 17 and 18.

Goldblatt companion: No direct coverage is required; use Goldblatt's categorical discipline rather than expecting a matching chapter.

Mathematical target: Compute nerves, coboundary matrices, H^0 , and H^1 for paths, cycles, and filled triangles.

SessionStream target: Declare one reliable overlap complex and one coefficient system before calculating anything.

Exercises: E17.1-E17.5, E18.1-E18.5, E18.7.

Executable artifact: Small linear-algebra cohomology engine.

Week 16 - SessionStream research lab

Custom text: Chapters 19 and 20, then Appendix D.38.

Goldblatt companion: Revisit whichever categorical definitions your capstone uses; do not add new theory merely to make the project sound sophisticated.

Mathematical target: Distinguish a global-section failure, a missing coordinate, a type mismatch, and a genuine nonzero cohomology class.

SessionStream target: Complete one full audit and one trace-derived cursor-complex experiment.

Exercises: E19.1-E19.8 and at least four capstone exercises.

Executable artifact: A report that includes contexts, restrictions, covers, evidence provenance, completion classification, and any cohomology representatives.

E.17 A sustainable review cycle

At the end of each four-week block:

1. Re-derive all definitions from diagrams.
2. Re-run executable artifacts against the latest repository version.
3. Mark every claim as exact model, proposed formalization, or analogy.
4. Record one example where the abstraction clarified a design decision.
5. Record one example where a simpler method was better.

This final item prevents abstraction from becoming a goal in itself.

Appendix F - Minimum Working Glossary

Adjunction. A pair of functors related by a natural bijection between hom-sets. It formalizes one construction being free or universal relative to another.

Amalgamation. A section over a larger region whose restrictions are a given matching family of local sections.

Arrow. A morphism admitted by a category. Its meaning depends on the declared category and equality.

Base category. The category of contexts over which a presheaf or sheaf varies.

Boundary object. A shared context to which two local regions restrict. In hydration, it may contain session, cut, and schema semantics.

Category. Objects and arrows with associative composition and identities.

Cech complex. A cochain complex assembled from sections on cover members and their multiple overlaps. This text uses elementary finite versions.

Characteristic map. A map from an object to a subobject classifier that records membership or contextual truth.

Cocone. A family of arrows from diagram objects into one apex, compatible with diagram arrows.

Cocycle. A cochain sent to zero by the next coboundary.

Coboundary. A cochain arising by applying the previous coboundary map; in degree one, an overlap difference explained by local vertex coordinates.

Coefficient system. The groups, vector spaces, or modules assigned to cells or contexts for a cohomology calculation, together with restriction maps.

Colimit. A universal cocone. It assembles pieces while imposing the diagram's declared identifications.

Commuting diagram. A diagram in which every two directed paths with common endpoints denote equal arrows.

Compatible family. Another name for a matching family: local sections agree after restriction to all overlaps.

Completion fibre. All global states restricting to one visible local state or API parameter assignment.

Cone. A family of arrows from one apex into diagram objects, compatible with diagram arrows.

Context. An explicitly modeled scope of information, observation, authorization, time, version, or subsystem visibility.

Contravariant. Reversing arrow direction. A context inclusion $V \rightarrow U$ induces a restriction $F(U) \rightarrow F(V)$.

Coproduct. A universal tagged choice receiving arrows from each summand.

Cover. A family of arrows declared sufficient to describe a target context for local-to-global reasoning.

Direct image. The right-adjoint side of a geometric morphism, conventionally written f_* .

Equalizer. The universal subobject on which two parallel arrows agree.

Epic. An arrow cancellable on the right.

Etale-space intuition. A way to visualize a sheaf as local values lying above base points, with sections selecting locally continuous values.

Exponential. An internal function object B^A satisfying the evaluation/currying universal property.

Fibre. The values lying over one base point under a projection, or the preimage of one local observation under restriction.

Forcing. The relation $U \text{ forces } \phi$ saying a formula is supported at context U according to local semantic clauses.

Functor. A translation preserving identities and composition.

Geometric morphism. An adjoint pair $f^* \dashv f_*$ between topoi whose inverse-image functor f^* preserves finite limits.

Germ. An equivalence class of local sections that agree after sufficient restriction around a point/context.

Global section. A section over the chosen whole context; often a coherent global state or execution witness.

Grothendieck topology. A specification of covering sieves or covering families satisfying stability and transitivity axioms.

Heyting algebra. The algebraic structure of intuitionistic propositions, with implication but not necessarily classical complements.

Hom-set. The collection of arrows from one object to another.

Identity arrow. The neutral arrow $1_A: A \rightarrow A$.

Image. Values attained by a map; in cohomology, $\text{im } \delta$ is the subspace of coboundaries.

Internal logic. Logic interpreted using the categorical structure inside a topos rather than externally in ordinary set theory.

Inverse image. The left-adjoint, left-exact side of a geometric morphism, conventionally written f^* .

Isomorphism. An arrow with a two-sided inverse.

Kan extension. A universal way to extend a functor along another functor. Left and right Kan extensions answer different free versus compatible extension questions.

Kernel. Values mapped to zero by a linear map; in cohomology, cocycles form a kernel.

Kripke-Joyal semantics. Rules for interpreting logical formulas at contexts in a sheaf or presheaf topos.

Left exact. Preserving finite limits.

Limit. A universal cone: the canonical coherent joint observation of a diagram.

Local section. A section defined only over a selected context or region.

Matching family. Local sections whose restrictions agree on every overlap.

Monic. An arrow cancellable on the left.

Natural transformation. A coherent family of arrows between functors, satisfying a naturality square for every source arrow.

Nerve. A simplicial object or complex recording which finite families of cover members overlap or possess accepted joint witnesses.

Object. A thing in a category, characterized categorically by its arrows rather than internal fields alone.

Opposite category. The category obtained by reversing every arrow.

Presheaf. A contravariant functor from a base category to sets or another coefficient category.

Pretopology. A covering-family presentation satisfying identity, pullback stability, and transitivity axioms.

Projection cursor. In this text, a semantic frontier of one projector. Its exact meaning must distinguish successful materialization from handled/skipped progress.

Pullback. A universal compatible pair over a shared target.

Pushout. A universal amalgamation of two arrows from a shared source.

Reflection. A left adjoint to the inclusion of a full subcategory; a universal repair into that subcategory.

Representable presheaf. A presheaf of the form $\text{Hom}(-, U)$. Yoneda says its maps into F correspond exactly to elements of $F(U)$.

Restriction map. The map that transports a section from a larger context to a smaller one by forgetting, truncating, or projecting.

Section. A local value over a region; in a bundle picture, a right inverse to the projection on its domain.

Separated presheaf. A presheaf in which matching families have at most one amalgamation.

Sheaf. A presheaf in which every matching family over every declared cover has exactly one amalgamation.

Sheafification. The universal map from a presheaf to a sheaf, identifying locally indistinguishable data and adding required local amalgamations.

Sieve. A family of arrows into an object closed under precomposition.

Site. A category equipped with a Grothendieck topology.

Stalk. The colimit of local sections around a point; informally, all sufficiently local germs at that point.

Subobject. An isomorphism class of monomorphisms into an object; often a context-stable predicate.

Subobject classifier. An object Ω classifying subobjects by characteristic maps.

Topos. A category with set-like structural features, such as finite limits, exponentials, and a subobject classifier; presheaf and sheaf categories are central examples.

Universal property. A definition by unique factorization through a canonical object or arrow.

Yoneda lemma. The principle that an object is fully characterized by maps into or out of it; formally, natural maps from a representable into F correspond to elements of F at the representing object.

Appendix G - Source Map, Scope, and Provenance

G.1 Attached book used as the pedagogical source

The attached EPUB is Robert Goldblatt's *Topoi: The Categorical Analysis of Logic*. This custom text uses the following parts as its pedagogical and terminological spine:

- **Preface and opening chapters:** motivation for replacing element-first descriptions with arrow-based analysis, and for approaching abstraction through concrete mathematical examples;
- **Chapter 3:** monic, epic, and invertible arrows; initial and terminal objects; products and coproducts; equalizers; pullbacks and pushouts; limits and colimits; exponentiation;
- **Chapter 4:** elementary topoi, subobjects and classifiers, bundles, and the entry into sheaf ideas;
- **Chapter 9:** functors, natural transformations, and functor categories;
- **Chapter 10:** truth in presheaf settings and subobject-classifier ideas;
- **Chapter 14:** context-indexed structures, sheaves, sites, Grothendieck topoi, and local/Kripke-Joyal truth;
- **Chapter 15:** adjunctions;
- **Chapter 16:** geometric morphisms and their logical role.

The custom sequence is intentionally different. It begins where the reader currently is - the limits portion of Chapter 3 - and postpones most general topos logic until SessionStream has supplied a concrete base category, presheaves, covers, and a gluing problem.

No chapter here attempts to reproduce Goldblatt chapter by chapter. Material not needed for the SessionStream route has been omitted. Conversely, the nerves and cohomology chapters extend beyond the main scope of Goldblatt's book.

G.2 Pedagogical features adapted from Goldblatt

The following teaching patterns were used throughout:

1. introduce a construction through a concrete example before stating the general definition;
2. replace internal/element language with arrows and commuting diagrams once the example is understood;
3. characterize constructions by unique factorization;
4. use duality to organize related concepts without pretending every dual has equal engineering value;
5. move from sets and posets to general categories gradually;
6. revisit the same construction in new categories rather than treating definitions as isolated vocabulary;
7. include exercises that alternate definition, proof, counterexample, and application.

The prose, examples, diagrams, SessionStream models, exercises, hints, and solutions in this textbook are newly written for this study project.

G.3 SessionStream repository basis

The code study is tied to:

```
Repository: go-go-golems/sessionstream
Branch:     main
Commit:     7d6cddb3864b91cf4df45ef01931298493a4208f
Inspected:  15 August 2026
```

The principal source paths were:

Path	Role in this textbook
README.md	Overall command-event-projection-hydration architecture and snapshot-before-live contract
pkg/sessionstream/types.go	SessionId, Command, Event, and Session
pkg/sessionstream/ordinals.go	Per-session ordinal assignment and stream-ID derivation
pkg/sessionstream/projection.go	UI/timeline projection interfaces, timeline entities, timeline views
pkg/sessionstream/hydration.go	Hydration, event, projection-cursor, and snapshot interfaces
pkg/sessionstream/hub.go	Dispatch, publication, projection, store application, cursor advance, fanout, replay, and error policies
pkg/sessionstream/hydration/sqlite/store.go	Transactional apply, entity versions, current/historical snapshots, and cursors
pkg/sessionstream/transport/ws/server.go	Hydrating/live subscription states, buffering, snapshot delivery, filtering by cut, and transition to live
.../heartbeat/machine.go	Pure heartbeat state machine and suspicion semantics
pkg/doc/topics/01-user-guide.md	User-facing architectural vocabulary and reconnect semantics

Descriptions of current behavior should be rechecked against the repository when the code changes. The mathematical frameworks remain useful, but the objects and equations may need revision.

G.4 What is exact and what is proposed

Several constructions in the book can be made exact with little ambiguity:

- event assignments on finite ordinal intervals form a presheaf and, for ordinary interval-union covers, a sheaf;
- parameter assignments with field projection form a presheaf;
- finite context posets produce presheaf categories;
- pointwise products and equalizers in presheaf categories are literal categorical constructions;
- finite nerves and cochain matrices can be computed exactly once overlaps and coefficients are declared.

Other constructions are proposed formalizations:

- the particular context category combining sessions, cuts, observers, and schema versions;
- the coverage declaring snapshot, boundary, and suffix sufficient for client reconstruction;
- the exact observational equivalence on client states;
- the cellular coefficient system for SessionStream cursor comparisons;
- the interpretation of specific transactions or tests as higher-dimensional witnesses.

These are useful only for the engineering question they were designed to answer. A different question may require a different category, site, or coefficient system.

G.5 Deliberate limits of this textbook

This is not:

- a formal proof that the SessionStream repository is correct;
- a replacement for database isolation analysis, distributed-systems semantics, model checking, or ordinary property testing;
- a complete course in elementary topos theory;
- a complete course in sheaf cohomology;
- a claim that all software architecture is naturally topological;
- a claim that nonzero cohomology is synonymous with a bug.

It is a bridge that makes the mathematical definitions operational enough to test against one real codebase.

G.6 Suggested later reading

The following are useful after completing the custom text. They are suggestions, not source dependencies for the chapters above.

- Emily Riehl, *Category Theory in Context* - a modern categorical reference with careful treatment of universal properties, adjunctions, and limits.
- Saunders Mac Lane and Ieke Moerdijk, *Sheaves in Geometry and Logic* - a deeper route from sheaves to topos theory and logic.
- Brendan Fong and David Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality* - examples of category-theoretic modeling closer to systems and compositionality.
- Justin Curry, *Sheaves, Cosheaves and Applications* - a route toward cellular sheaves and applied local-to-global methods.
- Robert Ghrist, *Elementary Applied Topology* - computational and geometric intuition for complexes, homology, and applied topology.

Read these only after you can state the SessionStream versions of section, restriction, cover, matching family, amalgamation, sieve, and cocycle without referring to the glossary.

G.7 Final checklist for future revisions

When SessionStream changes, revise this book by answering:

1. Did the canonical event identity or ordinal semantics change?
2. Did projection error policy or cursor meaning change?
3. Did store transactions change the accepted covers or higher witnesses?
4. Did snapshot reads gain an explicit isolation boundary?
5. Did WebSocket replay semantics change beyond snapshot-before-live?
6. Did observer reliability or ordering guarantees change?
7. Did schema migrations add new context arrows?
8. Which diagrams now commute that previously did not, and vice versa?
9. Which exercises should become regression tests?
10. Which proposed formalizations should be rejected rather than preserved?

A mathematically honest custom textbook should evolve with its case study.